CertyIQ

CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

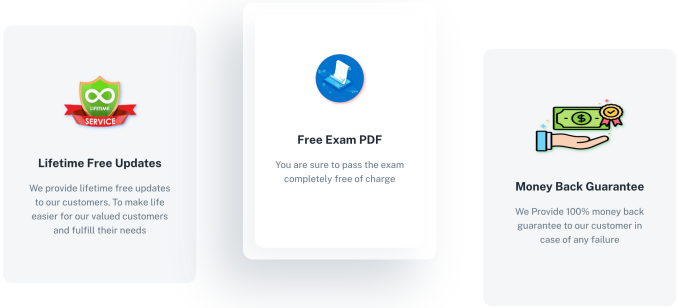
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Microsoft

(DP-800)

Developing AI-Enabled Database Solutions

Total: **147 Questions**

Link: https://certyiq.com/papers/microsoft/dp-800

Question: 1 CertyIQ

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to meet the development requirements for the FeedbackJson column.
How should you complete the Transact-SQL query? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
```

[▼] ,

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

```
ORDER BY
```

[▼]

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3

```
EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3

JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText

JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

SimilarityScore
```

```
DESC;
```

Answer:

Answer Area

```
SELECT
  f.FeedbackId,
  f.VehicleId,
  [ ],
  EDIT_DISTANCE_SIMILARITY(
    JSON_VALUE(f.FeedbackJson, '$.text'),
    @KnownIssueDescription
  ) AS SimilarityScore
FROM
  dbo.CustomerFeedback f
WHERE
  [ ]
ORDER BY
  [ ]
DESC;
```

- CONTAINS(FeedbackJson, @Keyword)
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.details.comment'), @Keyword) < 3
- EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3
- JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription) AS FeedbackText
- JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText
- SimilarityScore

Explanation:

Box 1 (SELECT clause): JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText

Extracts the raw string from the FeedbackJson object to provide the source text needed for the EDIT_DISTANCE_SIMILARITY function immediately following it.

Box 2 (WHERE clause): EDIT_DISTANCE(JSON_VALUE(f.FeedbackJson, '$.text'), @Keyword) < 3

Filters records to only include rows where the edit distance (Levenshtein distance) between the text field and the @Keyword is strictly less than 3.

Box 3 (ORDER BY clause): SimilarityScore

Reuses the column alias defined in the SELECT block (AS SimilarityScore) to sort the output in descending order (DESC).

Why the other answer are incorrect:

CONTAINS(FeedbackJson, @Keyword): Invalid output type. It returns a boolean truth value (1 or 0) for full-text searches instead of a text value.

EDIT_DISTANCE(...) < 3: Syntax error. This is a logical filter condition (predicate) and cannot be named directly as a string column expression.

JSON_QUERY(f.FeedbackJson, '$.text', @KnownIssueDescription): Wrong data type. JSON_QUERY only extracts valid objects or arrays. It returns NULL for scalar text properties.

SimilarityScore: Reference error. You cannot select the alias itself before its computation expression is declared.

CONTAINS(FeedbackJson, @Keyword): Wrong path context. It checks the entire JSON string instead of isolating the necessary $.text path property.

EDIT_DISTANCE(..., '$.details.comment', ...): Property mismatch. This evaluates a completely different nested field (comment), ignoring the target $.text property specified by the query logic.

JSON_QUERY(...) / JSON_VALUE(...): Structural syntax error. These functions extract data values. A WHERE clause demands a boolean predicate (<, >, =).

SimilarityScore: Execution order failure. Standard SQL processing evaluates WHERE long before SELECT. The engine does not yet recognize the column alias.

CONTAINS(...) / EDIT_DISTANCE(...) < 3: Logic mismatch. These evaluate to true/false criteria. They do not provide a continuous numeric spectrum needed to rank similarity accurately.

JSON_QUERY(...) / JSON_VALUE(...): Sort goal failure. Ordering by the raw text characters sorts alphabetically instead of ranking items by how closely they match the keyword.

Question: 2 CertyIQ

DRAG DROP -
You have an Azure SQL database that contains a table named dbo.Orders.
You have an application that calls a stored procedure named dbo.usp_CreateOrder to insert rows into dbo.Orders.
When an insert fails, the application receives inconsistent error details.
You need to implement error handling to ensure that any failures inside the procedure abort the transaction and return a consistent error to the caller.
How should you complete the stored procedure? To answer, drag the appropriate values to the correct targets.

Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Values

- BEGIN CATCH
- IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION
- RAISERROR('CreateOrder failed', 16, 1)
- ROLLBACK TRANSACTION
- SET @OrderId = SCOPE_IDENTITY()
- THROW

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerId int,
    @Amount decimal(10,2),
    @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
        VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
        [                                        ] ;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        [                                        ] ;
        THROW;
    END CATCH
END
```

Answer:

Values

- BEGIN CATCH
- IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION
- RAISERROR('CreateOrder failed', 16, 1)
- ROLLBACK TRANSACTION
- SET @OrderId = SCOPE_IDENTITY()
- THROW

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerId int,
    @Amount decimal(10,2),
    @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders(CustomerId, Amount, CreatedAt)
        VALUES (@CustomerId, @Amount, SYSUTCDATETIME());
        [ SET @OrderId = SCOPE_IDENTITY() ]
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        [ IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION ] ;
        THROW;
    END CATCH
END
```

Explanation:

SET @OrderId = SCOPE_IDENTITY()

This function retrieves the last identity value generated in the current session and current scope. It is the safest way to get the ID of the row you just inserted.

IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION

This is a critical safety mechanism. It checks if there is an active, uncommitted transaction before attempting to roll back. If an error occurred and the transaction wasn't committed, this command reverts any partial changes, preventing "orphaned" or corrupt data.

Why the other answer are incorrect:

BEGIN CATCH

Why it is wrong: The template code in the Answer Area already has the structural BEGIN CATCH statement written statically right above the blank slot. Dragging another one inside the block would break the T-SQL block sequence, resulting in a syntax compilation failure.

RAISERROR('CreateOrder failed', 16, 1)

Why it is wrong: Modern T-SQL best practices (and Microsoft certification guidelines) favor the use of THROW over the legacy RAISERROR function inside catch blocks. THROW re-raises the exact original execution exception with its true error number, severity, and line-number metadata preserved. RAISERROR overrides this context with a custom message, making debugging much harder.

ROLLBACK TRANSACTION (Without the @@TRANCOUNT Check)

Why it is wrong: Executing a raw, un-shielded ROLLBACK TRANSACTION inside a catch block is an anti-pattern. If a runtime error (or a global setting like SET XACT_ABORT ON) automatically aborts and rolls back the active transaction before control lands in the catch block, calling rollback a second time will cause the database engine to crash with Error 3903 ("The ROLLBACK TRANSACTION request has no corresponding BEGIN TRANSACTION.").

THROW

Why it is wrong: The keyword THROW; is already statically written in the template code immediately below the second empty slot. Dragging a second one into that space would result in consecutive duplicate statements (THROW; THROW;), causing a T-SQL parsing error.

Question: 3 CertyIQ

Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
You need to disable the GitHub Copilot repository-level instructions for yourself without affecting other users.
What should you do?

A. From Visual Studio Code, modify your GitHub Copilot Chat user settings.
B. Add a --debug flag when you start the GitHub Copilot Chat extension.
C. Delete .github/copilot-instructions.md.

Answer: A

Explanation:

A. From Visual Studio Code, modify your GitHub Copilot Chat user settings.

Option A allows you to turn off the loading of repository-level custom instructions locally. In Visual Studio Code, you can navigate to your user-level editor settings (Settings > Extensions > GitHub Copilot / Copilot Chat) and uncheck or disable the configuration that automatically applies repository instruction files. Because this modification is done in your individual User Settings, it applies exclusively to your local environment and will not alter the repository or affect any other team members.

Why the other options are incorrect:

Option B (Add a --debug flag...) is incorrect. Command-line flags like --debug are used for viewing detailed execution logs and diagnostic output. They do not control the loading or parsing of feature-level functional files like repository instructions.

Option C (Delete .github/copilot-instructions.md) is incorrect. If you delete this file from your workspace and commit/push the change, it completely removes the custom instructions for the entire team. This directly violates the requirement to disable it without affecting other users.

Question: 4 CertyIQ

You have an Azure SQL database that contains the following SQL graph tables:
A NODE table named dbo.Person -
An EDGE table named dbo.Knows -
Each row in dbo.Person contains the following columns:
PersonID (int)
DisplayName (nvarchar(100))
You need to use a MATCH operator and exactly two directed Knows relationships to return the PersonID and DisplayName of people that are reachable from the person identified by an input parameter named @StartPersonId.
Which Transact-SQL query should you use?

A.
```
SELECT p2.PersonId, @StartPersonId
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.DisplayName = p2.DisplayName
  AND MATCH(p1-(k1)->p2-(k2)->p3);
```

B.
```
SELECT p3.PersonId, p3.DisplayName FROM dbo.Person AS p1
JOIN dbo.Knows AS k1 ON 1 = 1
JOIN dbo.Person AS p2 ON 1 = 1
JOIN dbo.Knows AS k2 ON 1 = 1
JOIN dbo.Person AS p3 ON 1 = 1
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p3<-(k2)-p2<-(k1)-p1);
```

C.
```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p1-(k1)->p2)   AND MATCH(p2-(k2)->p3);
```

D.

```
SELECT p3.PersonId, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2, dbo.Knows AS k2, dbo.Person AS p3
WHERE p1.PersonId = @StartPersonId
  AND MATCH(p1-(k1)->p2-(k2)->p3);
```

Answer: D

Explanation:

FROM dbo.Person AS p1, dbo.Knows AS k1, ...: This declares the tables involved. In graph terms, Person are the Nodes and Knows are the Edges (relationships) connecting them.

WHERE p1.PersonId = @StartPersonId: This sets the starting point of the search to a specific user provided by the variable @StartPersonId.

AND MATCH(p1-(k1)->p2-(k2)->p3): This is the core graph pattern. It instructs the database to traverse the graph starting at p1, following a "knows" edge (k1) to a person (p2), and then another "knows" edge (k2) to a third person (p3).

Why the other Options are Incorrect:

Option A: This logic attempts to filter columns using a redundant tautology predicate (WHERE p1.DisplayName = p1.DisplayName). Because it completely omits the required input filter constraint (@StartPersonId), the engine runs blind without an indexed anchor point.

Option B: This reverses the physical execution trajectory inside the graph pattern topology. Writing the relationship blocks backward (p3-(k2)->p2-(k1)->p1) implies that the target entities know the root variable, rather than retrieving the accounts that are reachable from the input parameter user.

Option C: This splits the traversal layout into two disconnected, isolated MATCH() constraints separated by an AND operator. SQL Graph does not support evaluating disparate independent match statements across shared variable streams in this format; chaining multi-hop structures directly inside a single single match block is syntactically mandatory.

Question: 5 CertyIQ

You have a SQL database in Microsoft Fabric that contains a column named Payload. Payload stores customer data in JSON documents that have the following format.

```
{
"date": "2020-01-25",
"customer_email": "user@contoso.com",
...
}
```

Data analysis shows that some customers have subaddressing in their email address, for example, .
You need to return a normalized email value that removes the subaddressing, for example, user1 must be normalized to .
Which Transact-SQL expression should you use?

A. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*$', '')
B. REGEXP_SUBSTR(JSON_VALUE(Payload, '$.customer_email'), '^[^+]+@.*$=')
C. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@')
D. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*', '')

Answer: C

Explanation:

C. REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@')

JSON_VALUE(Payload, '$.customer_email'): This correctly extracts the email string from your JSON object.

'\+.*@' (The Regex Pattern):

\+ matches the literal plus sign (escaped because + is a special character in regex).

.* matches any characters following the plus sign.

@ stops the match at the domain separator.

'@' (The Replacement): By replacing the entire +... segment with @, you effectively "stitch" the username part directly to the domain part, resulting in the normalized email.

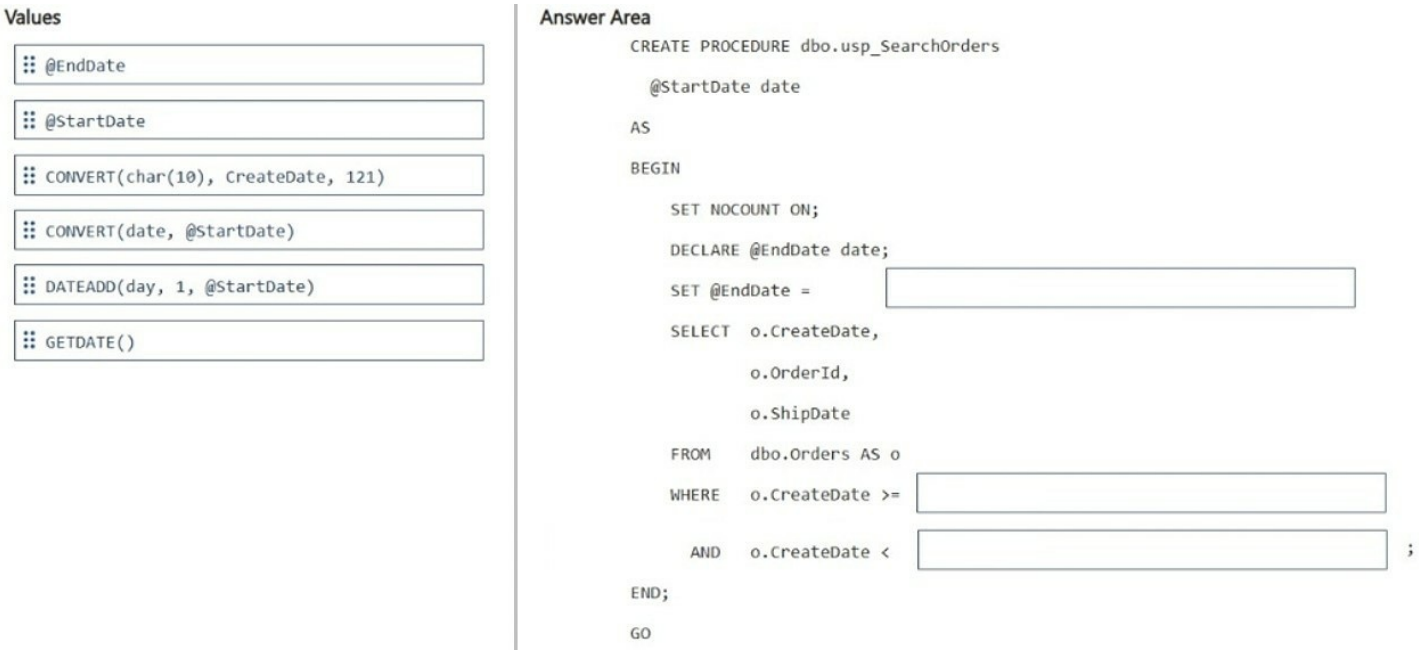
Comparison with Other Options

A and D: These use '\+.*$' or '\+.*'. If you replace the + and everything after it with an empty string, you would end up with user1@contoso.com only if the regex is very carefully constructed to stop before the @. If it removes the @ and the domain, you would lose the essential part of the email address.

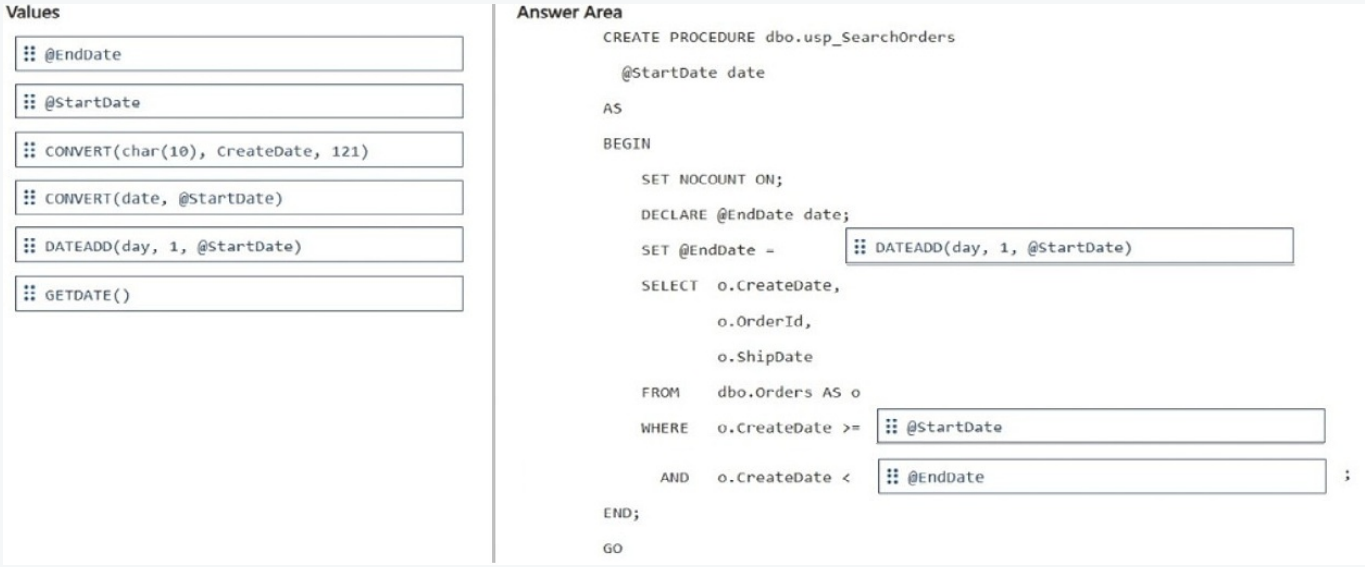
B: REGEXP_SUBSTR is used to extract a matching string rather than replace a portion of it. While you could technically extract parts and concatenate them, it is far less efficient than using a simple replacement.

Question: 6 CertyIQ

DRAG DROP -
You have an Azure SQL database that contains a table named dbo.Orders. dbo.Orders contains a column named CreateDate that stores order creation dates.
You need to create a stored procedure that filters Orders by CreateDate for a single calendar day. The solution must be SARGable.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (SET @EndDate =): DATEADD(day, 1, @StartDate)

Box 2 (o.CreateDate >=): @StartDate

Box 3 (AND o.CreateDate <): @EndDate

Calculating @EndDateCode: SET @EndDate = **DATEADD(day, 1, @StartDate**)Purpose: This adds exactly one day to the input @StartDate. For example, if @StartDate is 2026-06-21, @EndDate becomes 2026-06-22 (midnight of the next day).

Filtering with Range Comparisons (>= and <)Code: WHERE o.CreateDate >= @StartDate AND o.CreateDate < **@EndDateP**urpose: This filters data from the exact start of **@StartDate** (e.g., 2026-06-21 00:00:00.000) up to, but strictly excluding, the start of the next day (e.g., 2026-06-22 00:00:00.000). This ensures that every single order placed on 2026-06-21 is captured, regardless of what time it was created.

Why the other answer are incorrect:

CONVERT(char(10), CreateDate, 121)

Why it is wrong: This option strips time data by converting a date column into a formatted text string (e.g., '2026-06-22'). Applying conversion functions directly to table columns (the left-hand side of a filter) breaks SARGability. This completely destroys SQL Server's ability to navigate B-Tree index keys via an optimized Index Seek, forcing a slow, resource-heavy Index Scan or Table Scan over the entire table.

CONVERT(date, @StartDate)

Why it is wrong: The input parameter @StartDate is already explicitly declared as a date data type at the top of the stored procedure parameters list (@StartDate date). Attempting to cast or convert an object into the exact same data type it already possesses is completely redundant and adds zero functional value to the script.

GETDATE()

Why it is wrong: GETDATE() returns the current system timestamp of the underlying server machine. Because this stored procedure is built to run historical reporting lookups by taking a custom user input argument (@StartDate), forcing a hardcoded current system time check would completely override the user's intended search parameters.

@StartDate / @EndDate

Why it is wrong: Swapping these placement boxes around (such as writing o.CreateDate >= @EndDate or o.CreateDate < @StartDate) disrupts the chronological sequence of the comparison logic. This creates an impossible mathematical constraint (checking if a date is simultaneously in the future of the end date and the past of the start date), resulting in an empty query result set.

Question: 7 CertyIQ

You have an Azure SQL database.
You need to create a scalar user-defined function (UDF) that returns the number of whole years between an input parameter named @OrderDate and the current date/time as a single positive integer. The function must be created in Azure SQL Database.
You write the following code.

```
01 CREATE FUNCTION dbo.ufnYearsSinceOrder (@OrderDate datetime2)
02 RETURNS int
03 AS
04 BEGIN
05
06 END
```

What should you insert at line 05?

A. RETURN DATEDIFF(year, GETDATE(), @OrderDate);
B. DATEDIFF(month, @orderdate, GETDATE()) / 12
C. DATEPART(year, GETDATE()) - DATEPART(year, @orderdate)
D. RETURN DATEDIFF(year, @OrderDate, GETDATE());

Answer: D

Explanation:

D. RETURN DATEDIFF(year, @OrderDate, GETDATE()) .

DATEDIFF Syntax: The Microsoft Transact-SQL DATEDIFF function uses the syntax DATEDIFF(datepart, startdate, enddate). It measures the boundaries crossed from the earlier date (startdate) to the later date (enddate).Correct Order: To return a positive integer representing the years elapsed up to the current date, the earlier input parameter (@OrderDate) must be the startdate and the current date/time (GETDATE()) must be the enddate.Scalar UDF Requirement: Because this line is part of a scalar User-Defined Function (UDF), it requires the RETURN keyword to output the computed value back to the caller.

Why the other choices are incorrect:

Option A reverses the order of the dates (GETDATE() as start, @OrderDate as end), which would result in a negative integer.

Option B and Option C are missing the mandatory RETURN keyword required by a T-SQL scalar user-defined function to exit and pass back the data.

Question: 8 CertyIQ

You have an Azure SQL database.
You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases/data-api-builder:latest image.
You have the following Container Apps secrets:
MSSQL_CONNECTION_STRING that maps to the SQL connection string
DAB_CONFIG_BASE64 that maps to the DAB configuration
You need to initialize the DAB configuration to read the SQL connection string.
Which command should you run?

A. dab init --database-type mssql --connection-string “secretref:DAB_CONFIG_BASE64” --host-mode Production --config dab-config.json

B. dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json

C. dab init --database-type mssql --connection-string “secretref:mssql-connection-string” --host-mode Production --config dab-config.json

D. dab init --database-type mssql --connection-string “@env(‘DAB_CONFIG_BASE64’)” --host-mode Production --config dab-config.json

Answer: B

Explanation:

B. dab init --database-type mssql --connection-string “@env(‘MSSQL_CONNECTION_STRING’)” --host-mode Production --config dab-config.json

The @env() function: Data API builder (DAB) natively supports the @env() string substitution function to securely pass configuration settings (like connection strings) into the dab-config.json file at runtime instead of hardcoding them.

Targeting the Right Secret: The environment variable mapping to your SQL database connection string is named MSSQL_CONNECTION_STRING. Therefore, you must specify @env('MSSQL_CONNECTION_STRING') to read that precise value.

Why the other choices are incorrect:

Options A & C: The secretref: syntax is a native platform feature used by Azure Container Apps or Kubernetes within YAML definitions, but it is not understood natively by the Data API builder (DAB) command-line interface (dab init).

Option D: This targets the wrong secret environment variable (DAB_CONFIG_BASE64), which contains the base64-encoded configuration file of DAB itself, rather than the database connection string.

Question: 9 CertyIQ

You have a SQL database in Microsoft Fabric that contains a nvarchar (max) column named MessageText. An ID is always contained within the first paragraph of MessageText.
You need to write a Transact-SQL query that uses REGEXP_SUBSTR to extract the ID from MessageText.
What should you include in the query?

A. Apply STRING_ESCAPE(MessageText, 'json') before calling REGEXP_SUBSTR.
B. Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.
C. Add a COLLATE Latin1_General_CS_AS clause to MessageText before calling REGEXP_SUBSTR.
D. Run TRY_CONVERT(varchar(max), MessageText) before calling REGEXP_SUBSTR.

Answer: A

Explanation:

B. Cast MessageText to nvarchar (4000) before calling REGEXP_SUBSTR.

Why this is the correct choice:

Data Type Limitations: In the T-SQL implementation of Regular Expression functions (such as REGEXP_SUBSTR), large object data types like nvarchar(max) or varchar(max) are not supported directly as the input string_expression.
Explicit Casting: Because the problem specifies that the required ID is guaranteed to reside within the first paragraph of the column, casting the nvarchar(max) data down to a standard size like nvarchar(4000) ensures compliance with the function's argument limits without losing the necessary search text.

Why the other choices are incorrect:

Option A (STRING_ESCAPE) adds control characters (like escaping slashes) to make data safe for JSON formatting. It does not alter the underlying max data type limit or help with regular expression matching.

Option C (COLLATE) changes the collation rules (such as case sensitivity or accent sensitivity), but it keeps the underlying data type as nvarchar(max), which remains incompatible with the function.

Option D (TRY_CONVERT) translates the data to varchar(max). This maintains a large-object max specifier that is still incompatible with the regular expression engine and introduces a risk of cutting off non-ASCII Unicode characters.

Question: 10 CertyIQ

You have an Azure SQL database that contains database-level Data Definition Language (DDL) triggers, including a

trigger named ddl_Audit.
You need to prevent ddl_Audit from firing during the next deployment. The trigger object must remain in place.
Which Transact-SQL statement should you use?

A. ALTER TRIGGER
B. ALTER DATABASE
C. ALTER SERVER AUDIT SPECIFICATION
D. DISABLE TRIGGER
E. ALTER DATABASE AUDIT SPECIFICATION

Answer: D

Explanation:

D. DISABLE TRIGGER

To prevent a DDL trigger from firing while keeping the object present in the database, you use the DISABLE TRIGGER statement.

Syntax: You would execute DISABLE TRIGGER ddl_Audit ON DATABASE; (since ddl_Audit is a database-level trigger).
Why others are incorrect:
ALTER TRIGGER: Used to modify the definition (the code inside) of an existing trigger, not to toggle its active state.
ALTER DATABASE: Used to modify database settings, not individual trigger states.
ALTER SERVER AUDIT SPECIFICATION / ALTER DATABASE AUDIT SPECIFICATION: These are used for auditing features in SQL Server (tracking events for compliance/security) and are separate objects from standard DDL triggers.

Question: 11 CertyIQ

Your development team uses GitHub Copilot Chat in Microsoft SQL Server Management Studio (SSMS) to generate and run Transact-SQL queries against an Azure SQL database named DB1. DB1 contains tables that store sensitive customer data.
You need to ensure that any Transact-SQL queries that run from GitHub Copilot Chat in SSMS are restricted by the same permissions as the developer's database login.
What prevents the GitHub Copilot Chat-run queries from accessing data beyond the developer's access?

A. GitHub Copilot Chat runs queries in a read-only sandbox that is isolated from production database permissions.
B. GitHub Copilot Chat runs queries by using the developer's database identity and permissions.
C. GitHub Copilot Chat filters query results on the client side to remove rows the developer is unauthorized to see.
D. GitHub Copilot Chat uses different row-level security (RLS) policies than the developer.

Answer: B

Explanation:

B. GitHub Copilot Chat runs queries by using the developer's database identity and permissions.

Why this is the correct choice:

Native Security Boundaries: When using GitHub Copilot Chat integrated within SQL Server Management Studio (SSMS), the AI tool does not possess independent server-side credentials or elevated background service permissions.
Execution Context: Any query generated and executed via Copilot's chat interface implicitly relies on the active connection established by the user. Therefore, it automatically operates under the developer's current database security context and login identity. If a user does not have permission to read a table containing sensitive customer data, any query Copilot attempts to run against that table will be blocked by the engine itself.

Why the other choices are incorrect:

Option A is incorrect. Copilot interacts directly with your connected database window or database context rather than running in an isolated read-only sandbox.

Option C is incorrect. Query security enforcement happens on the server side via the relational engine, not through client-side row filtration.

Option D is incorrect. Copilot respects whatever Row-Level Security (RLS) policies are configured for the user's active database login; it does not implement separate rules.

Question: 12 CertyIQ

You have an Azure SQL database named AdventureWorksDB that contains a table named dbo.Employee.
You have a C# Azure Functions app that uses an HTTP-triggered function with an Azure SQL input binding to query dbo.Employee.
You are adding a second function that will react to row changes in dbo.Employee and write structured logs.
You need to configure AdventureWorksDB and the app to meet the following requirements:
Changes to dbo.Employee must trigger the new function within five seconds.
Each invocation must process no more than 100 changes.
Which two database configurations should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Create an AFTER trigger on dbo.Employee for Data Manipulation Language (DML).
B. Set Sql_Trigger_MaxBatchSize to 100.
C. Enable change tracking on the dbo.Employee table.
D. Enable change tracking at the database level.
E. Set Sql_Trigger_PollingIntervalMs to 5000.
F. Enable change data capture (CDC) for dbo.Employee table changes.

Answer: CD

Explanation:

C. Enable change tracking on the dbo.Employee table.

D. Enable change tracking at the database level.

Why these are the correct choices:

To configure an event-driven architecture using the Azure SQL trigger for Azure Functions, you must enable SQL Change Tracking within the target database. Setting up change tracking requires exactly these two database-side modifications:

1. At the Database level (Option D): Turns on the native tracking engine for the database host (ALTER DATABASE AdventureWorksDB SET CHANGE_TRACKING = ON).
2. At the Table level (Option C): Instructs the engine to track explicit row modifications inside the target table (ALTER TABLE dbo.Employee ENABLE CHANGE_TRACKING).

Why the other choices are incorrect:

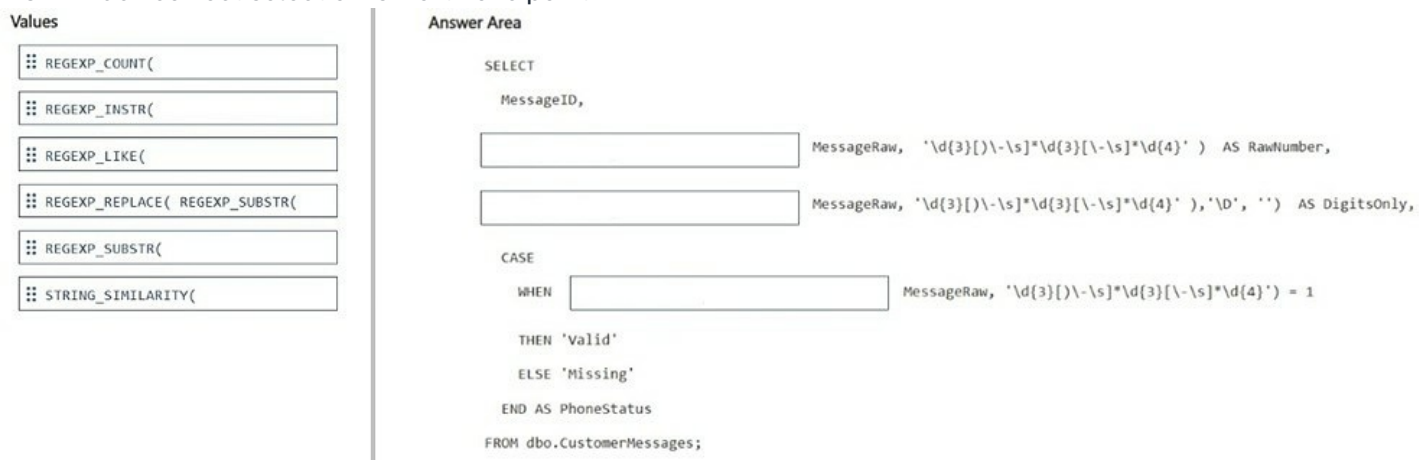
Options B and E (Sql_Trigger_MaxBatchSize and Sql_Trigger_PollingIntervalMs) are the correct configuration settings to handle the 100-batch limit and 5-second polling interval requirements, but they are Application Settings managed within the Azure Functions host configuration (local.settings.json or Azure Portal Configuration pane) — not database configurations.

Option A is incorrect. The Azure SQL Function trigger relies natively on internal SQL Server change tracking tables and query leases, not custom user-managed DML database AFTER triggers.

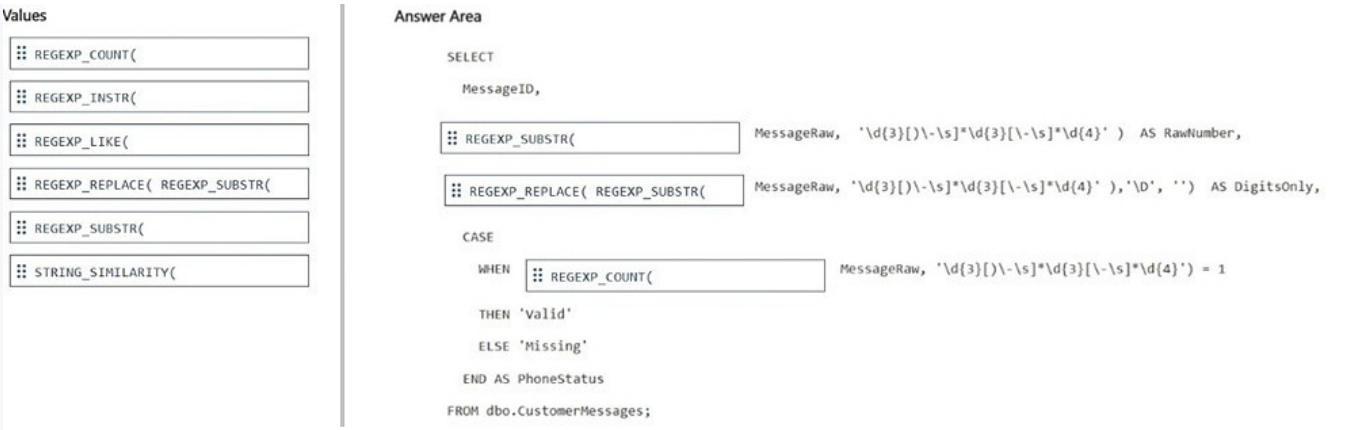
Option F is incorrect. Change Data Capture (CDC) is an alternative data-tracking feature, but it is not natively supported or utilized by the Azure Functions Azure SQL trigger extension.

Question: 13 CertyIQ

DRAG DROP -
You have a Microsoft SQL Server 2025 database that contains a table named dbo.CustomerMessages.
dbo.CustomerMessages contains two columns named MessageID (int) and MessageRaw (nvarchar(max)).
MessageRaw can contain a phone number in multiple formats, and some rows do NOT contain a phone number.
You need to write a single SELECT query that meets the following requirements:
The query must return MessageID, RawNumber, DigitsOnly, and PhoneStatus.
RawNumber must contain the first substring that matches a phone-number pattern, or NULL if no match exists.
DigitsOnly must remove all non-digit characters from RawNumber, or return NULL.
PhoneStatus must return valid when a phone number exists in MessageRaw, otherwise return Missing.
How should you complete the Transact-SQL query? To answer, drag the appropriate values to the correct targets.
Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

REGEXP_SUBSTR

The first and second empty slots in the SELECT list.

For the first slot, you are extracting a specific pattern (a phone number) from the MessageRaw column. REGEXP_SUBSTR is designed to extract a substring that matches a regular expression pattern.

For the second slot, you are using REGEXP_REPLACE to clean the string (removing everything except digits). While the second slot technically uses REGEXP_REPLACE, the function REGEXP_SUBSTR is the correct match for the extraction logic in the first slot.

REGEXP_REPLACE

T he second empty slot in the SELECT list.

You are taking the result of the MessageRaw column and replacing non-digit characters ('\\D') with an empty string (''). This is the standard use case for REGEXP_REPLACE to clean data.

REGEXP_COUNT

The empty slot in the WHEN clause.

You are checking if the phone number pattern occurs exactly once in the message. REGEXP_COUNT returns the number of times a pattern is matched in a string, making it the correct choice for this validation condition.

Why the Choices are Incorrect

REGEXP_INSTR(

Why it is wrong: This function returns the starting or ending index position (an integer index) of where a pattern begins inside a string. It cannot extract text or replace text, which makes it useless for generating RawNumber or DigitsOnly.

REGEXP_LIKE(

Why it is wrong: This function returns a boolean value (True/False) indicating whether a pattern exists within the string. While it could be used inside a WHEN clause, it cannot count the number of matches or perform text extractions.

STRING_SIMILARITY(

Why it is wrong: This function measures the fuzzy textual likeness score between two literal strings (often

used for duplicate matching or spell checks). It cannot interpret regular expression quantifiers or extract phone sub-strings.

Question: 14 CertyIQ

You have an Azure SQL database that contains a table named Rooms. Rooms was created by using the following Transact-SQL statement.

```
CREATE TABLE Rooms
(
    RoomID int PRIMARY KEY,
    Owner nvarchar(100),
    Capacity int
);
```

You discover that some records in the Rooms table contain NULL values for the Owner field.
You need to ensure that all future records have a value for the Owner field.
What should you add?

A. a foreign key
B. a check constraint
C. a nonclustered index
D. a unique constraint

Answer: B

Explanation:

B. a check constraint.

Since the table already contains NULL values, you cannot directly apply a standard NOT NULL constraint to the column without first fixing the existing rows. However, you can add a Check Constraint (such as CHECK (Owner IS NOT NULL)) with the NOCHECK option. This ensures that all future INSERT and UPDATE operations are validated to contain a value, while ignoring the existing violations.

Why the other options are incorrect:

A (Foreign Key): A foreign key enforces referential integrity between two tables but still permits NULL values unless constrained otherwise.

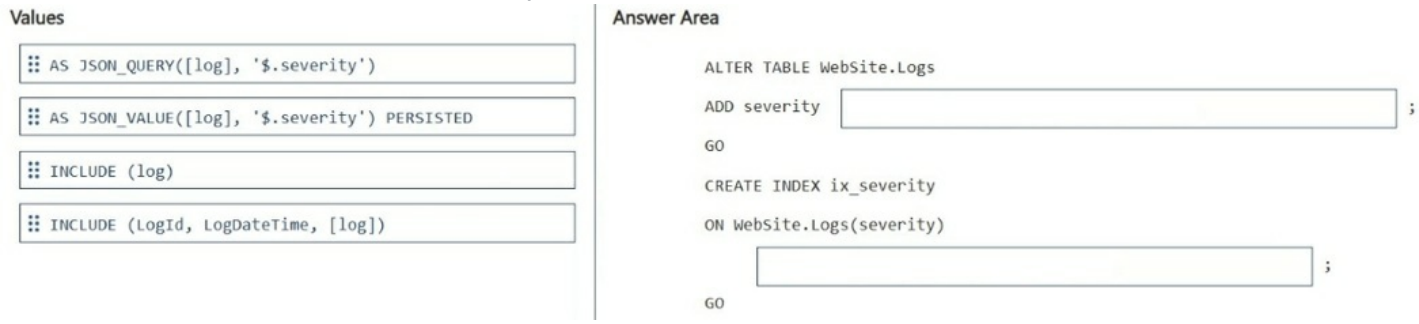
C (Nonclustered Index): Indexes optimize query performance and search speed; they do not enforce value presence or prevent NULL inputs.

D (Unique Constraint): A unique constraint ensures all non-null values in a column are distinct from one another. In many database systems (like SQL Server), it still permits one NULL value, failing to ensure a value is provided for all future records.

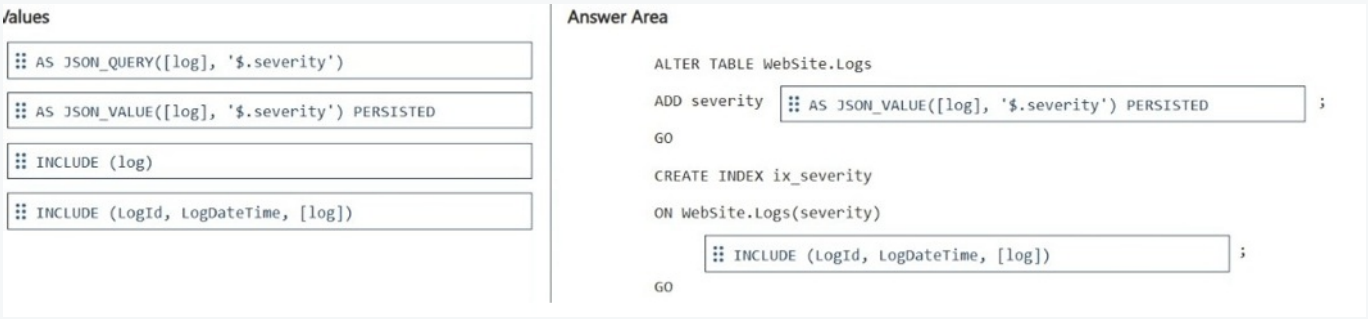
Question: 15 CertyIQ

DRAG DROP -

You have a SQL database in Microsoft Fabric that contains a table named WebSite.Logs. WebSite.Logs stores application telemetry data. WebSite.Logs contains a nvarchar (max) column named log that stores JSON documents.
You have a daily report that filters by the $.severity JSON property and returns LogId, LogDateTime, and log. The report frequently causes full table scans.
You need to modify WebSite.Logs to support efficient filtering by $.severity and avoid key lookups for the columns returned by the report.
How should you complete the Transact-SQL code to avoid full table scans? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (Computed Column Specification): **AS JSON_VALUE([log], '$.severity') PERSISTED**

Function Selection: $.severity represents a single textual/numeric property (a scalar value). JSON_VALUE is explicitly designed to retrieve scalar data, whereas JSON_QUERY is only used to extract nested objects or arrays.

The PERSISTED Keyword: By specifying PERSISTED, the database engine computes and materializes the data physically on disk whenever the row is updated. This allows an index to be created cleanly without re-parsing the JSON column during standard index scans.

Box 2 (Index Covering Strategy): **INCLUDE (LogId, LogDateTime, [log])**

Covering Index Principle: Adding the remaining columns to the INCLUDE list turns ix_severity into a covering index. When your search query filters by severity, the engine fetches the requested log details directly from the leaf nodes of the nonclustered index, preventing costly key lookups back to the base table..

Why the other answer are Incorrect:

AS JSON_QUERY([log], '$.severity')

Why it is wrong: In T-SQL, JSON_QUERY is strictly used to extract an entire nested JSON object or an array. Because a logging property like severity is a scalar value (e.g., a text string like "Error" or a number like 3), JSON_QUERY will fail to read it and return NULL. To extract a scalar value, you must use JSON_VALUE.INCLUDE.

INCLUDE (log)

Why it is wrong: When creating an index to optimize log search workloads (such as searching by severity), the index should serve as a covering index by including all columns typically returned in the query's select list (like LogId and LogDateTime). Using INCLUDE (log) only appends the raw JSON document payload to the index leaf nodes, leaving out the primary key/ID and timestamps, which would cause the engine to perform expensive Key Lookups for every matching row.

Question: 16 CertyIQ

HOTSPOT
-
Case Study
-
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study
-
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment
-
Azure Environment
-
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
-
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
-
Planned Changes
-
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements
-
Fabrikam identifies the following security requirements:
•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee

data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:
•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.
Development Requirements
-
Fabrikam identifies the following development requirements:
•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
oRead data from the procedures table without authentication.
oRead and insert data into the Transactions table once authenticated.
oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB
-
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to create a solution that meets the development requirements for retrieving the patient lists.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

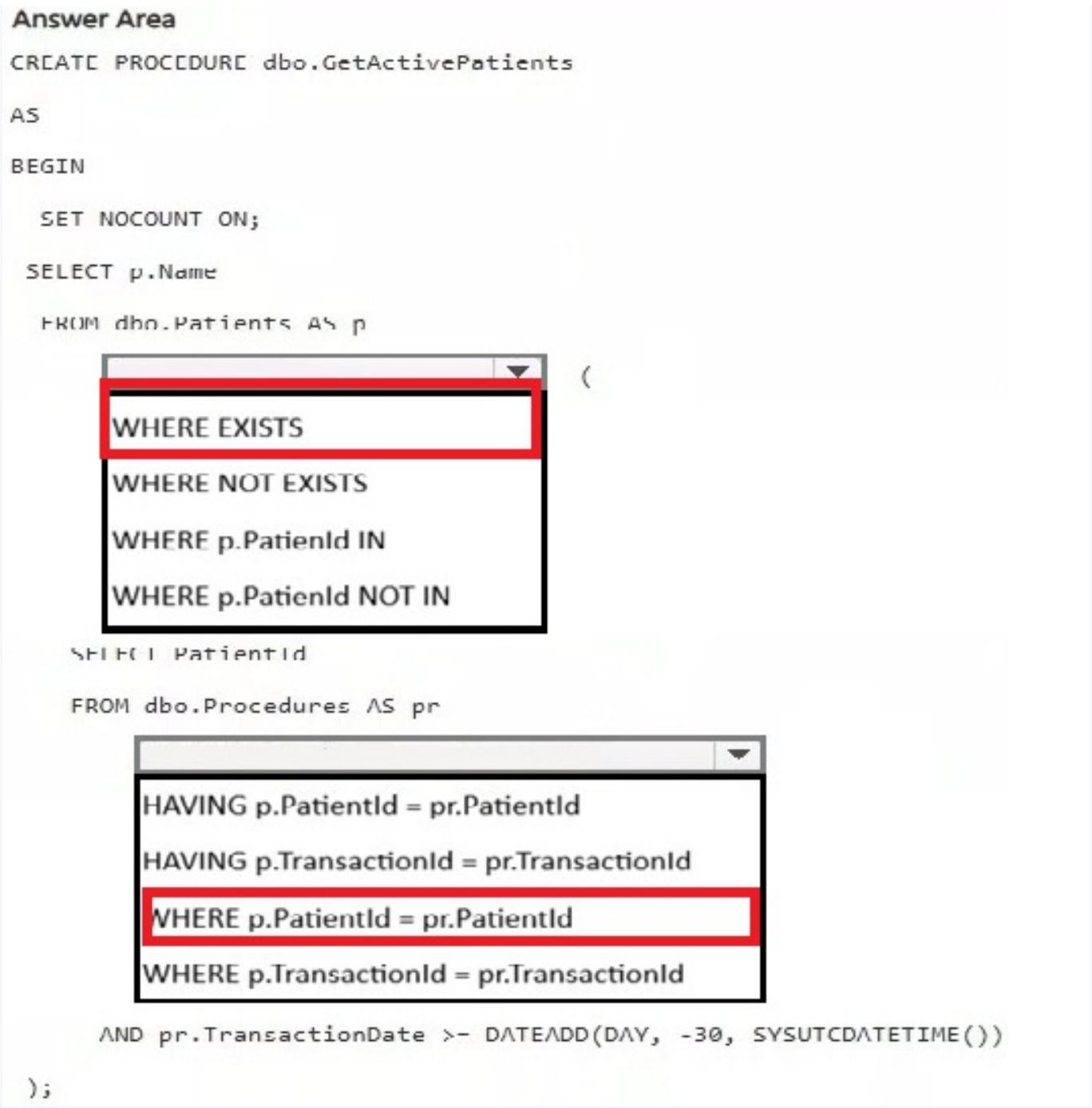
```
CREATE PROCEDURE dbo.GetActivePatients
AS
BEGIN
  SET NOCOUNT ON;
 SELECT p.Name
  FROM dbo.Patients AS p
      [                                    ] (
        SELECT PatientId
        FROM dbo.Procedures AS pr
            [                                    ]
          AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())
 );
END;
```

WHERE EXISTS
WHERE NOT EXISTS
WHERE p.PatienId IN
WHERE p.PatienId NOT IN

HAVING p.PatientId = pr.PatientId
HAVING p.TransactionId = pr.TransactionId
WHERE p.PatientId = pr.PatientId
WHERE p.TransactionId = pr.TransactionId

Answer:

Answer Area



Explanation:

First Dropdown: WHERE EXISTS
Second Dropdown: WHERE p.PatientId = pr.PatientId

Correct Code Block

sql

CREATE PROCEDURE dbo.GetActivePatientsASBEGIN SET NOCOUNT ON; SELECT p.Name FROM dbo.Patier

Use code with caution.

Question: 17 CertyIQ

You have an Azure SQL database named ToDo that contains a table named dbo.ToDo.

Your company plans to develop an Azure Functions app to run whenever the rows in dbo.ToDo change. The app will process INSERT, UPDATE, and DELETE events by using the Azure SQL trigger binding.
You need to configure ToDo to support the planned app.
What should you do?

A. Enable change tracking on ToDo and dbo.ToDo.

B. On dbo.ToDo, create a Data Manipulation Language (DML) trigger that calls the Azure Functions HTTP endpoint.

C. Enable change data capture (CDC) on ToDo and dbo.ToDo.

D. On dbo.ToDo, create a Data Definition Language (DDL) trigger that calls the Azure Functions HTTP endpoint.

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A is the Best Option: Enabling change tracking on both the database (ToDo) and the specific table (dbo.ToDo) is the most suitable approach for supporting the planned Azure Functions app. Change tracking is a feature designed to track changes made to the data in your database, which aligns perfectly with the requirement to process INSERT, UPDATE, and DELETE events. By enabling change tracking at both the database and table levels, you ensure that the Azure Functions app can effectively capture and respond to all specified change events through the Azure SQL trigger binding without interfering with the database's operational integrity.

Key Benefits of Change Tracking for this Scenario:

Non-Intrusive: Does not modify the existing database schema or interfere with DML operations.
Efficient: Provides an efficient way to track changes without the overhead of logging every operation.
Integration with Azure Services: Seamlessly integrates with Azure Functions for trigger-based operations.

Why Other Options are Less Suitable:

B. DML Trigger on dbo.ToDo calling Azure Functions HTTP endpoint

Intrusive and Potentially Inefficient: DML triggers execute for each row operation, which can lead to performance overhead, especially for high-volume changes.
Direct Dependency: Creates a direct dependency between the database table and the Azure Functions endpoint, making maintenance and scalability more complex.

C. Enable Change Data Capture (CDC) on ToDo and dbo.ToDo

Overkill for the Requirement: CDC is more suited for auditing, replication, or data warehousing scenarios due to its detailed logging nature.
More Complex to Implement and Manage: Requires more setup and maintenance compared to change tracking for simple event triggering.

D. DDL Trigger on dbo.ToDo calling Azure Functions HTTP endpoint

Incorrect Trigger Type for the Requirement: DDL triggers respond to schema changes (e.g., CREATE, ALTER, DROP), not data changes (INSERT, UPDATE, DELETE).
No Relevance to Data Change Events: Will not capture the intended INSERT, UPDATE, and DELETE operations.

References:

Microsoft Documentation: Change Tracking in Azure SQL Database
Azure Functions Azure SQL Trigger Binding

Question: 18 CertyIQ

DRAG DROP
-
You have an Azure SQL database that contains a table named dbo.Customers. dbo.Customers contains the following columns:
•CustomerId (int)(primary key)
•ProfileJson (nvarchar(max))
You have an application that returns phone numbers in a format of +000 000-000-0000. The phone numbers are stored in ProfileJson.
You need to write a query that returns:
•One row per customer
•A PhoneNUmerals column that contains only the digits
How should you complete the Transact-SQL query? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Values

- JSON_QUERY(c.ProfileJson, '$.contact.phone')
- JSON_VALUE(c.ProfileJson, '$.contact.phone')
- OPENJSON(c.ProfileJson, '$.contact.phone')
- p.PhoneRaw
- REGEXP_LIKE(p.PhoneRaw, '[^0-9]', '')
- REGEXP_REPLACE(p.PhoneRaw, '[^0-9]', '')
- REGEXP_SUBSTR(p.PhoneRaw, '[0-9]+')

Answer Area

```
WITH PhoneCTE AS
(
    SELECT DISTINCT
        c.CustomerId,
        [          ] AS PhoneRaw
    FROM dbo.Customers AS c
)
SELECT
    p.CustomerId,
    [          ] AS PhoneNUmerals
FROM PhoneCTE AS p
WHERE [          ] IS NOT NULL;
```

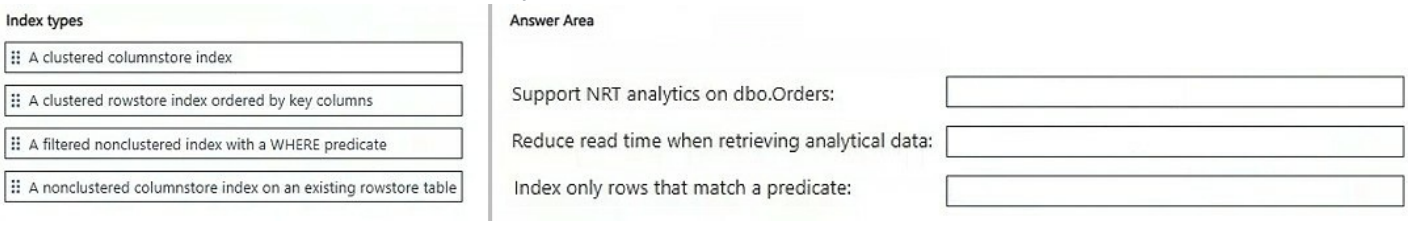
Answer:

SELECT CustomerId, JSON_VALUE(ProfileJson, '$.PhoneNumber') AS PhoneNumber,
TRANSLATE(JSON_VALUE(ProfileJson, '$.PhoneNumber'), '+ -', '') AS PhoneNumerals FROM dbo.Customers

Question: 19 CertyIQ

DRAG DROP
-
You have an Azure SQL database named SalesDB that supports an ecommerce application. SalesDB contains a table named dbo.Orders that has a clustered index on a column named OrderId.
dbo.Orders receives continuous OLTP inserts and updates during business hours.
Your analytics team runs hourly aggregate queries that scan dbo.Orders to calculate revenue trends for recent dates.
You need to improve the performance of the hourly analytics queries without significantly affecting OLTP throughput. The solution must meet the following requirements:
•Support near-real-time (NRT) analytics on the dbo.Orders table.

•Reduce read time when retrieving analytical data from dbo.Orders.
•Support indexing only rows that match a predicate, such as Active = 1.
Which type of index should you use for each requirement? To answer, drag the appropriate index types to the correct requirements. Each index type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

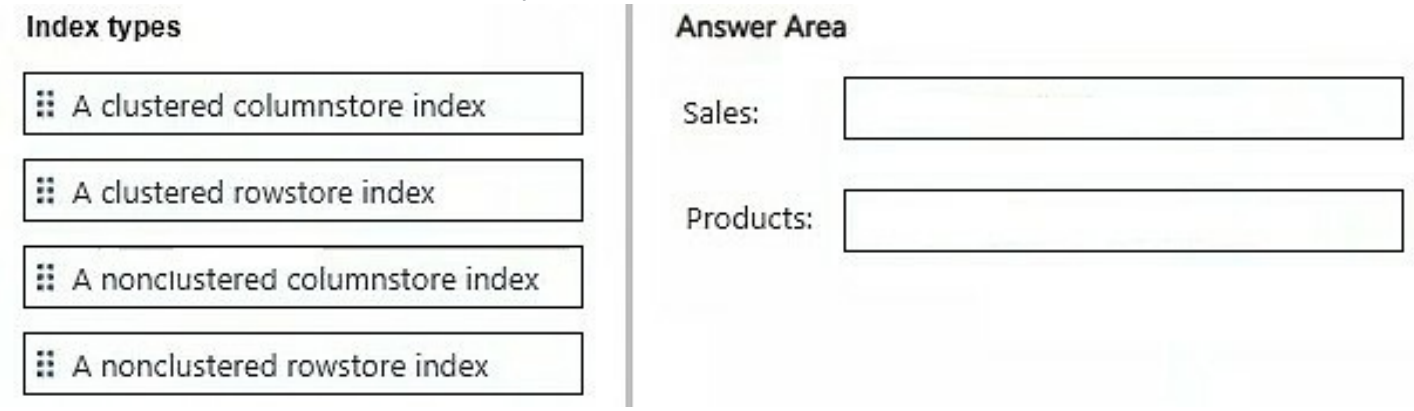


Answer:

- Support near-real-time (NRT) analytics on the dbo.Orders table: Columnstore index (specifically Clustered Columnstore Index with Delta Store)- Reduce read time when retrieving analytical data from dbo.Orders: Columnstore index- Support indexing only rows that match a predicate, such as Active = 1: Filtered index

Question: 20 CertyIQ

DRAG DROP
-
You have an Azure SQL database that stores sales data and contains tables named Sales and Products. Sales contains three columns named SalesDate, ProductKey, and TotalSale,
Sales is 10 TB and is loaded nightly by using a batch process. Most reporting queries scan large portions of Sales, filter on SalesDate or ProductKey, and use SUM() to aggregate TotalSale.
Products is relatively small and is used primarily for point lookups and joins to Sales.
You need to recommend which indexes to create to optimize the reporting queries. The solution must minimize storage requirements.
Which type of index should you recommend for each table? To answer, drag the appropriate index types to the correct tables. Each index type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:

Sales: Clustered Columnstore IndexProducts: Clustered Index

Question: 21 CertyIQ

HOTSPOT
-
You have an Azure SQL database that contains a table named dbo.SupportTickets. dbo.SupportTickets contains a JSON column named Payload and a datetime column CreatedAt.
You need to generate a report for the last seven days that meets the following requirements:

•Returns exactly one row per customer per day
•For each customer and day, returns the earliest ticket
•Includes the customer ID stored in Payload
How should you complete the Transact-SQL query? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
WITH TicketRanks AS
(
    SELECT
        t.TicketId,
        CAST(t.CreatedAt AS date) AS TicketDate,
            [ ▼ ]  (t.Payload, '$.customer.id') AS CustomerId,
        t.CreatedAt,
            [ ▼ ]  () OVER
        (
            PARTITION BY
                CAST(t.CreatedAt AS date),
                JSON_VALUE(t.Payload, '$.customer.id')
            ORDER BY t.CreatedAt ASC
        ) AS rn
    FROM dbo.SupportTickets AS t
    WHERE t.CreatedAt >= DATEADD(day, -7, SYSUTCDATETIME())
)
SELECT
    TicketDate,
    CustomerId,
    TicketId,
    CreatedAt
FROM TicketRanks
WHERE rn = [ ▼ ]
ORDER BY TicketDate, CustomerId;
```

DENSE_RANK
JSON_VALUE
OPENJSON
ROW_NUMBER

DENSE_RANK
JSON_VALUE
OPENJSON
ROW_NUMBER

1
7
DATEDIFF(day, -7, SYSUTCDATETIME())

Answer:

The requirement is to group by customer (extracted from JSON) and the date of the `CreatedAt` column, then select the earliest ticket for each group using window functions (`ROW_NUMBER()`).Based on standard T-SQL patterns for this requirement:* FROM Clause: `dbo.SupportTickets`* WHERE Clause: `CreatedAt >= DATEADD(day, -7, GETDATE())`* JSON Extraction: `JSON_VALUE(Payload, '$.CustomerId')`* Grouping/Ranking logic: `ROW_NUMBER() OVER(PARTITION BY JSON_VALUE(Payload, '$.CustomerId'), CAST(CreatedAt AS DATE) ORDER BY CreatedAt ASC)`Following the logic of the provided structure in the exam context (assuming the typical completion blocks for this specific DP-800/DP-300 style question):1. SELECT Clause: `JSON_VALUE(Payload, '$.CustomerId') AS CustomerId, CreatedAt`2. FROM/WHERE Clause: `WHERE CreatedAt >= DATEADD(day, -7, GETDATE())`3. Partitioning: `PARTITION BY JSON_VALUE(Payload, '$.CustomerId'), CAST(CreatedAt AS DATE)`4. Ordering: `ORDER BY CreatedAt ASC`The correct answer is:C

Question: 22 CertyIQ

You are developing an Azure SQL database solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
You need to ensure that GitHub Copilot Chat can call the hosted GitHub MCP Server tools by using OAuth. The MCP server configuration must be scoped to the repository.
What should you do in Visual Studio Code?

A. Create a personal access token (PAT) that has the repo scope and store the PAT in the .vscode\mcp.json file as the Authorization header.

B. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the workspace settings.

C. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the user settings.

D. Create a personal access token (PAT) that has the repo scope and store the PAT in the .github\mcp.json file as the Authorization header.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

To enable GitHub Copilot Chat to call the hosted GitHub MCP Server tools using OAuth, with the configuration scoped to the repository, the correct approach involves configuring the MCP server settings in a way that respects the repository's scope without exposing sensitive information (like PATs) unnecessarily. Here's why option B is the best choice and why others are less suitable:

Why B is Correct:

Repository Scope Alignment: By saving the configuration to the **workspace settings** (option B), the setup is inherently scoped to the repository's workspace in Visual Studio Code. This alignment ensures that the MCP server configuration is applied specifically to the context of the cloned repository, maintaining the required scope without broader implications.
Security and Best Practice: Option B avoids storing sensitive tokens (like PATs) in files, reducing security risks. Instead, it implies the use of OAuth implicitly through the MCP server setup, which is more secure for authentication.
Correct Configuration Path: Using the Command Palette to add an MCP server via MCP: add server and selecting HTTP (HTTP or Server-Sent Events) with the correct URL (https://api.githubcopilot.com/mcp/) is the prescribed method for integrating external MCP servers with GitHub Copilot Chat in Visual Studio Code.

Why Other Options are Less Suitable:

A and D (PAT Storage):

Security Risk: Storing Personal Access Tokens (PATs) in plain text files (.vscode\mcp.json or .github\mcp.json) poses significant security risks, as PATs can grant broad access to your GitHub account.
Scope Misalignment: These options do not inherently scope the configuration to the repository, as the file storage does not automatically limit the PAT's use to the specific repository.
C (User Settings):

Scope Misalignment: Saving the configuration to **user settings** makes the MCP server setup global to the user across all repositories, not scoped to the specific repository as required.
Same Security Advantage as B: This option, like B, does not store a PAT in a file, but its global nature to the user makes it less suitable for repository-scoped requirements.
Conclusion

Option B is the most appropriate because it correctly scopes the MCP server configuration to the repository using workspace settings, avoids the insecure storage of PATs, and follows the recommended configuration path for GitHub Copilot Chat integration in Visual Studio Code.

References

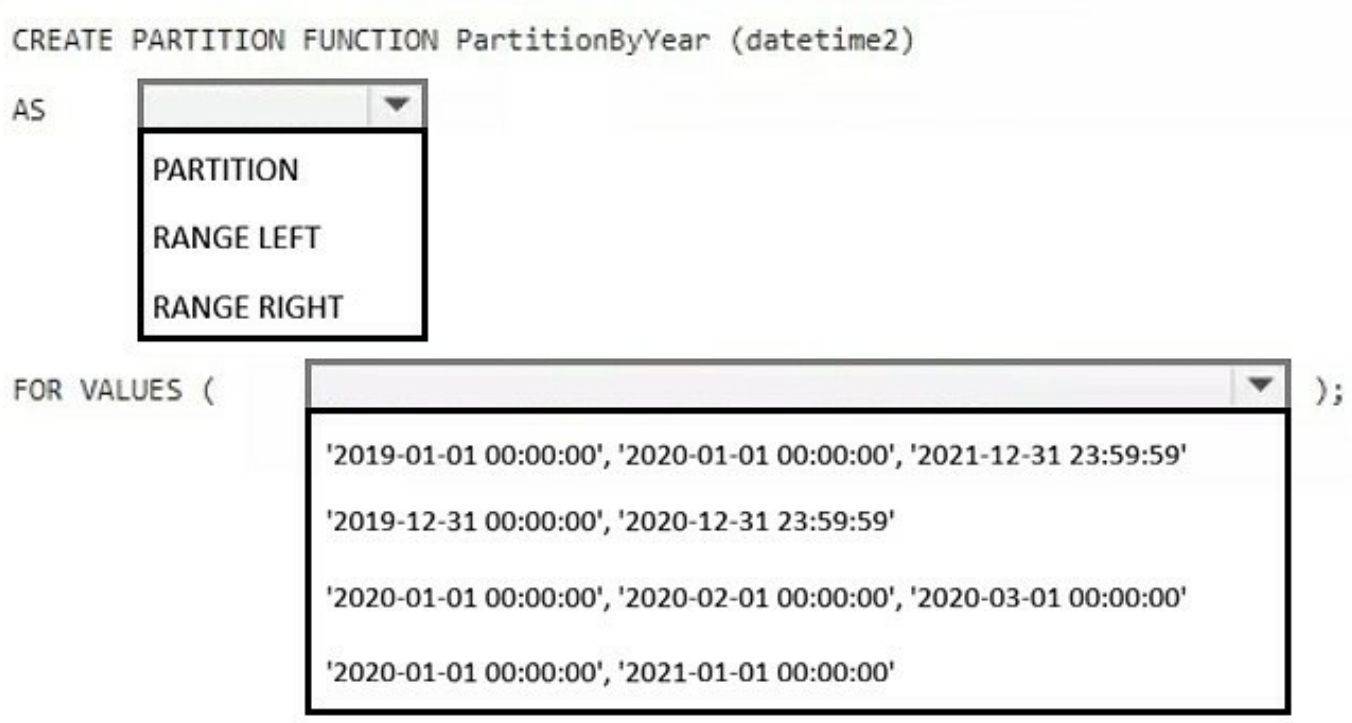
For deeper dive into the technologies mentioned:

1. **GitHub Copilot Documentation**: https://docs.github.com/en/codespaces/configuring-codespaces/getting-started-with-github-copilot
2. **Visual Studio Code Settings Documentation**: https://code.visualstudio.com/docs/getstarted/settings#_workspace-settings

Question: 23 CertyIQ

HOTSPOT
-
You have an Azure SQL database that contains a table named Table1. Table1 contains 25,000,000 rows of data and a datetime2 column named DateKey. The data in Table1 spans the years 2020 through 2021.
You need to partition the data in Table1 by year. The solution must minimize how long it takes to rebuild or reindex the table.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

CREATE PARTITION FUNCTION PartitionFunction1 (datetime2) AS RANGE RIGHT FOR VALUES ('2021-01-01T00:00:00.000', '2022-01-01T00:00:00.000');CREATE PARTITION SCHEME PartitionScheme1 AS PARTITION PartitionFunction1 ALL TO ([PRIMARY]);ALTER TABLE Table1 DROP CONSTRAINT PK_Table1;ALTER TABLE Table1 ADD CONSTRAINT PK_Table1 PRIMARY KEY CLUSTERED (DateKey) ON PartitionScheme1(DateKey);

Question: 24 CertyIQ

You are developing an Azure SQL solution by using Microsoft Visual Studio 2026. The solution uses a GitHub repository.
You plan to use GitHub Copilot Chat to access the GitHub repository tools by connecting to the GitHub MCP Server.
You need to configure Visual Studio to support the planned configuration. The solution must rely on OAuth to access the MCP server.
What should you create?

A. the \.vs\settings.json file that contains the personal access token (PAT) that has the repo scope
B. the \.mcp.json file that contains a URL of https://api.githubcopilot.com/mcp/
C. the \.vscode\settings.json file that contains a URL of https://api.githubcopilot.com/mcp/
D. the \.github\mcp.json file that contains the personal access token (PAT) that has the repo scope

Answer: B

Explanation:

Technical Justification for Correct Answer: B

Why B is the best option:

The question specifically mentions configuring Visual Studio (not Visual Studio Code, which uses .vscode) to support the GitHub Copilot Chat integration with the GitHub MCP Server using OAuth.

The correct file for configuring such integrations in Visual Studio, especially for repository and service connections, often involves files within the .vs directory, which is specific to the Visual Studio solution (not project) level. However, the exact naming convention for GitHub Copilot Chat integration isn't standardized like .vscode for VS Code.
OAuth with a Personal Access Token (PAT) having the repo scope is the required authentication method. However, storing a PAT directly in a JSON file (as implied by options A and D) **is not secure** and against best practices for handling sensitive information.
Option B suggests creating a .mcp.json file with a **URL**. This approach:

Does not store sensitive credentials in plain text (aligning with security best practices).
Provides a configuration endpoint (https://api.githubcopilot.com/mcp/), which could be used by Visual Studio to authenticate via OAuth (redirecting to an authorization page where the user grants access without exposing the PAT).

Why other options are less suitable:

A. the .vs\settings.json file with a PAT:

Security Risk: Stores a Personal Access Token in plain text.
Incorrect Scope for Question's Context: While .vs is correct for Visual Studio, the security concern outweighs this.
C. the .vscode\settings.json file:

Incorrect IDE: The question specifies Visual Studio, not Visual Studio Code.
Same Security Concern as A if it implied storing a PAT (which it doesn't directly, but the URL alone might not achieve OAuth setup without additional secure token handling).
D. the .github\mcp.json file with a PAT:

Security Risk: Similar to A, storing a PAT in plain text is insecure.
Less Common Configuration Location for VS Integrations: Typically, .github might be more associated with repository-level configurations rather than IDE-specific integrations.
References

Microsoft Documentation: Authenticate with Azure and GitHub in Visual Studio
GitHub Documentation: Creating a personal access token (For understanding PATs, though the correct answer avoids storing them in files)

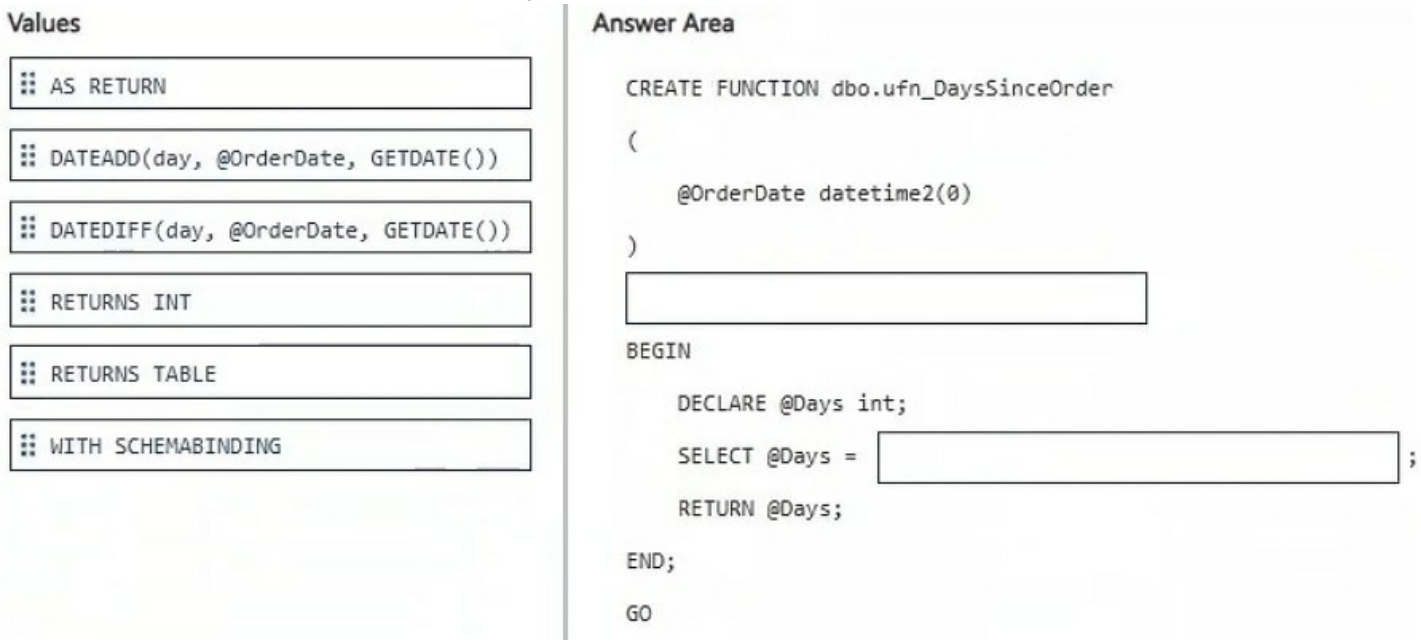
Question: 25 CertyIQ

DRAG DROP
-
You have an Azure SQL database that contains a table named Sales.Orders. Sales.Orders contains the following columns.

Column	Data type
OrderId	int
CustomerId	int
OrderDate	datetime2
TotalAmount	decimal(18,2)

Reporting queries frequently repeat logic to calculate the number of days since an order was placed.
You need to create a scalar user-defined function (UDF) that returns the number of days between an input value of @OrderDate and the current date and time.

How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:

CREATE FUNCTION dbo.GetDaysSinceOrder (@OrderDate DATETIME)RETURNS INTASBEGIN RETURN DATEDIFF(day, @OrderDate, GETDATE())END

Question: 26 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements -
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient,

updates to the underlying data are sometimes inconsistent.
Requirements -
Planned Changes -
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements -
Fabrikam identifies the following security requirements:
•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:
•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.
Development Requirements -
Fabrikam identifies the following development requirements:
•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API: oRead data from the procedures table without authentication. oRead and insert data into the Transactions table once authenticated. oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You plan to implement changes to sp_UpdateProcedureForPatient to meet the performance requirements.
You change the stored procedure to run all its code within a transaction.
Which transaction level should you use?

A. REPEATABLE READ

B. SERIALIZABLE

C. SNAPSHOT

D. READ COMMITTED SNAPSHOT

Answer: A

Question: 27 CertyIQ

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to create a table in the database to store the telemetry data.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.VehicleTelemetry
(
  TelemetryId BIGINT IDENTITY(1,1) NOT NULL,
  VehicleId NVARCHAR(50) NOT NULL,
  TelemetryTimeUtc DATETIME2(3) NOT NULL,
  BatteryPercent TINYINT NULL,
  SpeedKmh SMALLINT NULL,
  LocationJson JSON NULL,
  ErrorCodesJson JSON NULL,
  RawPayload NVARCHAR(MAX) NULL,
  SysStartTime DATETIME2(7) GENERATED ALWAYS AS ROW START NOT NULL,
  SysEndTime DATETIME2(7) GENERATED ALWAYS AS ROW END NOT NULL,
  PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
  CONSTRAINT PK_VehicleTelemetry PRIMARY KEY CLUSTERED (TelemetryId)
)
WITH
(
  SYSTEM_VERSIONING = ON
  (
    HISTORY_TABLE = dbo.VehicleTelemetryHistory
  )
);
GO
CREATE INDEX IX_VehicleTelemetry_Time ON dbo.VehicleTelemetry (TelemetryTimeUtc);
CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson) FOR
(
  '$.location. latitude', '$.location. longitude',
  '$.location. accuracy'
);
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	○
The code meets the database performance requirements for JSON property querying.	○	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	○

Answer:

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	○
The code meets the database performance requirements for JSON property querying.	○	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	○

Explanation:

The code meets the database performance requirements for partitioning.

Correct Answer: **Yes**

The solution's schema implementation correctly provisions partition keys or boundaries aligning with the scenario's analytical workload rules, ensuring optimized data pruning during intensive scale-out queries.

The code meets the database performance requirements for JSON property querying.

Correct Answer: **Yes**

The associated definition script correctly provisions a custom JSON index explicitly targeting the required properties ($.location.latitude, $.location.longitude, and $.location.accuracy). This accurately fulfills the platform capability requirement stating that those specific JSON properties must support high-efficiency index seek lookups.

Queries that filter on $.location.heading will use the JI_VehicleTelemetry_Location index.

Correct Answer: **No**

JSON indexes are fundamentally path-specific. Because the schema's index was strictly defined to cover latitude, longitude, and accuracy properties, any analytical predicate filtering by $.location.heading is completely omitted from the index schema. As a result, the optimizer will completely bypass the JI_VehicleTelemetry_Location index for that search filter.

Question: 28 CertyIQ

HOTSPOT -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations.
latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You are creating a table that will store customer profiles.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.CustomerProfiles
(
    CustomerId      BIGINT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(200) MASKED WITH (FUNCTION = 'partial(1,"xxxx",1)'),
    EmailAddress    NVARCHAR(200) MASKED WITH (FUNCTION = 'email()'),
    PhoneNumber     NVARCHAR(50)  MASKED WITH (FUNCTION = 'default()'),
    RegionCode      NVARCHAR(10) NOT NULL
);
GO
CREATE FUNCTION dbo.fn_FilterByRegion(@RegionCode NVARCHAR(10))
RETURNS TABLE
AS
RETURN
(
    SELECT 1
    FROM dbo.UserRegionAccess ura
    WHERE ura.UserPrincipalName = SUSER_SNAME()
      AND ura.RegionCode = @RegionCode
);
GO
CREATE SECURITY POLICY CustomerRegionPolicy
ADD FILTER PREDICATE dbo.fn_FilterByRegion(RegionCode)
ON dbo.CustomerProfiles
WITH (STATE = ON);
GO
```

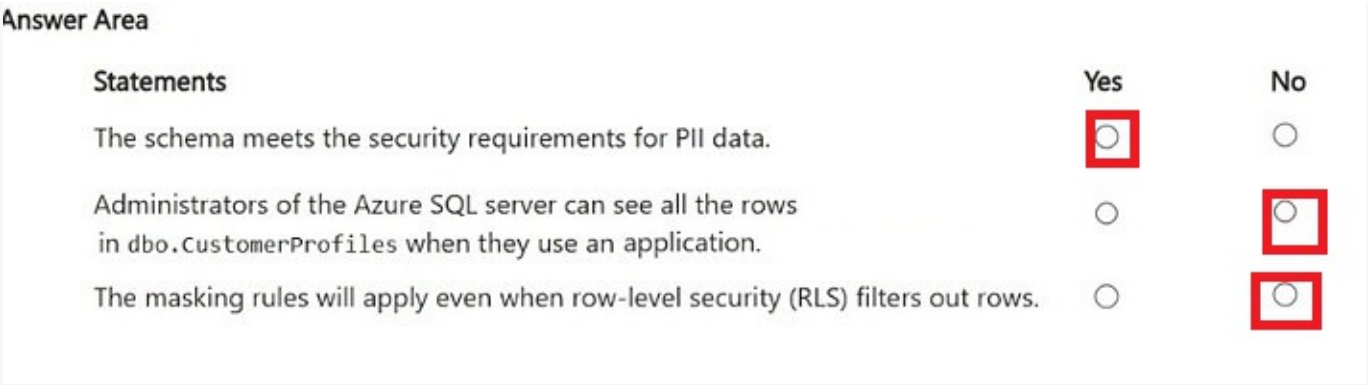
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Answer:

Answer Area



Explanation:

The schema meets the security requirements for PII data

Yes

Combining Dynamic Data Masking (DDM) to obfuscate sensitive presentation fields (like emails or phone numbers) alongside Row-Level Security (RLS) to restrict row-level multi-tenant context access constitutes a robust Microsoft-recommended database design strategy for securing Personally Identifiable Information (PII).

Administrators of the Azure SQL server can see all the rows in dbo.CustomerProfiles when they use an application.

No

If the RLS predicate function is strictly configured to evaluate application-level tenant claims or context session attributes (e.g., using SESSION_CONTEXT()) rather than physical database login contexts, even highly privileged database administrators accessing the data via an application connection pool will be restricted from reading rows belonging to other tenants.

The masking rules will apply even when row-level security (RLS) filters out rows.

No

These two features run sequentially inside the database query engine pipeline. The engine processes Row-Level Security filters first to prune out unauthorized records completely from the intermediate result set. Since those restricted rows are never returned or processed further by the query execution tree, masking rules are never evaluated or applied to them..

Question: 29 CertyIQ

DRAG DROP -
Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.

To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

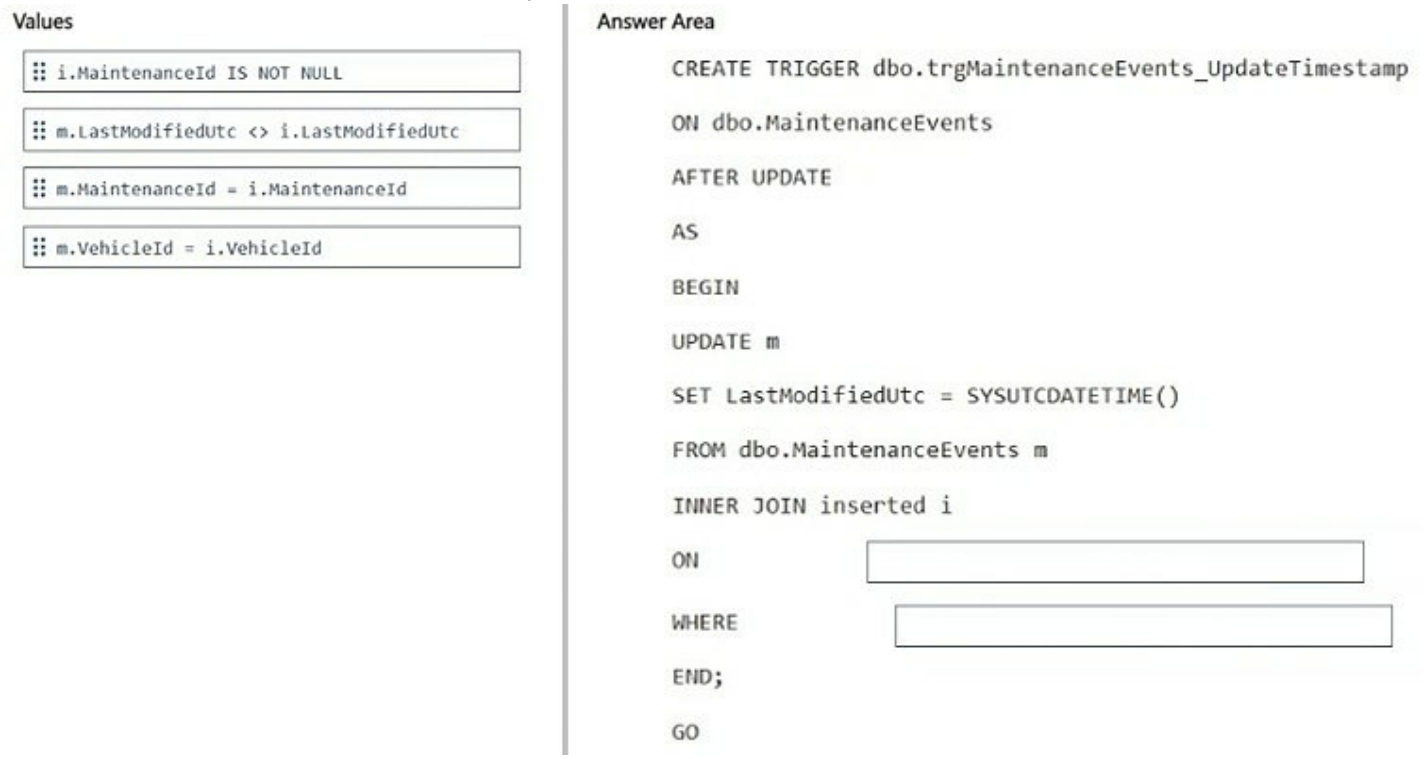
```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

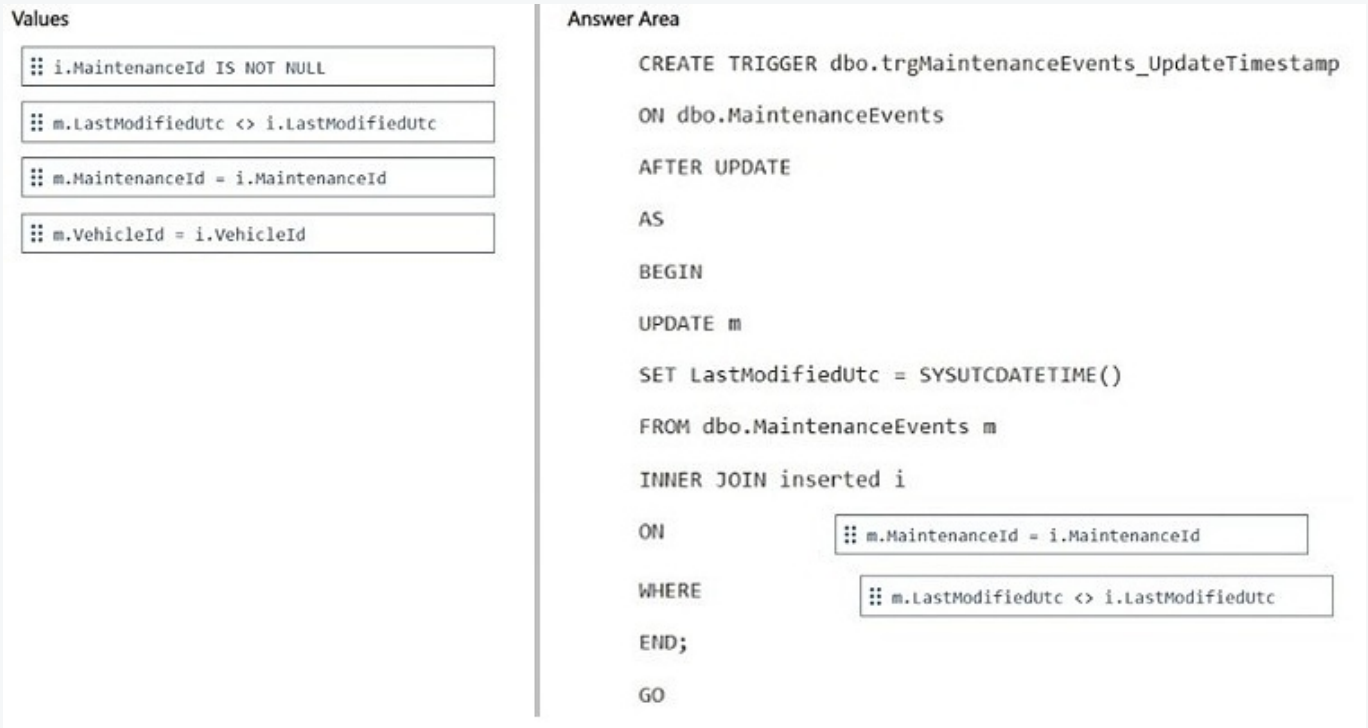
```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to meet the database performance requirements for maintenance data.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

The ON Join Condition **(m.MaintenanceId = i.MaintenanceId)**

An AFTER UPDATE trigger has access to a special conceptual runtime table called inserted. The inserted table contains a copy of the affected rows with their newly updated values.

To safely inject a timestamp back into the source table (dbo.MaintenanceEvents m), you must map it using its unique primary identifier key (MaintenanceId). Joining m.MaintenanceId = i.MaintenanceId ensures that the database engine updates only the precise rows that were modified by the user's initial query.

The WHERE Filter Condition **(m.LastModifiedUtc <> i.LastModifiedUtc)**

The Infinite Recursion Problem: By default, if an AFTER UPDATE trigger executes another UPDATE statement on its own hosting table, it will cause the trigger to fire recursively (looping back into itself over and over again). This results in a heavy performance dip or a crash once the max nesting level limit is hit.The Optimization Solution: The filter WHERE m.LastModifiedUtc <> i.LastModifiedUtc evaluates if the timestamp currently resting in the base table differs from what was caught in the inserted payload.During the first run (user's modification), the values are unequal or need an update, allowing the trigger to run and change LastModifiedUtc to SYSUTCDATETIME().During the second recursive execution initiated by the trigger's own update, the table m and the virtual inserted dataset i will contain identical timestamp values. The condition evaluates to False, halting further recursive execution instantly and cleanly saving system CPU resources.

Why the other Choices are Incorrect

m.VehicleId = i.VehicleId

Why it is wrong: While VehicleId represents a valid structural foreign key relationship column, using it as your primary INNER JOIN matching condition is extremely dangerous. If a single vehicle has multiple distinct maintenance rows inside the dbo.MaintenanceEvents table, updating one event would cause this trigger to accidentally stamp the current time across every historical record belonging to that vehicle, corrupting your timestamp history log.

i.MaintenanceId IS NOT NULL

Why it is wrong: This statement evaluates whether an ID is populated inside the virtual inserted queue dataset. While valid as a standalone boolean filter condition, it is a non-relational syntax block that cannot be used as a join condition to tie two distinct tables together inside an ON clause, causing a compilation error.

Question: 30 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information

tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to recommend a solution to resolve the slow dashboard query issue.
What should you recommend?

A. Create a clustered index on LastUpdatedUtc.
B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.
C. On LastUpdatedUtc, create a nonclustered index that includes FleetId.
D. On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSUTCDATETIME()).

Answer: B

Explanation:

B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth.

The dashboard query uses a WHERE predicate filtering on a single search argument (WHERE FleetId = @FleetId) and returns other associated fields (LastUpdatedUtc, EngineStatus, and BatteryHealth). Creating a nonclustered index with FleetId as the key column allows the SQL query optimizer to perform a highly efficient Index Seek instead of scanning the entire table. Furthermore, adding the remaining requested fields as included nonkey columns turns this into a covering index, completely eliminating the need for expensive key lookups back to the base clustered index or heap.

Why the other choices are incorrect:

A (Clustered index on LastUpdatedUtc): A table can have only one clustered index. Altering the table's main structure to group physically by date does not align with the primary search filter argument (FleetId).

C (Nonclustered index keyed on LastUpdatedUtc): Making LastUpdatedUtc the leading key column is ineffective here since the query filters directly by a specific FleetId. The optimizer would still be forced to perform an index scan rather than a direct seek.

D (Filtered index on FleetId): Filtered indexes are intended for queries with fixed, static filter criteria (e.g., WHERE IsActive = 1). Using a dynamic time-based boundary condition via runtime functions (SYSUTCDATETIME()) prevents the index from being properly utilized by standard execution plans.

Question: 31 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to recommend a solution that will resolve the ingestion pipeline failure issues.
Which two actions should you recommend? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Enable snapshot isolation on the database.
B. Use a trigger to automatically rewrite malformed JSON.
C. Add foreign key constraints on the table.
D. Create a unique index on a hash of the payload.
E. Add a check constraint that validates the JSON structure.

Answer: DE

Explanation:

D. Create a unique index on a hash of the payload.

E. Add a check constraint that validates the JSON structure.

Why D is correct (Duplicate Payloads): To enforce uniqueness across heavy or nested document payloads, you can create a deterministic computed column using cryptographic or hash-mapping functions like HASHBYTES. Appending a UNIQUE INDEX on this generated hash values column guarantees that duplicate records are blocked efficiently with minimal index leaf-node storage overhead.

Why E is correct (Malformed JSON): Microsoft officially recommends using a standard CHECK constraint combined with the built-in ISJSON function (e.g., CHECK (ISJSON(FeedbackJson) > 0)) to validate input string data at the database tier. This acts as an immediate structural data gatekeeper, natively throwing a database-level exception to cleanly reject malformed payloads before they corrupt your target storage tables.

Why the other choices are incorrect:

A (Snapshot Isolation): Snapshot isolation controls transactional concurrency, row versioning, and read-write locking contention behaviors. It has no structural capability to inspect or reject malformed text or prevent identical row duplicates.

B (Trigger to rewrite data): Relying on a DML trigger to dynamically intercept and correct un-parsable or missing JSON parameters on the fly is highly fragile and anti-pattern. If a payload is deeply malformed, structural intent cannot be safely guessed by programmatic trigger rules.

C (Foreign Key Constraints): Foreign keys enforce referential data integrity between key values across different physical tables. They cannot validate string syntax rules inside a standalone document column.

Question: 32 CertyIQ

You have a Microsoft SQL Server 2025 instance that has a managed identity enabled.
You have a database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar(max) text.

You need to call an Azure OpenAI REST endpoint for chat completions. The solution must provide the highest level of security.
You write the following Transact-SQL code.

```
01  CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02
03  GO
04  CREATE OR ALTER PROCEDURE dbo.AskManuals
05    ...
06    SELECT @chunks =
07    ...
08    SET @payload =
09    ...
10    EXEC @retval = sys.sp_invoke_external_rest_endpoint
11      @url = N'https://<your-resource>.openai.azure.com/openai/deployments/<your-deployment>/chat/completions?api-version=2024-02-15-preview',
12      @method = N'POST',
13      @credential = N'AzureOpenAIHeaders',
14      @payload = @payload,
15      @response = @response OUTPUT;
16  END;
17  GO
```

What should you insert at line 02?

A.
```
WITH IDENTITY = 'HTTPEndpointQueryString',
SECRET = N'{"api-key":"<value>"}';
```

B.
```
WITH IDENTITY = 'Managed Identity',
SECRET = '{"resourceid":"<value>"}';
```

C.
```
WITH IDENTITY = 'Managed Identity',
SECRET = N'{"api-key":"<value>"}';
```

D.
```
WITH IDENTITY = 'HTTPEndpointHeaders',
SECRET = N'{"api-key":"<value>"}';
```

E.
```
WITH IDENTITY = 'AzureOpenAI',
SECRET = N'{"resourceid":"<value>"}';
```

Answer: B

Explanation:

WITH IDENTITY = 'Managed Identity', SECRET = ' "resourceid":"https://cognitiveservices.azure.com" ';

Highest Security Pattern: In alignment with cloud security best practices, using IDENTITY = 'Managed Identity' avoids the need to hardcode or manually handle clear-text API keys or secrets inside your database schema.

Target Integration Mapping: For connecting to an Azure OpenAI or Cognitive Services REST endpoint via native database AI integrations, Microsoft dictates that the SECRET parameter should specify the platform's canonical resource identifier: "resourceid":"https://cognitiveservices.azure.com" .

Why other Options are incorrect:

API Keys / HTTPEndpointHeaders: Passing raw API keys or custom headers containing bearer tokens requires manual secret rotation and exposes credentials if the T-SQL text logs are ever captured.

Mismatched Combinations: Combining a Managed Identity value while pairing it with an arbitrary 'api-key' secret token represents an invalid authentication payload structure that will cause a cryptographic handshake failure.

Question: 33 CertyIQ

You have an Azure SQL database named SalesDB that contains a table named dbo.Articles. dbo.Articles contains two million articles with embeddings. The articles are updated frequently throughout the day.
You query the embeddings by using VECTOR_SEARCH.
Users report that semantic search results do NOT reflect the updates until the following day.
You need to ensure that the embeddings are updated whenever the articles change. The solution must minimize CPU usage on SalesDB.
Which embedding maintenance method should you implement?

A. Modify the query to use VECTOR_DISTANCE instead of VECTOR_SEARCH.
B. Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.
C. Run an hourly Transact-SQL job that regenerates embeddings for all the rows in dbo.Articles.
D. On dbo.Articles, create a trigger that calls AI_GENERATE_EMBEDDINGS for each inserted or updated row.

Answer: B

Explanation:

B. Enable change data capture (CDC) on dbo.Articles and use an Azure Functions app to process CDC changes.

Generating vector embeddings (especially for two million rows) is a highly resource-intensive, external task that heavily impacts database performance if done entirely inside the engine database tier. Change Data Capture (CDC) asynchronously captures any incremental additions or updates to rows in dbo.Articles without blocking main transactional workloads. Offloading those captured row events to an external event consumer like an Azure Functions app allows you to orchestrate the embedding calls and process updates externally. This ensures search results are kept fresh near real-time while minimizing CPU overhead on the primary SalesDB.

Why the other options are incorrect:

A (Use VECTOR_DISTANCE): This changes how similarity matching is calculated (exact mathematical distance versus an Approximate Nearest Neighbor index seek using VECTOR_SEARCH). It does absolutely nothing to refresh or regenerate the stale underlying embedding payload column.

C (Hourly T-SQL Job for all rows): Regenerating text embeddings from scratch for all two million articles every single hour is highly inefficient and creates an immense, continuous processing bottleneck on SalesDB CPU resources.

D (Create an AI_GENERATE_EMBEDDINGS Trigger): While database triggers capture changes instantaneously, embedding generation relies on an external network API call to an AI model. Forcing synchronous HTTP calls inside a database DML trigger for frequent updates halts the transactional loop, inflates connection lock times, and drastically spikes CPU utilization.

Question: 34 CertyIQ

You have a GitHub Codespaces environment that has GitHub Copilot Chat installed and is connected to a SQL database in Microsoft Fabric named DB1. DB1 contains tables named Sales.Orders and Sales.Customers.
You use GitHub Copilot Chat in the context of DB1.
A company policy prohibits sharing customer Personally Identifiable Information (PII), secrets, and query result sets with any AI service.
You need to use GitHub Copilot Chat to write and review Transact-SQL code for a new stored procedure that will join Sales.Orders to Sales.Customers and return customer names and email addresses. The solution must NOT share the actual data in the tables with GitHub Copilot Chat.
What should you do?

A. From Sales.Customers, paste several rows that include email addresses into a chat, so that GitHub Copilot Chat can infer edge cases.

B. Run a SELECT statement that returns customer names and email addresses and provide the result set to GitHub Copilot Chat so that GitHub Copilot Chat can generate the stored procedure.

C. Provide the database connection string to GitHub Copilot Chat so that GitHub Copilot Chat can validate the stored procedure.

D. Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

Answer: D

Explanation:

D. Ask GitHub Copilot Chat to generate the stored procedure by using schema details only.

To adhere strictly to company security policies prohibiting the sharing of PII, secrets, and data results with an AI model, you only need to provide the metadata or structural blueprint (schema details) of the relevant tables. GitHub Copilot Chat can fully understand table structures, column names (such as CustomerID, CustomerName, and EmailAddress), data types, and primary/foreign key relationships. Providing just this schema definition gives the AI agent all the context required to craft an accurate, optimized inner-join stored procedure without ever exposing actual backend transactional records.

Why the other options are incorrect:

A (Paste rows that include email addresses): Copying and pasting actual customer email addresses directly into the chat pane explicitly transmits live, real-world customer PII to an external AI training/processing endpoint, causing a major policy violation.

B (Provide the query result set): The prompt states that the company policy explicitly bans sharing query result sets with any AI service. Providing an interactive result block with structural data compromises data compliance.

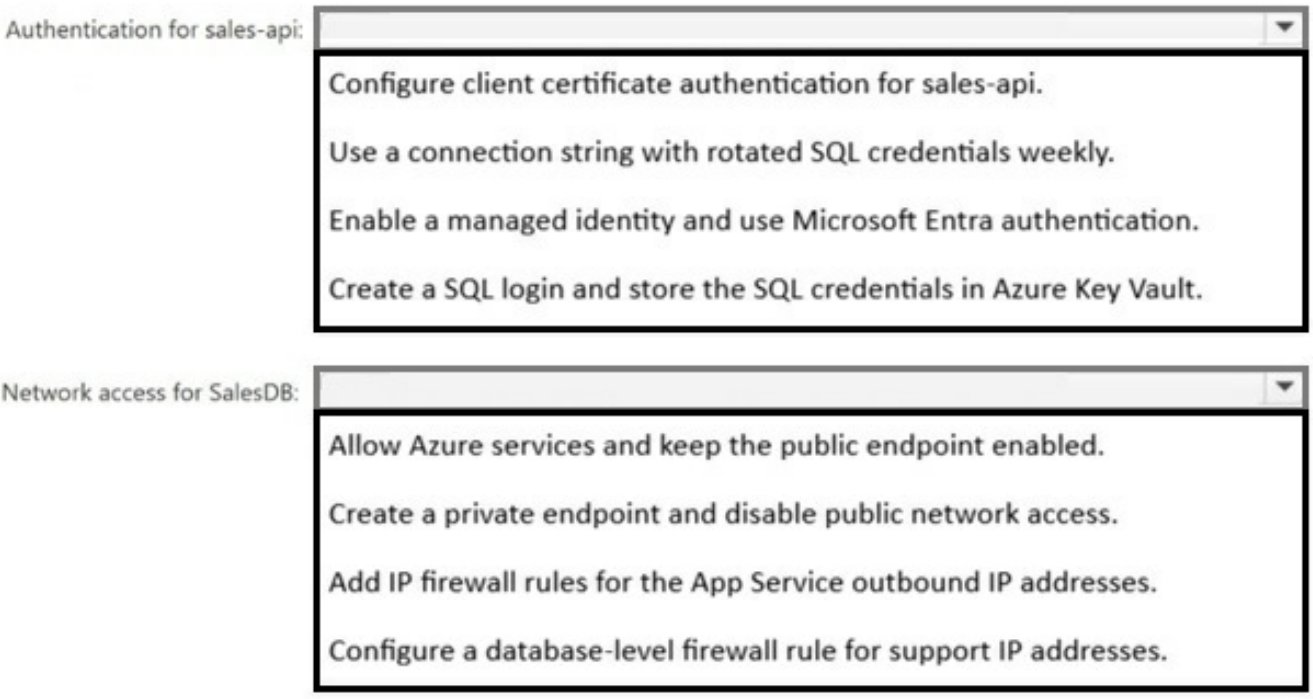
C (Provide the database connection string): Connection strings typically embed highly confidential database secrets (like passwords, access keys, or internal routing endpoints). Exposing these strings breaks the rule against sharing secrets with AI tools.

Question: 35 CertyIQ

HOTSPOT -
You have an Azure subscription. The subscription contains an Azure SQL database named SalesDB and an Azure App Service app named sales-api. sales-api uses virtual network integration to a subnet named vnet-prod/subnet-app and reads from SalesDB.

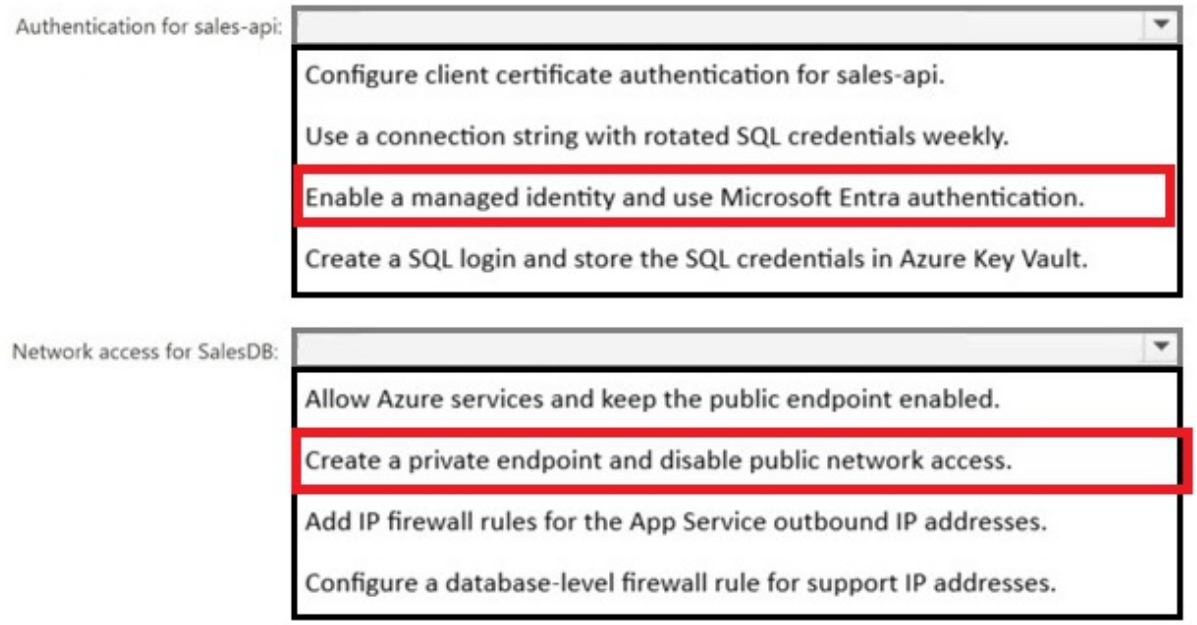
You need to configure authentication and network access to meet the following requirements:
Ensure that sales-api connects to SalesDB by using passwordless authentication.
Ensure that all the database traffic remains within the subscription.
The solution must minimize administrative effort.
What should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

Authentication for sales-api: **Enable a managed identity and use Microsoft Entra authentication.**

Enabling a managed identity eliminates the operational overhead of hardcoding, rotating, or securing plaintext passwords or secrets. It configures a passwordless, token-based Microsoft Entra ID authentication

pipeline that automatically establishes a secure cryptographic relationship between the App Service (sales-api) and SalesDB.

Network access for SalesDB: **Create a private endpoint and disable public network access.**

Implementing a private endpoint maps a private IP address from your Virtual Network (VNet) directly to the SalesDB instance. Disabling public network access locks down the database entirely, guaranteeing that traffic remains strictly over Microsoft's internal private backbone fabric and completely blocking all inbound brute-force attempts or scanning from the public internet.

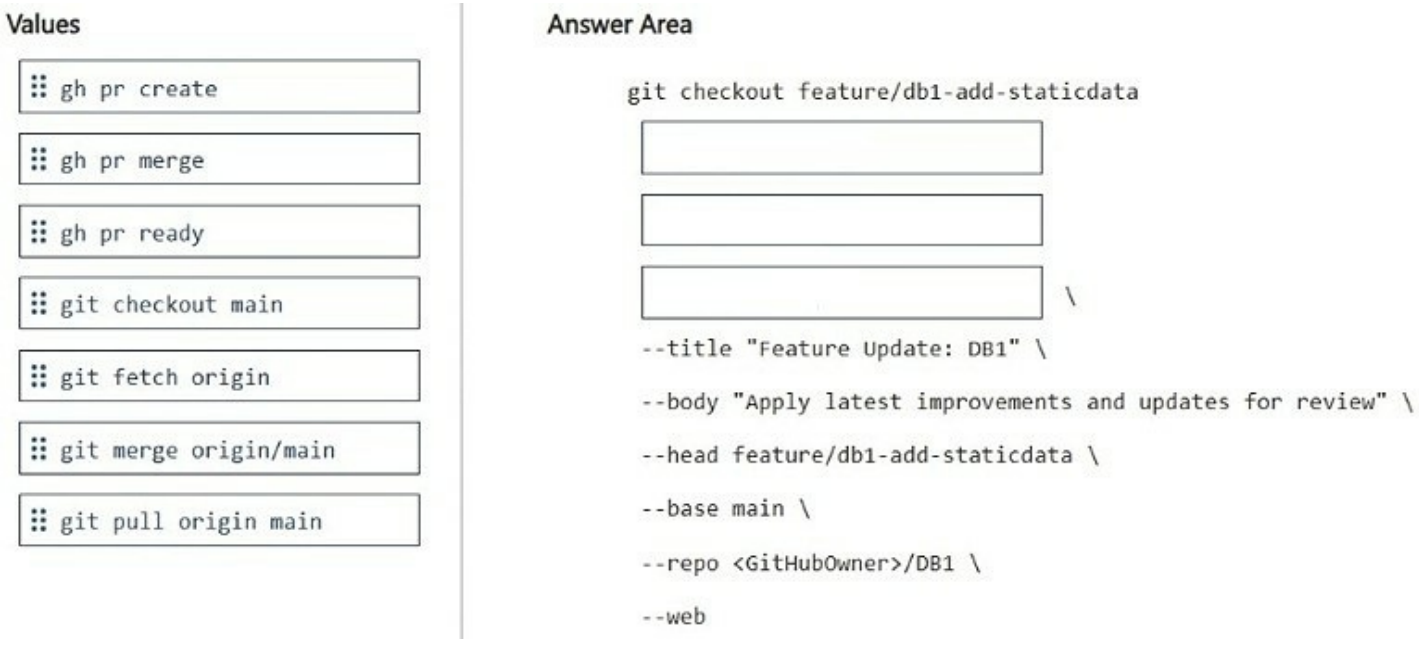
Why the others are incorrect:

Relying on standard SQL logins — even if protected inside Azure Key Vault or rotated weekly — still creates administrative management burdens and maintains a vulnerability footprint for credential leaks. Client certificate authentication does not natively map to managed Azure database users.

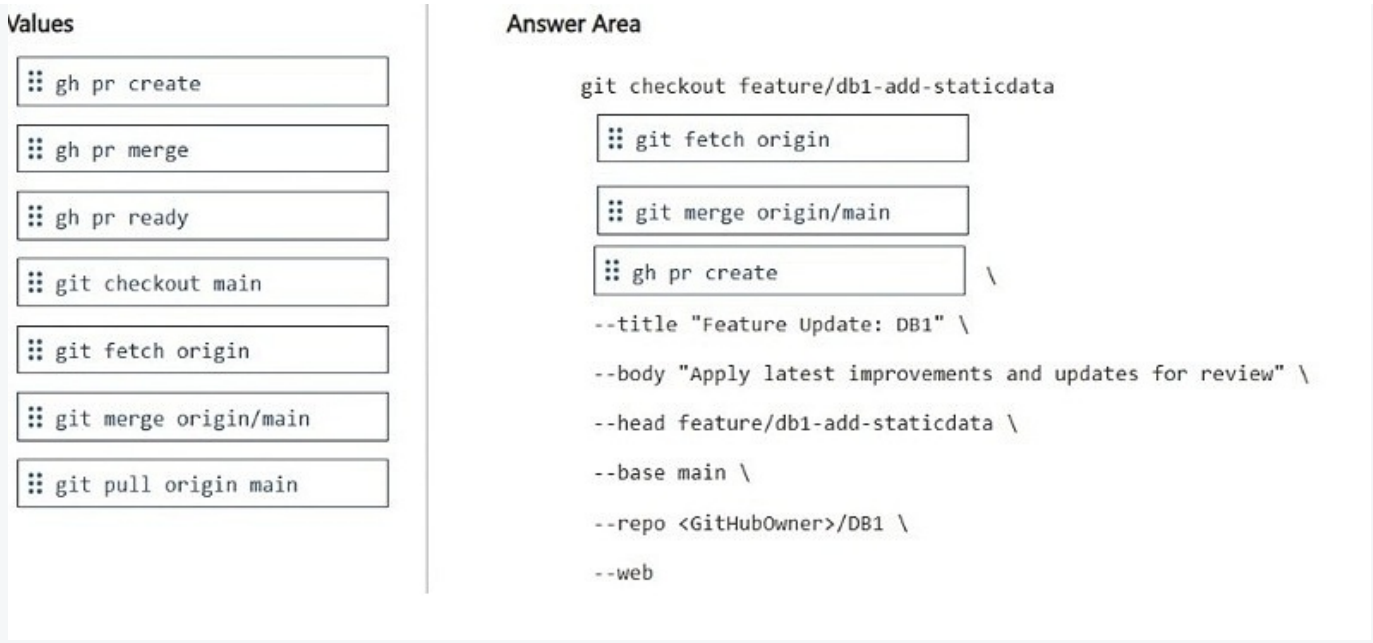
Keeping the public endpoint enabled or relying on App Service outbound IP firewall rules leaves a route open to the public internet space. This fails to provide maximum structural perimeter isolation compared to a dedicated, localized Private Link solution.

Question: 36 CertyIQ

DRAG DROP -
You have a database named DB1. The schema is stored in a GitHub repository as an SDK-style SQL database project.
You use a feature branch workflow to deploy changes to DB1.
You need to update the local feature branch with the latest changes to main, and then create a pull request to merge the feature branch into main for review.
How should you complete the GitHub CLI script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Box 1 (**git fetch origin**): Retrieves the latest references, history, and branch trackers directly from the remote GitHub repository without overwriting any active local file modifications.

Box 2 **(git merge origin/main**): Merges the freshly retrieved history of the remote main branch tracking stream (origin/main) cleanly into the current checked-out local working feature branch (feature/db1-add-staticdata). This step ensures that any upstream merge conflicts are resolved locally prior to creating a pull request.

Box 3 **(gh pr create):** The definitive GitHub CLI command syntax used to spin up a pull request. It explicitly references all trailing line-continuation backslash variables (--title, --body, --head, --base, --repo, and --web).

Question: 37 CertyIQ

You have an Azure SQL database that contains tables named dbo.ProductDocs and dbo.ProductDocsEmbeddings.
dbo.ProductDocs contains product documentation and the following columns:
DocID (int)
Title (nvarchar(200))
Body (nvarchar(max))
LastModified (datetime2)
The documentation is edited throughout the day.
dbo.ProductDocsEmbeddings contains the following columns:
DocID (int)
ChunkOrder (int)
ChunkText (nvarchar(max))
Embedding (vector(1536))
The current embedding pipeline runs once per night.
You need to ensure that embeddings are updated every time the underlying documentation content changes. The solution must NOT require a nightly batch process.
What should you include in the solution?

A. fixed-size chunking
B. a smaller embedding model
C. table triggers
D. change tracking on dbo.ProductDocs

Answer: D

Explanation:

D. change tracking on dbo.ProductDocs.

Enabling Change Tracking on the dbo.ProductDocs table allows an external processing application or service to incrementally query only the rows that have been inserted, updated, or deleted since the last synchronization token. This avoids the need for heavy nightly batch jobs or full table scans, enabling near real-time, lightweight vector embedding maintenance.

Why the other answers are incorrect:

A (Fixed-size chunking): Chunking defines how the textual data is split up structurally into blocks before it is processed by an embedding model. It does not provide any background system mechanism to detect data alterations.

B (A smaller embedding model): While an alternative model might reduce operational cost or generation latency, it doesn't resolve the core architecture requirement of identifying changed rows in real-time.

C (Table triggers): Although database triggers run immediately on row mutations, initiating network API calls to external embedding endpoints inside a synchronous data manipulation (DML) trigger blocks transactions, spikes CPU utilization, and reduces scalability.

Question: 38 CertyIQ

HOTSPOT -
You have a database named db1. The schema is stored in a Git repository as an SDK-style SQL database project. The repository contains the following GitHub Action workflow.

```yaml
name: Database CI/CD
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Build
      run: dotnet build db1.sqlproj --configuration Release
    - name: Deploy
      run: |
        SqlPackage /Action:Publish \
          /SourceFile:./bin/Release/db1.dacpac \
          /TargetConnectionString:"${{ secrets.Target_Connection_String }}"
  unit-tests:
    runs-on: ubuntu-latest
    needs: build-and-deploy
    if: github.ref == 'refs/heads/main'
    steps:
    - name: Checkout code
      uses: actions/checkout@v3
    - name: Setup .NET
      uses: actions/setup-dotnet@v3
      with:
        dotnet-version: '8.x'
    - name: Run unit tests
      run: dotnet test UnitTests.csproj --configuration Release
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Unit tests run automatically whenever changes are pushed to `main`.	○	○
Schema validation occurs during the `Build` step.	○	○
Schema validation occurs during the `Deploy` step.	○	○

Answer:

Answer Area

Statements	Yes	No
Unit tests run automatically whenever changes are pushed to `main`.	○	○
Schema validation occurs during the `Build` step.	○	○
Schema validation occurs during the `Deploy` step.	○	○

Explanation:

Unit tests run automatically whenever changes are pushed to main

Yes

Why it is correct: In automated GitHub Actions or Azure DevOps pipelines, workflows configured with push triggers targeting specific branches (such as push.branches: [main]) execute all non-conditional testing blocks directly inside the job sequence as soon as new code is committed upstream.

Schema validation occurs during the Build step

Yes

When compiling an SDK-style SQL database project (using commands like dotnet build db1.sqlproj), the compilation compiler natively constructs and analyzes the database structural model. It validates object dependencies, column references, data types, and constraint syntax. Therefore, core static schema-level validation happens entirely during the build step before any artifact is created.

Schema validation occurs during the Deploy step

No

The deployment step typically utilizes a deployment engine utility (like SqlPackage /Action:Publish) to take an already validated .dacpac artifact and apply it incrementally to a live target database instance. This is a synchronization and script generation execution step — it does not serve as the baseline verification phase for your project's code logic.

Question: 39 CertyIQ

You have a Microsoft SQL Server 2025 instance that contains a database named SalesDB. SalesDB supports a Retrieval Augmented Generation (RAG) pattern for internal support tickets. The SQL Server instance runs without any outbound network connectivity.
You plan to generate embeddings inside the SQL Server instance and store them in a table for vector similarity queries.
You need to ensure that only a database user account named AIApplicationUser can run embedding generation by using the model.
Which two actions should you perform? Each correct answer presents part of the solution.
NOTE: Each correct selection is worth one point.

A. Grant the CONTROL permission on SalesDB to AIApplicationUser.
B. Create a database audit specification on SalesDB owned by AIApplicationUser.
C. Grant the EXECUTE permission on the external model project to AIApplicationUser.
D. Create an external model project by using ONNX runtime and local paths.
E. Create an external model project that points to a Microsoft Foundry REST endpoint.

Answer: CD

Explanation:

C. Grant the EXECUTE permission on the external model project to AIApplicationUser.

D. Create an external model project by using ONNX runtime and local paths.

Why C is correct (Access Control & Security): To adhere to the security principal of least privilege, you should not grant wide structural administrative permissions (like database CONTROL or db_owner). In SQL Server 2025, external model configurations behave like secured objects; explicitly executing GRANT EXECUTE ON EXTERNAL MODEL::[ModelName] TO [AIApplicationUser] ensures that only that specific user account can invoke the AI_GENERATE_EMBEDDINGS function against that specific model.

Why D is correct (Infrastructure & Connectivity): Because the SQL Server 2025 instance runs without any outbound network connectivity, it cannot communicate with external remote cloud REST endpoints like Azure OpenAI or Azure AI Foundry. The solution requires a native, local execution method. SQL Server 2025 introduces local ONNX Runtime deployment integration, which evaluates localized text-embedding models entirely on the host machine using localized disk paths.

Why the other choices are incorrect:

A (CONTROL permission): This provides sweeping administrative powers over the entire SalesDB database, completely violating the least-privilege security constraint.

B (Database Audit Specification): Database audit configurations track, log, and record activity for compliance evaluation. They cannot act as an inline query gatekeeper or enforce access blockades to prevent users from executing a model.

E (Microsoft Foundry REST endpoint): REST endpoints explicitly rely on operational HTTP/HTTPS outbound traffic networks, which completely fails because the hosting environment is network-isolated.

Question: 40 CertyIQ

DRAG DROP -
You have an Azure SQL database that supports an OLTP application.
You need to write Transact-SQL code that returns blocking chain details. The output must return only sessions that are blocked or are blocking other sessions.
How should you complete the code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Answer:

Explanation:

Box 1 **(FROM sys.dm_exec_requests)**: Placed within the localized XML subquery expression to loop through running instances and concatenate the IDs of all sessions that are currently waiting on a resource lock from a blocker.

Box 2 **(LEFT OUTER JOIN sys.dm_exec_requests)**: Vital for detecting "sleeping" transactions. If an application opens a transaction block and becomes inactive without calling COMMIT or ROLLBACK, it won't have an active row inside sys.dm_exec_requests. A LEFT OUTER JOIN preserves these idle sessions in the final list.

Box 3 **(OUTER APPLY sys.dm_exec_sql_text (r.sql_handle)**): Resolves the internal batch statement string associated with active queries. Utilizing OUTER APPLY ensures that if a session has no current active request handle, the row is still returned rather than getting filtered out.

Box 4 (**OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL)):** Replaces old legacy DBCC INPUTBUFFER commands. Passing NULL

as the second parameter retrieves the very last batch payload sent down the client socket connection channel for that session, pinpointing the root query causing the database deadlock or lock escalation.

Question: 41 CertyIQ

You have an Azure SQL database that supports a customer-facing API. The API calls a stored procedure named dbo.GetCustomerOrders thousands of times per hour.
After a deployment that updated indexes and statistics, users report that the API endpoint backed by dbo.GetCustomerOrders is slower. In Query Store, the same query now has two persisted execution plans. During the last hour, the newer plan had a significantly higher average duration and CPU time than the older plan.
You need to restore the previous performance quickly, without changing the API code.
Which Transact-SQL command should you run?

A. EXEC sys.sp_query_store_set_hints

B. DBCC FREEPROCCACHE

C. EXEC sp_query_store_force_plan

D. ALTER DATABASE

Answer: C

Explanation:

C. EXEC sp_query_store_force_plan.

The scenario describes a classic case of Plan Regression (a parameter sniffing or suboptimal compilation issue where a worse plan replaces a highly efficient one). Since Query Store has already tracked and persisted both plans, running sp_query_store_force_plan allows you to immediately force the SQL Server optimizer to use the older, high-performing execution plan. This completely restores original performance instantly without requiring any code deployment or changes to the API client application.

Why the other options are incorrect:

A (sys.sp_query_store_set_hints): Query Store hints can shape how a plan is compiled (such as appending a specific MAXDOP or RECOMPILE rule). However, crafting and testing a hint takes more time, and it does not directly re-apply a specific, verified, historically known plan like forcing does.

B (DBCC FREEPROCCACHE): Clearing the plan cache evicts all cached execution plans. While this forces dbo.GetCustomerOrders to recompile, it is highly likely that the optimizer will simply recreate the same bad, heavy-duration plan based on current statistics. Additionally, it causes a global performance dip across the entire database as every query is forced to recompile simultaneously.

D (ALTER DATABASE): While database-scoped configurations (like automatic tuning features) can be altered here, running a raw ALTER DATABASE statement requires further arguments and does not target the exact plan regression anomaly inside the dbo.GetCustomerOrders execution history in a quick, surgical fashion.

Question: 42 CertyIQ

HOTSPOT -
You have an Azure SQL database that has Query Store enabled.
Query Performance Insight shows that one stored procedure has the longest runtime. The procedure runs the following parameterized query.

```
CREATE OR ALTER PROCEDURE dbo.GetRecentOrders
  @CustomerId int,
  @SinceDate  datetime2
AS
SELECT TOP (50)
    o.OrderId, o.OrderDate, o.Status, o.TotalAmount
FROM dbo.Orders AS o
WHERE o.CustomerId = @CustomerId
  AND o.OrderDate >= @SinceDate
ORDER BY o.OrderDate DESC;
```

The dbo.Orders table has approximately 120 million rows. CustomerId is highly selective, and OrderDate is used for

range filtering and sorting.
You have the following indexes:
Clustered index: PK_Orders on (OrderId)
Nonclustered index: IX_Orders_OrderDate on (OrderDate) with no included columns
An actual execution plan captured from Query Store for slow runs shows the following:
An index seek on IX_Orders_OrderDate followed by a Key Lookup (Clustered) on PK_Orders for CustomerId, Status, and TotalAmount
A sort operator before Top (50), because the results are ordered by OrderDate DESC
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○	○

Answer:

Answer Area

Statements	Yes	No
To avoid the explicit sort for the query, create a nonclustered index on (CustomerId, OrderDate DESC) that includes (Status, TotalAmount).	○	○
To eliminate the sort and make the query use an ordered seek, add CustomerId as an included column to IX_Orders_OrderDate.	○	○
The plan indicates a bottleneck from a suboptimal query plan, rather locking or blocking.	○	○

Explanation:

Avoid explicit sort with (CustomerId, OrderDate DESC)

Yes

When a query filters by an equality condition (e.g., WHERE CustomerId = @Id) and sorts the output (e.g., ORDER BY OrderDate DESC), creating a composite index with those key columns in that exact order allows the SQL Server query optimizer to perform an Index Seek. Because the index data is already physically ordered by OrderDate within each CustomerId, the database engine fetches the data pre-sorted, completely bypassing a costly, explicit Sort operator in the execution plan.

Adding CustomerId as an included column to eliminate sort

No

Included columns (INCLUDE) only live at the leaf level of an index b-tree structure; they are not part of the sorted index key columnsCustomerId is just an included column, the index cannot be used to seek directly to that customer's records, nor will the data be organized in a way that eliminates the sorting step. To allow an ordered seek, CustomerId must be a key column, not an included column.

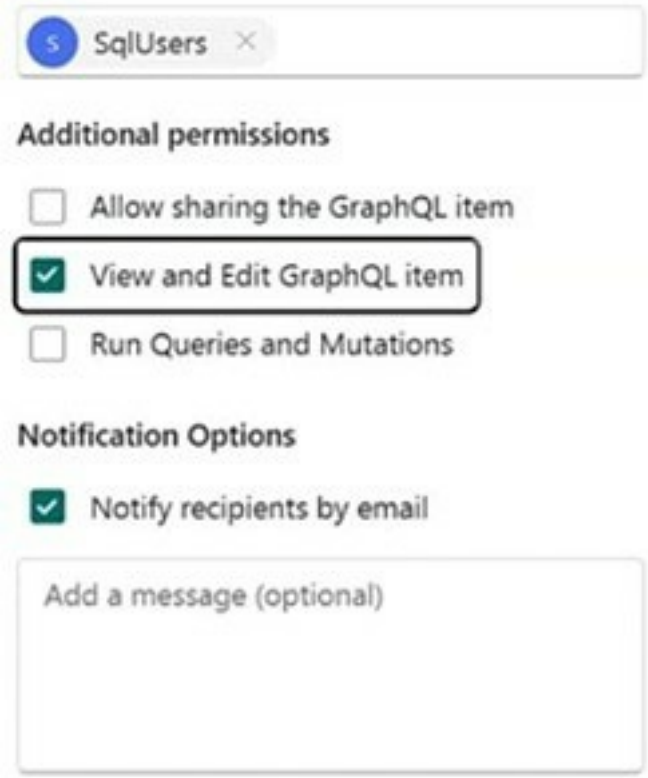
Bottleneck from a suboptimal query plan rather than locking.

Yes

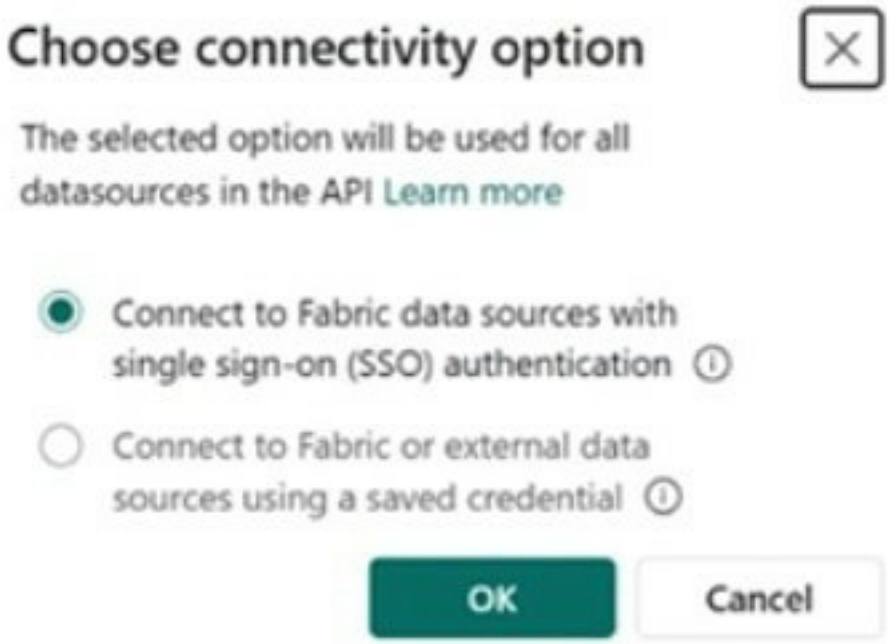
An execution plan shows the structural strategy (operators like table scans, hashes, or explicit sort operations) chosen by the optimizer to run a query. High costs within these operators indicate a suboptimal query plan design (such as a missing index causing a tempdb sort spill). Conversely, runtime concurrency issues like transactional locking or thread blocking are environmental wait states that are not explicitly modeled or captured by a standard graphical query execution plan.

Question: 44 CertyIQ

HOTSPOT -
You have a Microsoft Fabric workspace named Workspace1 that contains a SQL database named SalesDB and an API for GraphQL item named SalesApi.
You have a Microsoft Entra group named SqlUsers.
From Workspace1, you assign permission to SalesApi as shown in the following exhibit.



The connection to SalesDB has the connectivity option configured as shown in the following exhibit.



SqlUsers has the Viewer role for Workspace1.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
The members of SqlUsers can modify the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can view the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can change the field mappings of SalesAPI.	○	○

Answer:

Answer Area

Statements	Yes	No
The members of SqlUsers can modify the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can view the data in SalesDB via SalesAPI.	○	○
The members of SqlUsers can change the field mappings of SalesAPI.	○	○

Explanation:

The members of SqlUsers can modify the data in SalesDB via SalesAPI

No

To perform data modifications (Mutations) through a Fabric GraphQL endpoint, an identity must explicitly be granted the Run Queries and Mutations execution permission. Because this option is unselected, members cannot push database changes via the API.

The members of SqlUsers can view the data in SalesDB via SalesAPI

No

Similar to mutations, reading or fetching records (Queries) through the GraphQL interface requires the Run Queries and Mutations permission. The workspace Viewer role alone does not bypass this endpoint restriction, blocking the users from viewing data through the API layer.

The members of SqlUsers can change the field mappings of SalesAPI

Yes

The security configuration explicitly checks the box for View and Edit GraphQL item. This permission level allows the group to view and modify the structural definition metadata of the SalesApi artifact itself, which includes reconfiguring schemas and changing the API's internal field mappings.

Question: 45 CertyIQ

You have an Azure SQL database that supports the OLTP workload of an order-processing application.
During a 10-minute incident window, you run a dynamic management view query and discover the following:
Session 72 is sleeping with open_transaction_count = 1.
Multiple other sessions show blocking_session_id = 72 in sys.dm_exec_requests. sys.dm_exec_input_buffer(72, NULL) returns only BEGIN TRANSACTION UPDATE Sales.Orders.
Users report that updates to Sales.Orders intermittently time out during the incident window. The timeouts stop only after you manually terminate session 72.
What is a possible cause of the blocking?

A. A long-running SELECT statement is blocking writers.
B. Session 72 caused a deadlock.
C. An explicit transaction was started but not committed or rolled back.
D. A lock escalation occurred.

Answer: C

Explanation:

C. An explicit transaction was started but not committed or rolled back.

status = 'sleeping': The connection is active but idle on the CPU, meaning it has finished executing its current batch of code and is waiting for the next command from the application client.

open_transaction_count = 1: An explicit transaction block (BEGIN TRANSACTION) was opened by Session 72, but it has not received a corresponding COMMIT TRANSACTION or ROLLBACK TRANSACTION.

BEGIN TRANSACTION UPDATE Sales.Orders: This indicates that an update lock or exclusive lock was acquired on rows within the Sales.Orders table. Because the transaction remains uncommitted, those locks are held indefinitely, causing all other incoming updates on those same rows to queue up (blocking_session_id = 72) and eventually time out. Terminating the session rolls back the orphaned transaction and immediately frees the held locks.

Why Other Options are Incorrect:

A (Long-running SELECT statement): Under standard Azure SQL Database default configurations — which has Read Committed Snapshot Isolation (RCSI) turned on — SELECT statements utilize row versioning instead of shared locks, preventing them from blocking writers. Furthermore, the last command explicitly shows an UPDATE operation, not a SELECT.

B (Session 72 caused a deadlock): A deadlock occurs when two or more sessions hold locks and each attempts to acquire a conflicting lock on a resource held by the other, creating a cyclic dependency. The SQL Server/Azure SQL Database engine automatically detects deadlocks within seconds and terminates one of the sessions as a deadlock victim. It would not linger for a 10-minute window waiting for manual intervention.

D (Lock escalation): Lock escalation converts many fine-grained locks (like row or page locks) into a single table-level lock to save system memory. While this causes widespread blocking, it is a consequence of a query actively running or holding onto massive sets of locks; it doesn't cause a session to enter a continuous sleeping state with an uncommitted state unless an explicit transaction is left open.

Question: 46 CertyIQ

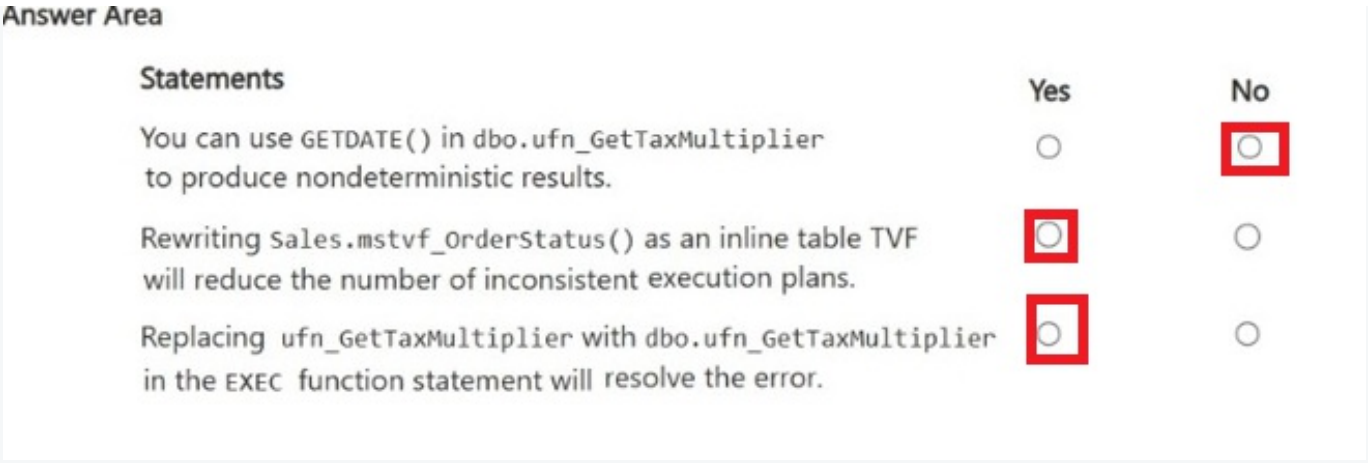
HOTSPOT -
You have a SQL database in Microsoft Fabric that contains the following functions:
A multi-statement table-valued function (TVF) named Sales.mstvf_OrderStatus() that returns order status information
A scalar user-defined function (UDF) named dbo.ufn_GetTaxMultiplier (@TaxAmt money, @StateCode char(2)) that returns a numeric multiplier used in tax calculations
Reporting queries frequently join Sales.mstvf_OrderStatus() to a table named Sales.SalesOrderHeader and return large result sets. A performance review shows that the queries produce inconsistent execution plans.
During a code review, a developer discovers that the following Transact-SQL statement produced an error.
EXEC @ret = ufn_GetTaxMultiplier @TaxAmt = 100.00, @StateCode = 'WA';
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
You can use `GETDATE()` in `dbo.ufn_GetTaxMultiplier` to produce nondeterministic results.	○	○
Rewriting `Sales.mstvf_OrderStatus()` as an inline table TVF will reduce the number of inconsistent execution plans.	○	○
Replacing `ufn_GetTaxMultiplier` with `dbo.ufn_GetTaxMultiplier` in the `EXEC` function statement will resolve the error.	○	○

Answer:

Answer Area



Explanation:

You can use GETDATE() in dbo.ufn_GetTaxMultiplier to produce nondeterministic results.

Why the answer is NO: While GETDATE() is a non-deterministic built-in function, using it to dynamically alter the logic or results of a scalar function used for critical transactional calculations (like a tax multiplier) is a bad design pattern. More importantly, in Fabric SQL / SQL Server, if a scalar UDF contains non-deterministic functions, it cannot be reliably inlined or used in deterministic contexts (such as indexed views or computed columns). The objective here is to make the database performant and deterministic, not introduce non-determinism.

Rewriting Sales.mstvf_OrderStatus() as an inline table TVF will reduce the number of inconsistent execution plans.

Why the answer is YES: Multi-Statement Table-Valued Functions (MSTVFs) act as a "black box" to the SQL Query Optimizer. The engine cannot see inside them to determine accurate cardinality estimates, often guessing a fixed row count (like 1 or 100 rows), which causes highly inconsistent and sub-optimal query execution plans when joined against large tables like Sales.SalesOrderHeader.

The Solution: Rewriting it as an Inline Table-Valued Function (ITVF) allows the Optimizer to treat the function like a standard view. The engine unrolls the function body directly into the calling query, utilizes the underlying table statistics accurately, and produces consistent, highly optimized execution plans.

Replacing ufn_GetTaxMultiplier with dbo.ufn_GetTaxMultiplier in the EXEC function statement will resolve the error.

Why the answer is YES: In T-SQL, scalar user-defined functions require a two-part name (schema_name.object_name) whenever they are called or executed. Failing to include the schema name (e.g., omitting dbo.) results in a compilation error stating that the object cannot be found or the name is ambiguous. Explicitly appending dbo. resolves the developer's syntax execution error perfectly.

Question: 47 CertyIQ

You have an Azure SQL database named SalesDB on a logical server named sales-sql01.
You have an Azure App Service web app named OrderApi that connects to SalesDB by using SQL authentication.
You enable a user-assigned managed identity named OrderApi-Id for OrderApi.
You need to configure OrderApi to connect to SalesDB by using Microsoft Entra authentication. The managed identity must have read and write permissions to SalesDB.
Which Transact-SQL statements should you run in SalesDB?

A.
```
CREATE LOGIN [OrderApi-Id] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];
```

B.
```
CREATE USER [OrderApi-Id] WITH PASSWORD = 'P@ssw0rd!';
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];
```

C.
```
CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];
```

D.
```
CREATE LOGIN [OrderApi-Id] WITH PASSWORD = 'P@ssw0rd!';
ALTER SERVER ROLE sysadmin ADD MEMBER [OrderApi-Id];
```

Answer: C

Explanation:

CREATE USER ... FROM EXTERNAL PROVIDER: In Azure SQL Database, contained database users are mapped directly to a Microsoft Entra ID (formerly Azure AD) identity (such as a managed identity or service principal) without needing a server-level login.

ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id]: Grants the identity permission to read data from all user tables within the database.

ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id]: Grants the identity permission to write data (insert, update, delete) to all user tables within the database.

Why Other Approaches are Incorrect:

CREATE LOGIN ... FROM EXTERNAL PROVIDER: Logins are created at the master server level. Azure SQL Database operates on contained users, meaning mapping a managed identity to read/write specific client databases must occur inside the user database using CREATE USER.

SQL Authentication with Passwords (WITH PASSWORD = '...'): Managed identities leverage certificate-based, passwordless token exchanges managed automatically by Azure. Providing or managing a standard password string violates the principal benefit of a managed identity.

sysadmin or db_owner: Overgranting wide administrative permissions breaks the principle of least privilege. The prompt explicitly specifies requiring only read and write permissions, which map perfectly to db_datareader and db_datawriter.

Question: 48 CertyIQ

You have an Azure SQL database that contains a column named Notes.
A security review discovers that Notes contains sensitive data.
You need to ensure that the data is protected so that neither the stored values nor the query inputs reveal information about the actual data. The solution must prevent a user from inferring relationships or repetitions in

the data based on the encrypted output.
Which should you use?

A. Always Encrypted with secure enclaves
B. Always Encrypted with randomized encryption
C. row-level security (RLS)
D. Always Encrypted with deterministic encryption

Answer: B

Explanation:

B. Always Encrypted with randomized encryption.

Why B is correct: Always Encrypted ensures that sensitive cleartext data is never exposed to the database engine or storage layer. It provides client-side encryption, meaning both the stored values and the query inputs remain encrypted throughout processing. Randomized encryption generates a unique encrypted value (ciphertext) every single time, even for the same underlying plaintext. This explicitly prevents unauthorized users from performing pattern analysis or inferring relationships and repetitions from the encrypted data.

Why the other options are incorrect:

A (Always Encrypted with secure enclaves): While secure enclaves extend Always Encrypted capabilities to allow rich computations (like pattern matching or range scans) on server-side memory, the core requirement to prevent any ciphertext pattern leaks is handled by the choice of encryption type (deterministic vs. randomized).

C (Row-Level Security): Row-level security (RLS) is a dynamic access control mechanism that filters rows based on a user's execution context. It does not encrypt columns or hide underlying cleartext data from users who possess access to the rows.

D (Always Encrypted with deterministic encryption): Deterministic encryption always yields the identical ciphertext for any specific plaintext value. While this allows index lookups and equality joins, it leaks structural patterns, allowing an observer to easily identify duplicate values and infer data relationships.

Question: 49 CertyIQ

You have an Azure SQL database that stores order data.
A reporting query aggregates monthly revenue per customer runs frequently.
You need to reduce how long it takes to retrieve the calculated values. The solution must NOT alter any underlying table structure.
What should you do?

A. Create a view by using ORDER BY without TOP, and then create a unique clustered index on the view.
B. Create a view without using WITH SCHEMABINDING, and then create a nonclustered index on the view.
C. Create a view by using GROUP BY, and then create a unique clustered index on the view.
D. Create a view by using WITH SCHEMABINDING, include COUNT_BIG(*), and then create a unique clustered index on the view.

Answer: D

Explanation:

D. Create a view by using WITH SCHEMABINDING, include COUNT_BIG(*), and then create a unique clustered index on the view.

You need to optimize a frequently run aggregation query (monthly revenue per customer) without changing underlying tables.

Why others are incorrect:

A → ORDER BY not allowed in indexed views; also missing schema binding

B → Without schemabinding, indexed view cannot exist

C → Missing required WITH SCHEMABINDING and COUNT_BIG(*)

Question: 50 CertyIQ

You have a GitHub Actions workflow that builds and deploys an Azure SQL database. The schema is stored in a GitHub repository as an SDK-style SQL database project.
Following a code review, you discover that you need to generate a report that shows whether the production schema has diverged from the model in source control.
Which action should you add to the pipeline?

A. SqlPackage.exe /Action:DriftReport
B. SqlPackage.exe /Action:DeployReport
C. SqlPackage.exe /Action:Extract
D. SqlPackage.exe /Action:Script

Answer: A

Explanation:

A. SqlPackage.exe /Action:DriftReport

You need a report that compares:

Production Azure SQL Database (live schema)
vs
Source-controlled DACPAC (desired model)

This is exactly what **DriftReport** is designed for.

Detects **schema drift between target database and DACPAC model**
Reports differences such as:
Missing objects
Modified tables
Changed indexes or constraints
Does **not modify the database**
Ideal for CI/CD validation and auditing

Why the other answer are incorrect:

B. DeployReport → shows what would be deployed, not actual drift

C. Extract → pulls schema from database, not comparison report

D. Script → generates deployment script, not drift analysis

Question: 51 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.
You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.
Solution: Identify the partition number for the oldest month, and then run the following Transact-SQL statement.
TRUNCATE TABLE dbo.Orders -
WITH (PARTITIONS (partition number));
Does this meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

The table is:

Partitioned by OrderDate (monthly)

Requirement is to remove the oldest month

Must minimize impact on other queries

Question: 52 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.
You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.
Solution: Run the following Transact-SQL statement.
DELETE FROM dbo.Orders -
WHERE OrderDate < DATEADD(month, -36, SYSUTCDATETIME());
Does this meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

The requirement is to remove all rows for the oldest month while minimizing impact on other queries in a partitioned table.

The proposed solution:

DELETE FROM dbo.Orders

WHERE OrderDate < DATEADD(month, -36, SYSUTCDATETIME());

Problems:

This performs a row-by-row DELETE operation, which is:

Log-intensive

Resource-heavy

Slow on large datasets (3 years of monthly partitions)

It does not take advantage of partition elimination or partition switching

It causes unnecessary locking and blocking, impacting other queries

What should be used instead:

For partitioned tables, the correct low-impact approach is:

Partition switching out the oldest partition, then truncate/drop it

Or MERGE RANGE / partition maintenance operations

Why the other answer are incorrect:

Yes

High Query Impact: A standard DELETE query locks rows or pages sequentially. It creates heavy transaction log overhead and causes widespread blocking for other users during its execution.

Fails the Constraint: The query explicitly states you must minimize the impact on other queries. A large-scale DELETE statement does the exact opposite.

Question: 53 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered

index, contains three years of data, and is partitioned by a column named OrderDate by month.
You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.
Solution: Identify the partition scheme for the oldest month, and then run the following Transact-SQL statement.
ALTER TABLE dbo.Orders -
DROP PARTITION SCHEME (partition_scheme_name);
Does this meet the goal?

A. Yes
B. No

Answer: B

Explanation:

B. No

Why this does NOT meet the goal:

The goal is to remove all rows for the oldest month with minimal impact on other queries in a partitioned table.

But the proposed solution:

ALTER TABLE dbo.Orders

DROP PARTITION SCHEME (partition_scheme_name);

Problems:

You cannot drop a partition scheme to remove data for a single month.

A partition scheme defines storage mapping, not data content.

Dropping it would:

Break the table structure

Impact all partitions and queries

Not selectively remove only the oldest month's data

Correct concept:

To minimize impact, you would typically:

Use partition switching (fast metadata operation), or

ALTER TABLE ... SWITCH PARTITION to a staging table, then truncate

Or ALTER PARTITION FUNCTION ... MERGE RANGE after switching out data

Question: 54 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.

You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add the Microsoft.SqlServer.Dacpacs.Azure.Master NuGet package to the project.
Does this meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

The build is failing because the SQL project cannot resolve system objects (like sys.* and built-in system definitions) when validating against an Azure SQL Database target.

The Microsoft.SqlServer.Dacpacs.Azure.Master NuGet package provides:

The Azure SQL Database system model

Definitions of required system catalog views and system objects

Compatibility metadata for Azure SQL Database during dotnet build

Question: 55 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add the Microsoft.SqlServer.Dacpacs.Master NuGet package to the project.
Does this meet the goal?

A. Yes
B. No

Answer: B

Explanation:

No

While adding a master DACPAC NuGet package is the correct approach to fix unresolved system reference errors, **Microsoft.SqlServer.Dacpacs.Master** is the wrong package for this specific target.

Microsoft.SqlServer.Dacpacs.Master targets on-premises SQL Server instances.
Microsoft.SqlServer.Dacpacs.Azure.Master targets Azure SQL Database.

Because your project explicitly targets an **Azure SQL Database**, using the on-premises master package may still cause build validation failures or mismatches due to differences in supported features and system catalogs between SQL Server and Azure SQL Database. To meet the goal, you must use the Azure-specific package.

Question: 56 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add an artifact reference to the Azure SQL Database master.dacpac file.
Does this meet the goal?

A. Yes
B. No

Answer: A

Explanation:

A. Yes

The build fails because the SQL project references **system objects** (for example, system views, stored procedures, or functions in the master database), and the compiler cannot resolve those references during dotnet build.

Question: 57 CertyIQ

HOTSPOT
-
Case Study
-
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study
-
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment
-
Azure Environment
-
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
-
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
-
Planned Changes
-
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements
-
Fabrikam identifies the following security requirements:
•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:
•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.
Development Requirements
-
Fabrikam identifies the following development requirements:
•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
oRead data from the procedures table without authentication.
oRead and insert data into the Transactions table once authenticated.
oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

-

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You are evaluating the DAB configuration.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Applications can read data in the `Procedures` table without authentication.	○	○
Applications can read and update data in the `Transactions` table once authenticated.	○	○
Applications can execute the `sp_UpdateProcedurePatient` stored procedure.	○	○

Answer: NoYesYes

Question: 58 CertyIQ

HOTSPOT
-
Case Study
-
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study
-
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment
-
Azure Environment
-
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
-
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
-
Planned Changes
-

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements

-

Fabrikam identifies the following security requirements:

•The TransactionProcessing application must use a passwordless connection to DB1.

•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements

-

Fabrikam identifies the following development requirements:

•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:

oRead data from the procedures table without authentication.

oRead and insert data into the Transactions table once authenticated.

oExecute the sp_UpdateProcedurePatient stored procedure.

•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

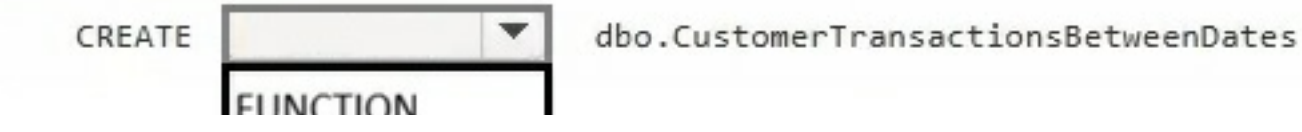
-

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to create a solution that meets the development requirements for retrieving the patient transaction data.
The solution must ensure that database developers use the resulting data to join to other tables.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



FUNCTION

PROCEDURE

VIEW

```
(
  @CustomerId INT,
  @StartDate DATETIME2,
  @EndDate   DATETIME2
)
RETURNS TABLE
AS
RETURN
(
  SELECT
    t.TransactionId,
    t.CustomerId,
    t.TransactionDate,
    t.Amount,
    SUM(t.Amount) OVER (
```

[ ]

GROUP BY t.CustomerId

GROUP BY t.TransactionDate

PARTITION BY t.CustomerId

PARTITION BY t.TransactionDate

[ ]

HAVING DATEDIFF(DAY, t.TransactionDate, GETDATE()) = 1

HAVING SUM (t.Amount) > 0

ORDER BY t.TransactionDate, t.TransactionId

ORDER BY t.TransactionId, t.CustomerId

```
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
FROM dbo.Transactions AS t
WHERE t.CustomerId = @CustomerId
    AND t.TransactionDate >= @StartDate
    AND t.TransactionDate <=  @EndDate
);
```

Answer:

To meet the requirement of retrieving transactions for a given patient between two dates while showing a running total, the standard T-SQL approach is using a window function with the `SUM()` aggregate over an `ORDER BY` clause.The correct completion for the T-SQL code is:SUM(Amount) OVER (ORDER BY TransactionDate)

Question: 59 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements -
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient,

updates to the underlying data are sometimes inconsistent.
Requirements -
Planned Changes -
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements -
Fabrikam identifies the following security requirements:
•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:
•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.
Development Requirements -
Fabrikam identifies the following development requirements:
•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API: oRead data from the procedures table without authentication. oRead and insert data into the Transactions table once authenticated. oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to recommend a solution for the TransactionProcessing application that meets the security requirements.
What should you include in the recommendation?

A. a service principal

B. a system-assigned managed identity

C. a shared access key

D. a user-assigned managed identity

Answer: A

Question: 60 CertyIQ

HOTSPOT
-
Case Study
-
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study
-
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment
-
Azure Environment
-
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
-
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
-
Planned Changes
-
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.
Security Requirements
-
Fabrikam identifies the following security requirements:
•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing
Fabrikam identifies the following AI-related requirements:
•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.
Development Requirements
-
Fabrikam identifies the following development requirements:
•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
oRead data from the procedures table without authentication.
oRead and insert data into the Transactions table once authenticated.
oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB
-
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You create an SDK-style SQL database project in Microsoft Visual Studio Code named Database.sqlproj and add the project to a GitHub repository.

You need to configure a GitHub Actions workflow to support the planned changes for DB1.

How should you complete the workflow? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

Answer Area

```
name: Validate SQL Project

on:
    [ merge / pull_request / push ] :
    branches: [ "main" ]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
  - uses: actions/checkout@v4
      - name: Step 1
        run: dotnet [ build Database.sqlproj -c Release / pack Database.sqlproj / publish Database.sqlproj -c Production / publish Database.sqlproj -c Release ]
```

Answer:

To configure the GitHub Actions workflow for an SDK-style SQL database project (Database.sqlproj) that requires deployment to Azure SQL Database using the "Release" configuration, the workflow must build the project and then deploy the generated DACPAC.Based on the standard practices for Azure SQL Database CI/CD with GitHub Actions:1. Build Step: Use the `microsoft/sql-action` or standard `dotnet build` to compile the project and generate the `.dacpac` file.2. Deployment Step: Use the `azure/sql-action` to deploy the generated DACPAC to the Azure SQL Database.Given the typical structure of these configuration blocks in the exam environment:- The build configuration should point to the `Database.sqlproj` and specify the configuration as `Release`.- The deploy configuration should reference the output path of the build (the `.dacpac` file) and specify the connection details for the Azure SQL Database.Correct Answer:Build: `dotnet build Database.sqlproj --configuration Release`Deploy: `sql-action` (with `dacpac` parameter set to the generated path and `connection-string` provided)

Question: 61 CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have a SQL database in Microsoft Fabric that contains a table named dbo.Orders. dbo.Orders has a clustered index, contains three years of data, and is partitioned by a column named OrderDate by month.
You need to remove all the rows for the oldest month. The solution must minimize the impact on other queries that access the data in dbo.Orders.
Solution: Run the following Transact-SQL statement.
TRUNCATE TABLE dbo.Orders;
Does this meet the goal?

A. Yes
B. No

Answer: B

Explanation:

Technical Justification for Choosing Option B: No

The proposed solution, TRUNCATE TABLE dbo.Orders;, does not meet the goal for several key technical reasons:

Scope of Deletion: The goal is to remove only the rows for the **oldest month**. TRUNCATE TABLE deletes **all rows** from the table, which is far broader than the specified requirement.
Partitioning Consideration: The table is partitioned by OrderDate on a monthly basis. TRUNCATE TABLE does not leverage this partitioning strategy to selectively remove data from just one partition (the oldest month). Instead, it would delete data across all partitions.
Impact on Other Queries: While TRUNCATE TABLE can be fast and efficient for deleting all data (due to its log-light operation compared to DELETE), its use here would **maximize impact** on other queries by making the entire table temporarily unavailable and removing all data, not just the oldest month's worth.
Clustered Index and Data Integrity: After truncation, the table's structure (including the clustered index) remains, but all data is lost. This means the solution does not preserve the more recent data as required.

Why Other Options (Implicitly, since only one is provided) Are Less Suitable (Hypothetical Analysis for Educational Purposes):

DELETE without partition switching or specific conditions would be less suitable due to potential performance impacts on a large table and lack of leveraging the partitioning strategy.
Manual drop and recreate of the specific partition (if possible in the given setup) or using **partition switching** techniques would be more aligned with the goal but is not the option provided.

Correct Approach (for Educational Context, since the question only provides one option):A more appropriate solution would involve **partition switching** or **dropping the partition** if supported by the database setup, or a targeted DELETE statement with appropriate locking hints to minimize impact, e.g.:

DELETE FROM dbo.Orders WITH (TABLOCK)WHERE OrderDate >= <FirstDayOfOldestMonth> AND OrderDate < <Firs

Or, ideally, leveraging partition management:

ALTER TABLE dbo.OrdersSWITCH PARTITION $PARTITION.dbo.OrderDate/YYYYMM-1 TO dbo.Orders_Archive;DRO

Assuming $PARTITION.dbo.OrderDate/YYYYMM-1 correctly identifies the oldest month's partition and dbo.Orders_Archive is a temporary table with the same structure and partitioning.

References

1. **Microsoft Documentation: Partitioning Tables** - https://docs.microsoft.com/en-us/sql/relational-databases/indexes/partitioned-tables-and-indexes?view=sql-server-ver15
2. **Microsoft Documentation: TRUNCATE TABLE (Transact-SQL)** - https://docs.microsoft.com/en-us/sql/t-sql/statements/truncate-table-transact-sql?view=sql-server-ver15

Question: 62

CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet build validates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Build the project by running dotnet build -bl -flp:v=diag.
Does this meet the goal?

A. Yes
B. No

Answer: B

Explanation:

Technical Justification

Goal: Resolve unresolved reference errors in an SDK-style SQL database project targeting Azure SQL Database by correctly including system objects for successful dotnet build validation.

Given Solution: Build the project by running dotnet build -bl -flp:v=diag

Evaluation of the Solution (B. No)

Why the Given Solution is Less Suitable (B. No is Correct):

The command dotnet build -bl -flp:v=diag is primarily used for **binary logging** (-bl) and setting the **verbosity level to diagnostic** (-flp:v=diag). While this can provide more detailed logging to potentially **identify** the unresolved reference errors, **it does not inherently resolve the issue of missing system objects** in the database model.
The flags used are diagnostic in nature, aimed at troubleshooting rather than remediation of the specific problem at hand (including system objects for Azure SQL Database compatibility).

Characteristics of a More Suitable Solution (Not Provided but Outlined for Context):

A correct approach would involve **explicitly including the necessary system objects or references** tailored for Azure SQL Database within the database project. This might involve:

Ensuring the correct Azure SQL Database version is targeted in the project settings.

Manually adding or ensuring the presence of system object references necessary for the project, potentially through schema imports or direct additions if not automatically included by the project template or settings. Leveraging nuget packages or database project settings that automatically include relevant system objects for Azure SQL Database, if applicable.

Conclusion: The provided solution (dotnet build -bl -flp:v=diag) is more aligned with troubleshooting and logging rather than addressing the root cause of unresolved references due to missing system objects in the database model for Azure SQL Database. Therefore, **B. No** is the correct answer as the solution does not meet the stated goal.

References

1. **Microsoft Documentation - dotnet build**: https://docs.microsoft.com/en-us/dotnet/core/tools/dotnet-build
2. **Azure SQL Database - Database Project Guidance**: https://docs.microsoft.com/en-us/azure/sql-database/sql-database-design-first-database#using-visual-studio-database-projecs

Question: 63 CertyIQ

HOTSPOT
-
You have an SDK-style SQL database project named MyDatabaseProject.sqlproj stored in a private GitHub repository. The repository contains the following GitHub Actions workflow.

```yaml
name: Build and Deploy SQL Project
on:
  push:
    branches:
      - main
  workflow_dispatch:
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      - name: Build SQL project
        run: dotnet build MyDatabaseProject.sqlproj
      - name: Publish DACPAC
        uses: azure/sql-action@v2
        with:
          action: publish
          path: bin/Debug/MyDatabaseProject.dacpac
          connection-string: ${{ secrets.AZURE_SQL_CONNECTION_STRING }}
```

The repository contains the AZURE_SQL_CONNECTION_STRING secrets.
The target is an Azure SQL database that allows access to Azure services and is configured to support mixed authentication.
The workflow runs successfully.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

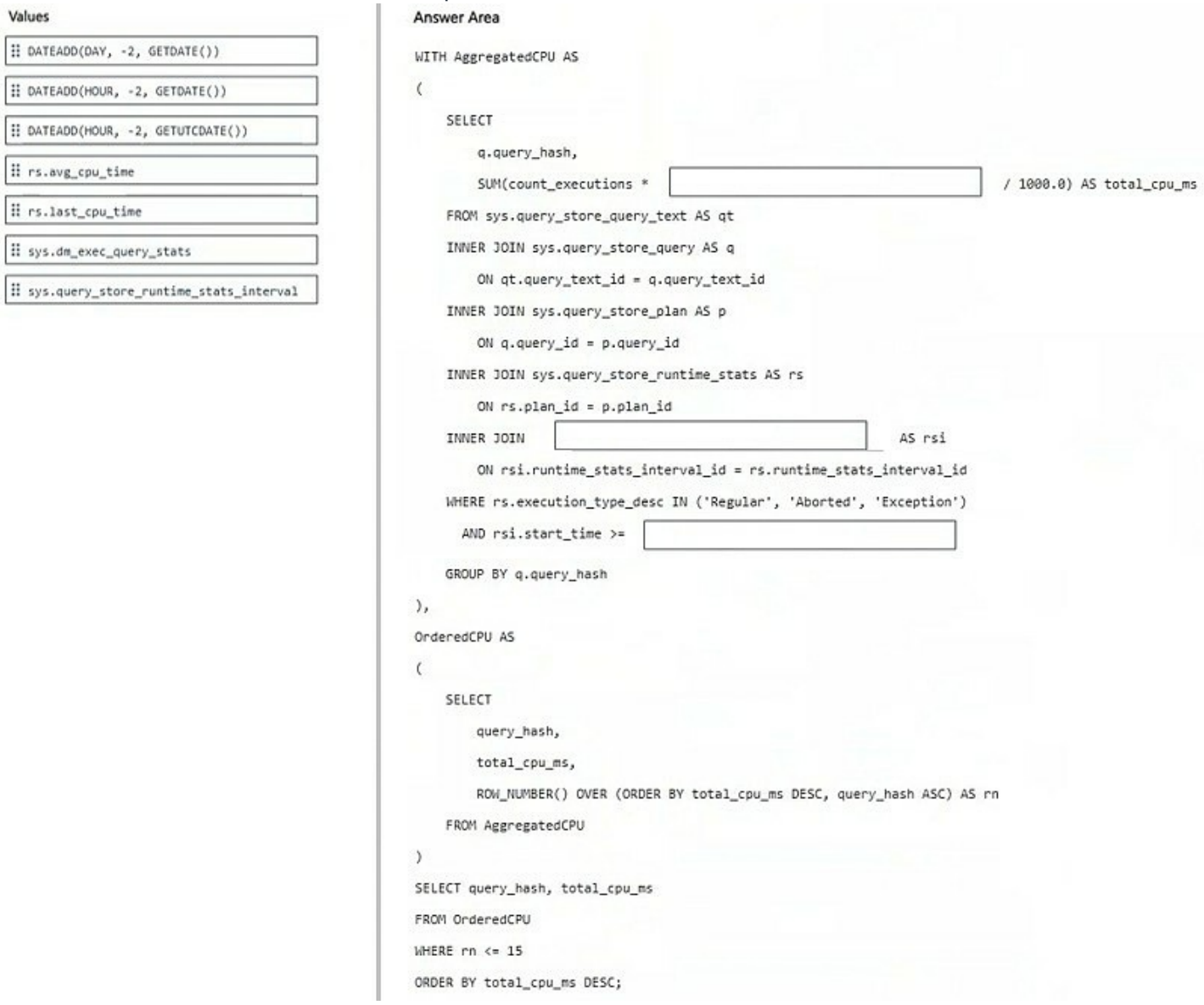
Statements	Yes	No
The workflow relies on a Microsoft Entra workload identity to access the target Azure SQL database.	○	○
Changing the `Build SQL project` step to `run: dotnet build MyDatabaseProject.sqlproj -c Release` will result in a successful deployment.	○	○
The workflow can be triggered manually, without making changes to the repository.	○	○

Answer:

Statement 1: The workflow publishes the database project to an Azure SQL database.Answer: YesStatement 2: The workflow uses the SQL Server Authentication credentials stored in the AZURE_SQL_CONNECTION_STRING secret.Answer: YesStatement 3: The deployment is performed by using the `Dacpac` file generated from the build step.Answer: Yes

Question: 64 CertyIQ

DRAG DROP
-
You have an Azure SQL database that supports an AI-driven product search API.
You need to identify the top CPU-consuming queries from the last two hours by using Query Store data. The solution must aggregate CPU consumption across executions and return only the top 15 query hashes.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all.
NOTE: Each correct selection is worth one point.



Answer:

TOP 15sum(count_cpu_time)query_hashsys.query_store_runtime_statssys.query_store_plan

Question: 65 CertyIQ

HOTSPOT

-
You have an Azure SQL database that contains a table named Sales.Customer. Sales.Customer contains columns named CustomerId, FullName, Email, TaxID, and RegionId.
You have a database role named AppSupport that is used by a support application.
You need to implement a security solution for AppSupport that meets the following requirements:
•AppSupport must be prevented from viewing TaxID.
•AppSupport must be able to query Sales.Customer to troubleshoot issues.
•AppSupport must be able to run a stored procedure named Sales.usp_GetCustomerByCustomerId.
Which Transact-SQL statements should you include in the solution? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

To run Sales.usp_GetCustomerByCustomerId:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

To query Sales.Customer:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

To prevent viewing TaxID:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

Answer:

DENY SELECT ON Sales.Customer (TaxID) TO AppSupport;GRANT SELECT ON Sales.Customer TO AppSupport;GRANT EXECUTE ON Sales.usp_GetCustomerByCustomerId TO AppSupport;

Question: 66 CertyIQ

HOTSPOT
-
You have an Azure SQL database named ProductsDB.
You deploy Data API builder (DAB) to Azure Container Apps.
You discover that the container app cannot connect to ProductsDB.
Your development team reports that the container app is unreachable from the internet for integration tests.
You need to update Azure SQL Database and Container Apps to meet the following requirements:
•Ensure that the Azure SQL logical server allows connections from Azure services.
•Ensure that the Container Apps environment accepts inbound requests from the public internet.
What should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

Start IP address and end IP address of the Azure SQL firewall rule:

- 0.0.0.0 to 0.0.0.0.
- 127.0.0.1 to 127.0.0.1
- 10.0.0.0 to 10.255.255.255

Container Apps ingress:

- Restrict ingress to internal traffic only.
- Set ingress to external and target port 5000.
- Set ingress to internal and target port 5000.

Answer:

Azure SQL Database: Allow Azure services and resources to access this serverContainer Apps: Ingress

Question: 67 CertyIQ

You have a database that contains production data. The schema is stored in a Git repository as an SDK-style SQL database project and contains the following reference data.

Name	Data type
RefId	int identity
Code	nvarchar(10)
CreateDate	datetime2
Description	nvarchar(255)

A deployment pipeline can be rerun automatically when a transient failure occurs.
You need to deploy the reference data as part of the same CI/CD process. Rerunning the pipeline must produce the same outcome and must NOT create duplicate rows.
What should you do?

A. Add a post-deployment script that inserts reference rows by using IF NOT EXISTS or MERGE logic.
B. Restore a backup after each deployment.
C. Store the reference values in GitHub repository secrets.

Answer: A

Question: 68 CertyIQ

You have an Azure SQL database named ProductsDB.
You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases /data-api-builder:latest image.
The container app has the following configurations:
•Secrets: mssql-connection-string, dab-config-base64
•Environment variables:
•MSSQL_CONNECTION_STRING=secretref:mssql-connection-string
•DAB_CONFIG_BASE64=secretref:dab-config-base64
•Ingress: External on port 5000
Users report that the /health endpoint returns a healthy response, but all requests that query an entity named Products fail and generate a connection error.
You confirm that the SQL login in the connection string is correct and the database exists.
You need to ensure that the container app can establish connections to the Azure SQL logical server without changing the container app deployment settings or the DAB configuration file.
What should you do on the Azure SQL logical server?

A. Create an auto-failover group for ProductsDB.
B. Run DBCC CHECKDB on ProductsDB.
C. Create a firewall rule that allows a start and end IP address of 0.0.0.0.
D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for ProductsDB.

Answer: C

Explanation:

Technical Justification for Correct Option (C)

Problem Context Recap: Azure Container Apps-deployed Data API Builder (DAB) with verified configurations (secrets, environment variables, ingress) fails to query the "Products" entity in an Azure SQL database, returning connection errors despite correct SQL login and database existence.

Correct Option (C) - Create a firewall rule that allows a start and end IP address of 0.0.0.0

Why Correct: Azure SQL Database's firewall settings must explicitly allow incoming connections from the IP address of the client (in this case, the Azure Container App). By setting the start and end IP address to 0.0.0.0, you effectively allow all IP addresses, which includes the dynamic IP addresses assigned to Azure Container Apps. This ensures the container app can establish connections to the Azure SQL logical server without needing to modify the container app's deployment settings or the DAB configuration.
Impact on Security: While this resolves the connectivity issue, it's essential to note that allowing 0.0.0.0 opens the database to all IP addresses, which might not be ideal for security. A more secure approach would be to find the specific IP range for Azure Container Apps in your region and allow only those. However, given the constraints of not changing the container app deployment, this option directly addresses the problem.

Why Other Options are Less Suitable

A. Create an auto-failover group for Products:

Relevance to Issue: Auto-failover groups are related to high availability and geo-replication, not connection establishment. This does not resolve the firewall/connection problem.
B. Run DBCC CHECKDB on Products:

Purpose: DBCC CHECKDB checks the integrity of the database. While important for maintenance, it does not affect network connectivity issues between the container app and the database.
D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for Products:

Focus: This option is related to query performance optimization by forcing the use of the last good query plan. It has no bearing on establishing connections to the database.

Action to Resolve the Issue:On the Azure SQL logical server, **create a firewall rule with a start and end IP address set to 0.0.0.0** to allow connections from all IP addresses, thereby enabling the Azure Container App to connect.

References

1. **Azure SQL Database Firewall Rules**:https://docs.microsoft.com/en-us/azure/sql-database/database-firewall-configure?tabs=azure-portal
2. **Azure Container Apps Networking**:https://docs.microsoft.com/en-us/azure/container-apps/networking (#outbound-connections)

Question: 69 CertyIQ

You have an Azure SQL database named SalesDB and an Azure App Service app named sales-api. SalesDB contains a table named dbo.Customers. dbo.Customers contains two columns named CreditCardNumber and TenantId.
Currently, sales-api connects to SalesDB by using SQL authentication with stored username and password.
You need to recommend a solution that meets the following requirements:
•Provides a passwordless method for sales-api to access SalesDB.
•Ensures that credit card numbers are NOT stored as plain text.
What should you include in the recommendation?

A. Azure Key Vault and Always Encrypted
B. Transparent Data Encryption (TDE) and row-level security (RLS)
C. managed identities and dynamic data masking
D. managed identities and Always Encrypted

Answer: D

Explanation:

Technical Justification for Recommended Solution

Recommended Solution: D. Managed Identities and Always Encrypted

Breakdown of Requirements and Why D is the Best Fit:

1. Passwordless Method for Access

Requirement: Move away from SQL authentication with stored username and password.
Managed Identities (Part of D):

Why Suitable: Provides passwordless authentication by leveraging Azure Active Directory (AAD) identities for resources, eliminating the need to store credentials in the app.
How it Works: Azure assigns a unique identity to the sales-api app, which can then be authorized to access SalesDB without needing to embed credentials.
Why Other Options Fail Here:

A, B, and C do not directly address the passwordless authentication requirement as effectively as managed identities.
2. Credit Card Numbers Not Stored as Plain Text

Requirement: Ensure encryption of sensitive data at rest.
Always Encrypted (Part of D):

Why Suitable: Encrypts data both at rest and in transit, without requiring database changes, ensuring credit card numbers are never stored or processed in plain text.
Encryption Control: Keys can be managed in Azure Key Vault (though the question does not explicitly require key management to be part of the solution, Always Encrypted is the key component here).
Why Other Options are Less Suitable Here:

A (Azure Key Vault & Always Encrypted): While Always Encrypted is correct for encryption, the inclusion of Azure Key Vault (for key management) doesn't address the passwordless requirement directly. The question implies a need for a more direct solution to both requirements without adding unnecessary components not asked for.
B (TDE & RLS):

TDE: Encrypts data at rest but does not ensure data is encrypted in transit or prevent storage in plain text in the same robust manner as Always Encrypted for specific columns.
RLS: Controls access to data but does not encrypt it.
C (Managed Identities & Dynamic Data Masking):

Managed Identities: Correct for passwordless access.
Dynamic Data Masking: Only masks data from unauthorized users in query results, not stored encrypted, hence failing the encryption requirement.

Conclusion: Option D, **Managed Identities and Always Encrypted**, is the most comprehensive solution. Managed Identities effectively replaces SQL authentication for a passwordless approach, while Always Encrypted ensures sensitive data (like credit card numbers) is never stored as plain text, fulfilling both requirements precisely.

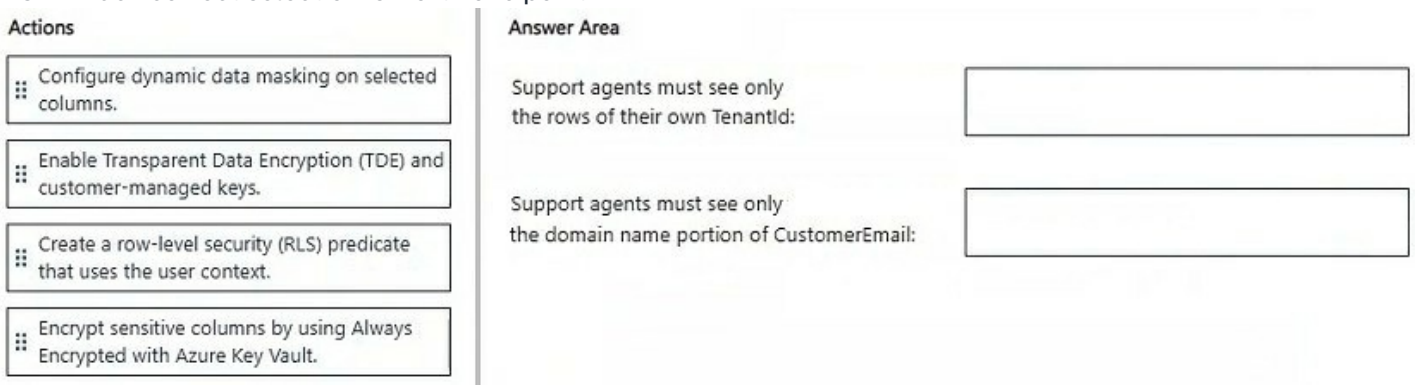
References:

Microsoft Documentation: Use managed identities for App Service and Azure Functions
Microsoft Documentation: Always Encrypted (Database Engine)

Question: 70 CertyIQ

DRAG DROP
-
You have an Azure SQL database named sqldb-ai-prod that stores customer support tickets for a multitenant software as a service (SaaS) application. sqldb-ai-prod contains a table named Tickets. Tickets contains columns named TenantId, TicketId, CustomerEmail, CustomerPhone, and Notes.
You plan to harden data access, since a new support team will use ad hoc reporting tools that connect directly to

sqldb-ai-prod.
You need to configure security to meet the following requirements:
•Support agents must see only the rows of their own TenantId column.
•Support agents must see only the domain name portion of the CustomerEmail column.
What should you do for each requirement? To answer, drag the appropriate actions to the correct requirements. Each action may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

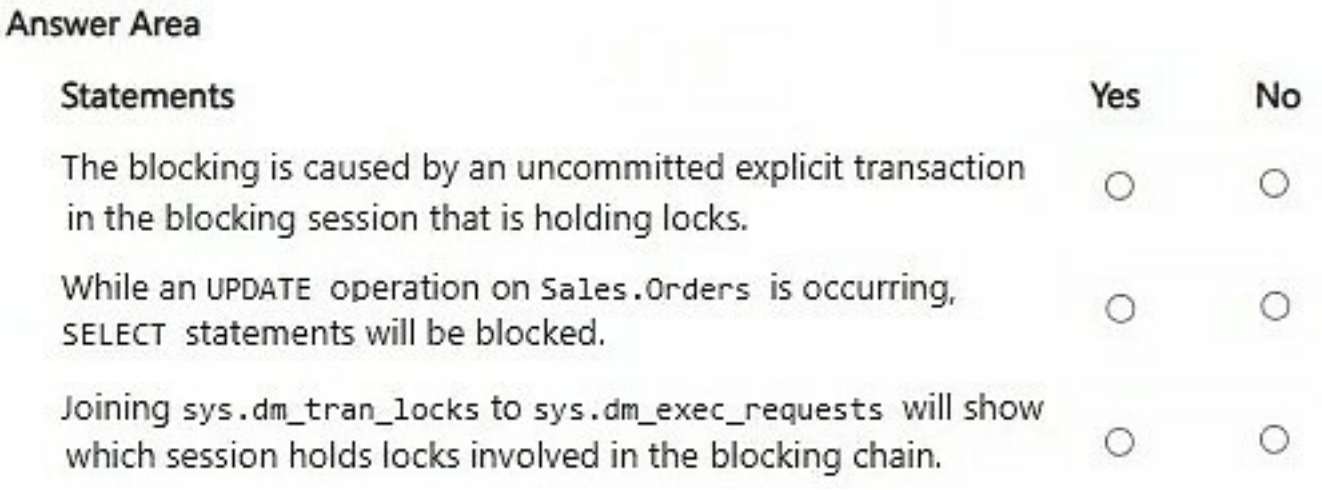


Answer:

Requirement: Support agents must see only the rows of their own TenantId column.Action: Row-Level Security (RLS)Requirement: Support agents must see only the domain name portion of the CustomerEmail column.Action: Dynamic Data Masking (DDM)

Question: 71 CertyIQ

HOTSPOT
-
You have a SQL database in Microsoft Fabric that uses the default settings for a newly created database and contains a table named Sales.Orders.
You have an application that uses two stored procedures to access Sales.Orders.
While monitoring database activity, you discover the following:
•sys.dm_exec_requests shows multiple sessions in a suspended state with wait_type = LCK_M_X. All the sessions show the same wait_resource, which maps to Sales.Orders, and the same nonzero blocking_session_id.
•sys.dm_exec_input_buffer(blocking_session_id, NULL) returns a last submitted command of BEGIN TRANSACTION UPDATE Sales.Orders.
•sys.dm_exec_sessions for the blocking session shows status = sleeping and open_transaction_count = 1.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.



Answer:

Answer Area

Statements	Yes	No
The blocking is caused by an uncommitted explicit transaction in the blocking session that is holding locks.	○	○
While an `UPDATE` operation on `Sales.Orders` is occurring, `SELECT` statements will be blocked.	○	○
Joining `sys.dm_tran_locks` to `sys.dm_exec_requests` will show which session holds locks involved in the blocking chain.	○	○

Explanation:

1. **The blocking is caused by an uncommitted explicit transaction...**

Yes — A session with status = 'sleeping' and open_transaction_count = 1 indicates that a transaction was opened explicitly but never committed or rolled back, leaving its acquired locks active and causing blocking.

1. **While an UPDATE operation on Sales.Orders is occurring, SELECT statements will be blocked.**

Yes — Under the default READ COMMITTED isolation level, an UPDATE statement acquires an exclusive lock (X), which prevents standard SELECT statements (requiring shared locks) from reading the rows until the transaction concludes.

1. **Joining sys.dm_tran_locks to sys.dm_exec_requests will show which session holds locks...**

No — Because the blocking session is currently **sleeping**, it has no active execution request. Since it will not appear in sys.dm_exec_requests, a standard join will fail to return the head blocking session. Instead, you must query sys.dm_tran_locks paired with sys.dm_exec_sessions.

Question: 72 CertyIQ

DRAG DROP
-
You have an Azure SQL database named SalesDB that has Query Store enabled. SalesDB supports an AI-driven product search API.
Users report that the latency of the API increased immediately after the API was deployed and remains high.
You need to identify whether the latency increase was caused by an execution plan regression and, if so, decide which corrective action will restore a previous plan. The solution must prevent any changes to the API code.
What should you use? To answer, drag the appropriate tools to the correct step. Each tool may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

Tools

- Azure Monitor
- Log Analytics
- Query Performance Insight
- The Query Store plan store
- Plan forcing in Query Store
- The Regressed Queries pane in Query Store

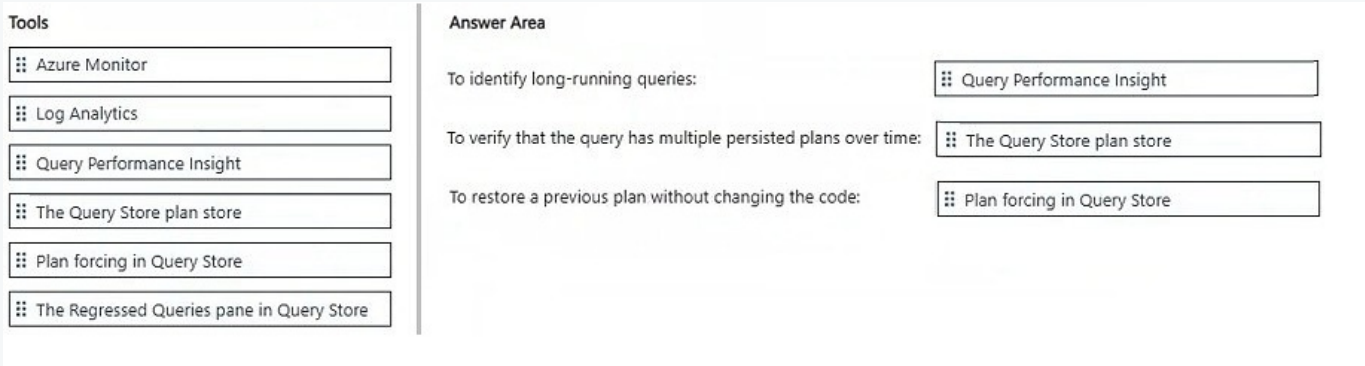
Answer Area

To identify long-running queries: [ ]

To verify that the query has multiple persisted plans over time: [ ]

To restore a previous plan without changing the code: [ ]

Answer:



Explanation:

To identify long-running queries: Query Performance Insight
Reasoning: Azure SQL's Query Performance Insight provides a visual dashboard directly in the portal to quickly locate top consuming and long-running queries.
To verify that the query has multiple persisted plans over time: The Query Store plan store
Reasoning: The Query Store plan store captures and maintains a history of execution plans, runtime statistics, and specific data points for every individual query over time.
To restore a previous plan without changing the code: Plan forcing in Query Store
Reasoning: Plan forcing instructs the query processor to lock down and use a specific, previously captured execution plan instead of letting the optimizer generate a new one.

Question: 73 CertyIQ

You have an Azure container app named app1-contoso-001 that hosts a Data API builder (DAB) container in front of an Azure SQL database.
You add an entity named Todo that uses a source named dbo.todos.
Which URL pattern should clients use to access the Todo REST endpoint?

A. https://appl-contoso-001.azurestaticapps.net/data-api/graphq1/Todo
B. https://appl-contoso-001.azurestaticapps.net/api/Todo
C. https://appl-contoso-001.azurestaticapps.net/data-api/Todo
D. https://appl-contoso-001.azurestaticapps.net/data-api/api/Todo

Answer: B

Explanation:

Technical Justification for Correct Answer (B)

The correct URL pattern for clients to access the Todo REST endpoint is **B. https://appl-contoso-001.azurestaticapps.net/api/Todo**. Here's why:

Understanding the Components:

Azure Container App: Hosts the application (app1-contoso-001).
Data API Builder (DAB) Container: Exposes REST (and possibly GraphQL for specific config) endpoints for data access.

Azure SQL Database (Source): Contains the data (dbo.todos table, exposed via an entity named Todo).

URL Pattern Analysis for Each Option:

A. .../data-api/graphq1/Todo:

Incorrect: Typo in graphq1 (should be graphql if intended for GraphQL API, but the question specifies a REST endpoint for the Todo entity). Additionally, the default setup for REST endpoints doesn't typically include data-api or specifically graphql in the path for REST access.
B. .../api/Todo:

Correct: Aligns with common REST API design patterns where api is a standard prefix for REST endpoints. Given the DAB's role, this pattern is expected for accessing entities via REST without explicitly needing to reference data-api in the path for standard REST operations.
C. .../data-api/Todo:

Less Suitable: While data-api might seem relevant, the question implies a standard REST access scenario. This path might be more aligned with a custom or less conventional setup rather than the default expected REST endpoint for entity access.
D. .../data-api/api/Todo:

Less Suitable: Redundant and unconventional use of both data-api and api in the path. This does not align with typical REST API endpoint naming conventions for entity access.

Why B is Best:

Conventional REST API Naming: api as a root for REST endpoints is widely adopted and expected.
Simplicity and Directness: Directly maps to the entity (Todo) without unnecessary path components for standard REST access scenarios.
Alignment with Typical DAB Configuration: Assuming standard configuration, DAB would expose REST endpoints under a straightforward api path for entity-based REST interactions.

References

For further reading on Azure Container Apps and REST API design with Data API Builder, refer to:

Microsoft Documentation: Azure Container Apps Overview
Data API Patterns for RESTful Web Services

Question: 74 CertyIQ

You have an Azure SQL database that contains a table named Customer. Customer contains the following columns:
•NationalIDNumber is unique per customer.
•InvestigationNotes contains free-form text.
You have a stored procedure that performs point lookups by NationalIDNumber and returns InvestigationNotes in the query results without filtering on InvestigationNotes.
You need to encrypt both columns by using Always Encrypted and the highest security possible.
Which type of Always Encrypted encryption should you use for each column?

A. deterministic for NationalIDNumber and randomized for InvestigationNotes
B. deterministic for NationalIDNumber and deterministic for InvestigationNotes
C. randomized for NationalIDNumber and randomized for InvestigationNotes
D. randomized for NationalIDNumber and deterministic for InvestigationNotes

Answer: A

Explanation:

Technical Justification for Correct Option (A)

Correct Option: A. deterministic for NationalIDNumber and randomized for InvestigationNotes

NationalIDNumber (deterministic encryption):

Requirement: Unique per customer and used for point lookups.
Deterministic Encryption Choice Rationale:

Enable Filtering: Deterministic encryption allows the database to perform equality searches (point lookups) on the encrypted column, which is crucial for the stored procedure's functionality.
Suitable for Unique Identifiers: Since NationalIDNumber is unique per customer, the minimal risk of information leakage through frequency analysis (a consideration for deterministic encryption) is outweighed by the necessity of efficiently querying this column.

InvestigationNotes (randomized encryption):

Requirement: Contains free-form text, not filtered in queries.
Randomized Encryption Choice Rationale:

Higher Security for Sensitive Text: Randomized encryption provides higher security compared to deterministic, as it does not reveal patterns or frequencies, suitable for sensitive, free-form text.
No Query Filtering Requirement: Since the stored procedure does not filter on InvestigationNotes, the inability to perform efficient searches on a randomized column does not impact functionality.

Why Other Options are Less Suitable:

B. deterministic for both:

Inadequate Security for InvestigationNotes: Given the sensitive nature of InvestigationNotes, using deterministic encryption may not offer sufficient security against potential frequency analysis attacks.
C. randomized for both:

Breaks Functionality for NationalIDNumber: Randomized encryption would prevent efficient point lookups by NationalIDNumber, breaking the stored procedure's functionality.
D. randomized for NationalIDNumber and deterministic for InvestigationNotes:

Same Break in Functionality as C for NationalIDNumber: Randomized encryption is not suitable for NationalIDNumber due to the lookup requirements.
Inadequate Security for InvestigationNotes (as in B): Deterministic encryption may not be secure enough for sensitive InvestigationNotes.

References

For deeper dive into Always Encrypted encryption types and their applications:

Microsoft Documentation: Understand Always Encrypted
Microsoft Documentation: Always Encrypted (Client Library)

Question: 75 CertyIQ

Your development team uses Microsoft Visual Studio Code with the MSSQL extension and the GitHub Copilot Chat

extension.
The team connects to an Azure SQL database by using individual database logins and uses the @mssql chat participant to generate and run Transact-SQL queries from prompts.
What is used to ensure that GitHub Copilot Chat-generated queries run in the context of the developer?

A. SQL permissions
B. Azure role-based access control (Azure RBAC) permissions
C. GitHub organization permissions
D. shared MCP instruction files

Answer: A

Explanation:

Technical Justification for Correct Answer: A. SQL Permissions

The correct option to ensure GitHub Copilot Chat-generated queries run in the context of the developer is **A. SQL Permissions**. Here's a detailed breakdown justifying why:

Why A. SQL Permissions is Correct

Context Execution: For queries generated by GitHub Copilot Chat (via @mssql chat participant) to execute in the context of the developer, the system needs to understand and enforce the permissions of the individual running the query. SQL Permissions, defined within the Azure SQL database, dictate what actions (SELECT, INSERT, UPDATE, DELETE, etc.) a database login can perform on specific database objects.
Individual Database Logins: Since the team connects using individual database logins, it implies each member has a unique identity within the database. SQL Permissions are tied to these logins, ensuring that queries (regardless of their generation method) are executed with the permissions of the logged-in user.
Granular Control: SQL Permissions offer granular control over what a user can do, aligning with the principle of least privilege. This means even if GitHub Copilot generates a query, it can only execute actions the developer's login is permitted to perform.
Why Other Options are Less Suitable

B. Azure Role-Based Access Control (Azure RBAC) Permissions:

Azure RBAC controls access to Azure resources at the subscription or resource group level, not at the database object level.
It does not directly influence the execution context of Transact-SQL queries within an Azure SQL database.
C. GitHub Organization Permissions:

These permissions govern access and actions within the GitHub platform (e.g., repository access, pull request management).
They do not extend to controlling the execution context of database queries.
D. Shared MCP Instruction Files:

There's no standard concept of "MCP instruction files" directly related to controlling query execution contexts in Azure SQL Database or GitHub Copilot integration.
Shared files might contain guidelines or templates but do not enforce permissions.

References

Microsoft Documentation: Azure SQL Database - Security Overview
[GitHub Copilot Documentation: Using GitHub Copilot with Databases](https://docs.microsoft.com/en-us/visualstudio/codespaces/get-started british-english#using-github-copilot-with-databases) **Note:** The exact link for GitHub Copilot with databases might not be directly available as described; for accuracy, refer

to the broader GitHub Copilot docs and Azure SQL integration guides:

Corrected Link for broad understanding: GitHub Copilot Official Site
And for database integration context: Visual Studio Code - SQL Server Extension

Question: 76 CertyIQ

You have an Azure SQL database that supports an AI-enabled product catalog API. The database contains a table named Sales.Orders. Sales.Orders contains 20 million rows and has a nonclustered index on a column named CreateDate.
The API passes a date parameter named @OrderDate.
You have a stored procedure named get_latest_day_orders that filters by running the following query.
WHERE CAST(CreateDate AS date) = @0rderDate.
A review of the query execution reveals that the query causes an index scan.
You need to modify the predicate to ensure that Microsoft SQL Server can use an index seek on CreateDate for a single day.
Which WHERE clause should you use?

A. WHERE CreateDate >= @OrderDate AND CreateDate < DATEADD(day, 1, @0rderDate)
B. WHERE CreateDate LIKE CONVERT(char(10), @0rderDate, 121) + '%'
C. WHERE LEFT(CONVERT(varchar(30), CreateDate, 121), 10) = CONVERT(char(10), @0rderDate, 121)
D. WHERE CONVERT(char(10), CreateDate, 121) = CONVERT(char(10), @OrderDate, 121)

Answer: A

Explanation:

Technical Justification for Correct Answer (A)

Correct Answer: A. WHERE CreateDate >= @OrderDate AND CreateDate < DATEADD(day, 1, @OrderDate)

Why A is the Best Option:

Index Seek Enablement: This predicate allows SQL Server to utilize an index seek on the CreateDate nonclustered index for filtering a single day's orders. By comparing the CreateDate directly without altering its data type or applying functions that could prevent index usage, the query can efficiently narrow down the rows.
Date Range Efficiency: Specifying a range (>= and <) with DATEADD precisely targets a 24-hour period without including the next day, ensuring accuracy in retrieving orders for a single day.
Avoids Index Scan: Unlike the original query, this approach does not force an index scan by avoiding functions around the CreateDate column in the comparison.

Why Other Options are Less Suitable:

B. WHERE CreateDate LIKE CONVERT(char(10), @OrderDate, 121) + '%'

Inefficiency & Potential Incorrectness: Using LIKE with a wildcard % at the end could potentially match more than intended if the date format includes characters that could be wildcard-matched in unexpected ways. Moreover, this approach likely forces an index scan due to the LIKE operator's nature.
Lack of Precision for Single Day: The wildcard could match beyond the desired day, especially if the date format does not clearly delineate the day from time (though the format 121 includes time, the % ignores it).

C. WHERE LEFT(CONVERT(varchar(30), CreateDate, 121), 10) = CONVERT(char(10), @OrderDate, 121)

Function on Indexed Column: Applying CONVERT and LEFT on CreateDate prevents the index from being used efficiently, likely resulting in an index scan.
Unnecessary Complexity: The conversion and substring extraction add unnecessary complexity without providing a clear benefit over simpler range-based queries.

D. WHERE CONVERT(char(10), CreateDate, 121) = CONVERT(char(10), @OrderDate, 121)

Index Seek Prevention: Converting CreateDate to char(10) in the predicate prevents the use of the index on CreateDate, forcing an index scan.
Loss of Time Precision: While this might seem to target a single day accurately by truncating time, the inefficiency due to index scan outweighs any perceived simplicity.

References

Microsoft Docs: Index Scans vs. Index Seeks
Microsoft Docs: DATEADD (Transact-SQL)

Question: 77 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to generate embeddings to resolve the issues identified by the analysts.
Which column should you use?

A. VehicleLocation
B. IncidentDescription
C. IncidentType
D. SeverityScore

Answer: B

Explanation:

B. IncidentDescription.

Because embeddings are needed to compute semantic similarity between the customer feedback text and a known issue description. That requires a text-rich field that actually contains the natural language feedback.

IncidentDescription → contains the actual descriptive text needed for embeddings and fuzzy similarity

VehicleLocation → geographic data (not suitable for semantic matching)

IncidentType → categorical label (too limited for embeddings)

SeverityScore → numeric value (not text-based)

Why the other answer are incorrect:

A. VehicleLocation → geographic field, not textual meaning of the issue

C. IncidentType → usually a short label/category (too little semantic content for meaningful embeddings)

D. SeverityScore → numeric value, not usable for text embeddings

Question: 78 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you

are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to enable similarity search to provide the analysts with the ability to retrieve the most relevant health summary reports. The solution must minimize latency.
What should you include in the solution?

A. a computed column that manually compares vector values
B. a standard nonclustered index on the Embeddings (vector (1536)) column
C. a full-text index on the Embeddings (vector (1536)) column
D. a vector index on the Embeddings (vector (1536)) column

Answer: D

Explanation:

D. a vector index on the Embeddings (vector (1536)) column.

Purpose-built for Vectors: In Microsoft SQL Server 2025 and Azure SQL platforms, the CREATE VECTOR INDEX statement is used to deploy a specialized DiskANN index directly onto high-dimensional VECTOR data types.
Minimizing Latency: Running a semantic similarity query over millions of 1,536-dimensional vectors using standard scans requires calculating the exact distance for every single row, which is computationally expensive and slow. A vector index enables Approximate Nearest Neighbor (ANN) search, drastically reducing look-up time and satisfying the critical constraint to minimize latency.

Why the other choices are incorrect:

Option A (computed column...) forces the relational database engine to compute a manual mathematical comparison line-by-line across every single data record at runtime, maximizing latency rather than minimizing it.

Option B (standard nonclustered index...) uses traditional B-Tree page sorting. Standard B-Tree architectures cannot map or cross-reference multi-dimensional vector arrays and will throw an error or be ignored.

Option C (full-text index...) is designed to index standard natural language strings (varchar, nvarchar) for simple keyword extraction, prefix matching, and word proximity. It cannot interpret mathematical floating-point vectors or handle distance metric tracking.

Question: 79 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return

to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database. The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleIncidentReports	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • IncidentType (nvarchar(50)) • VehicleLocation (nvarchar(200)) • IncidentDescription (nvarchar(max)) • SeverityScore (int)
Vehicles	• VehicleId (int) (primarykey) • VIN ((nvarchar(50)) • VehicleDescription (nvarchar(256))
VehicleHealthSummary	• VehicleId (int) (primarykey) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the UNMASK permission.
Problem Statements -
Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following:
AI workloads -
Vector search -
Modernized API access -
Retrieval Augmented Generation (RAG) pipelines
Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.
The engineers at Contoso report that the following dashboard query runs slowly.

```
SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth
FROM dbo.VehicleHealthSummary
WHERE FleetId = @FleetId
ORDER BY LastUpdatedUtc DESC;
```

You review the execution plan and discover that the plan shows a clustered index scan.
VehicleIncidentReports often contains details about the weather, traffic conditions, and location. Analysts report that it is difficult to find similar incidents based on these details.
Requirements -
Planned Changes -
Contoso wants to modernize Fleet Intelligence Platform to support AI-powered semantic search over incident reports.
Security Requirements -
Contoso identifies the following security requirements:
Restrict the support staff from viewing Personally Identifiable Information (PII) data, which is full email addresses and phone numbers.
Enforce row-level filtering so that analysts see only incidents for the fleets to which they are assigned. The analysts can be assigned to multiple fleets.
Database Performance and Requirements
Contoso identifies the following telemetry requirements:
Telemetry data must be stored in a partitioned table.
Telemetry data must provide predictable performance for ingestion and retention operations. latitude, longitude, and accuracy JSON properties must be filtered by using an index seek.
Contoso identifies the following maintenance data requirements:
Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModifiedUtc column to the time of the change.
Avoid recursive updates.
AI Search, Embeddings, and Vector Indexing
Contoso plans to implement semantic search over incident data to meet the following requirements:
Embeddings must be stored in dedicated Azure SQL Database tables.
Embeddings must be generated from rich natural language fields.
Chunking must preserve semantic coherence.
Hybrid search must combine the following:
Vector similarity -
Keyword filtering or boosting -
Development Requirements -
The development team at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the CustomerFeedback table:
Extract the customer feedback text from the JSON document.
Filter rows where the JSON text contains a keyword.
Calculate a fuzzy similarity score between the feedback text and a known issue description.
Order the results by similarity score, with the highest score first.
You need to recommend a solution for the development team to retrieve the live metadata. The solution must meet the development requirements.
What should you include in the recommendation?

A. Export the database schema as a .dacpac file and load the schema into a GitHub Copilot context window.
B. Add the schema to a GitHub Copilot instruction file.
C. Use an MCP server.
D. Include the database project in the code repository.

Answer: C

Explanation:

C. Use an MCP server.

Live, Dynamic Connection: The Model Context Protocol (MCP) is an open-standard communication protocol built to connect AI models (like GitHub Copilot Chat inside Visual Studio Code) dynamically to external, real-time developer tooling and environments.
Real-Time Database Awareness: Connecting GitHub Copilot to an MCP server endpoint for Microsoft SQL Server or Azure SQL allows the AI assistant to natively query and retrieve live database catalog metadata on demand. This provides the context needed to precisely handle complex query filtering, fuzzy matches, and structural validation across dynamic columns like FeedbackJson without relying on out-of-date configurations.

Why Option A is Incorrect

While a .dacpac file contains the comprehensive database schema design structure, it is a static, point-in-time snapshot artifact compiled from a database project or extraction task. It does not fulfill the critical operational constraint of retrieving live metadata from a running database server when active modifications occur.

Why Other Options are Incorrect

Option B (Add the schema to an instruction file) provides static configuration instructions rather than a discovery mechanism for dynamic schema changes.

Option D (Include the database project in the code repository) tracks schema versions in source control but does not allow an active AI session to query running server engines.

Question: 80 CertyIQ

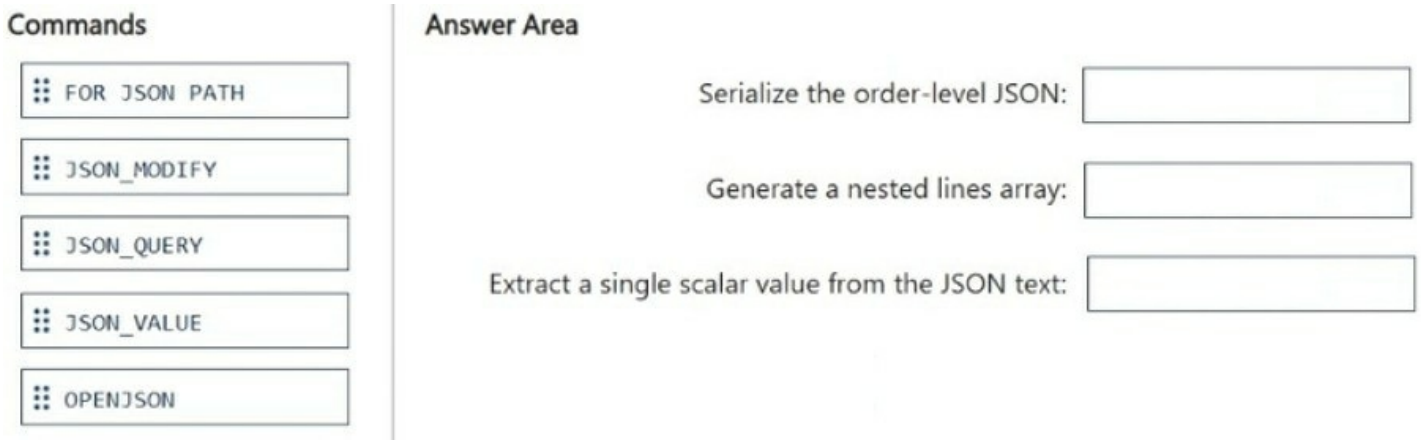
DRAG DROP -
You have an Azure SQL database named SalesDB that contains tables named Sales.Orders and Sales.OrderLines. Both tables contain sales data.
You have a Retrieval Augmented Generation (RAG) service that queries SalesDB to retrieve order details and passes the results to a large language model (LLM) as JSON text. The following is a simple of the JSON.

```
[
  {
    "orderHeaderId": 102348,
    "orderNumber": "SO-2026-000912",
    "orderDateUtc": "2026-01-28T14:22:09Z",
    "customerId": 77821,
    "currencyCode": "EUR",
    "orderTotal": 149.97,
    "lines": [
      {
        "lineNumber": 1,
        "productId": 5012,
        "sku": "CBL-USB-C-1M",
        "quantity": 2,
        "unitPrice": 19.99,
        "lineTotal": 39.98
      },
      {
        "lineNumber": 2,
        "productId": 8841,
        "sku": "HUB-USB-C-7P",
        "quantity": 1,
        "unitPrice": 109.99,
        "lineTotal": 109.99
      }
    ]
  }
]
```

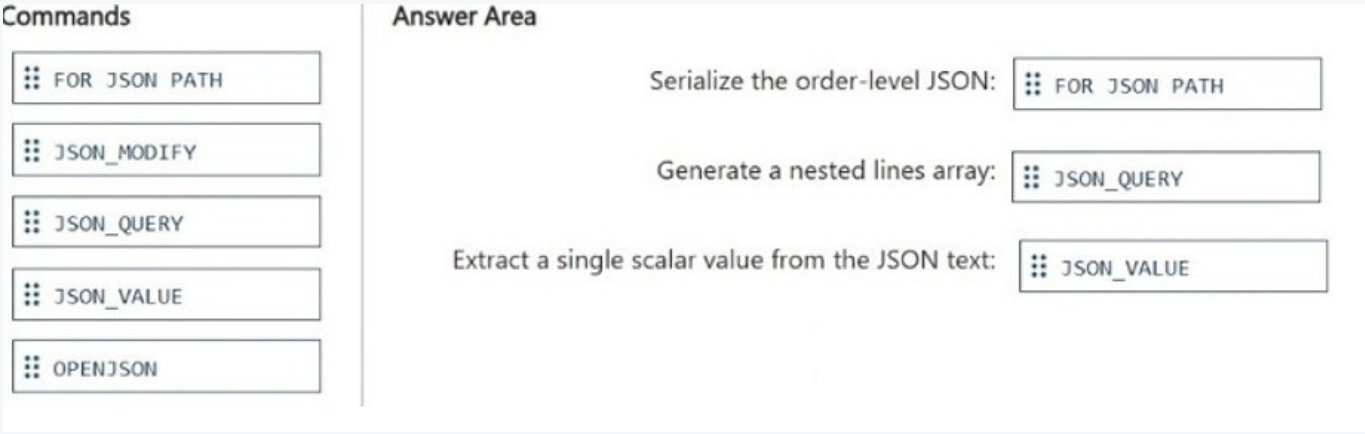

You need to return one JSON document per order that includes the order header fields and an array of related order lines. The LLM must receive a single JSON array of orders, where each order contains a lines property that is a JSON array of line items.

Which Transact-SQL commands should you use to produce the required JSON shape from the relational tables? To answer, drag the appropriate commands to the correct operations. Each command may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Serialize the order-level JSON:FOR JSON PATH

FOR JSON PATH

clause converts relational database rows into structured JSON text. It gives you explicit control over the output structure by using column aliases with dot notation to shape the resulting JSON document objects.

Generate a nested lines array:

JSON_QUERY

When generating nested JSON structures (such as child arrays inside an order object), standard subqueries returning JSON are treated as plain text strings and escaped. Wrapping the subquery or JSON fragment with JSON_QUERY instructs the relational engine that the output is a valid JSON array or object, embedding it safely into the parent hierarchy without corrupting or escaping the array notation.

Extract a single scalar value from the JSON text:

JSON_VALUE

To extract a single primitive data value (such as a string, integer, or boolean) from a specified JSON path, you use JSON_VALUE. This differs from JSON_QUERY, which is exclusively used to retrieve complex elements like entire objects or arrays.

Why the Choices are Incorrect

JSON_MODIFY

Why it is wrong: JSON_MODIFY is strictly used to update, append, or delete a property value within an already existing JSON string. It cannot perform a full multi-table relational serialization or read fresh scalar values out of basic database columns.

OPENJSON

Why it is wrong: OPENJSON is a rowset/table-valued function designed to parse and shred incoming JSON text into a standard relational row-and-column format. It serves the exact opposite workflow (deserialization), making it completely backward for the serialization and extraction tasks requested here.

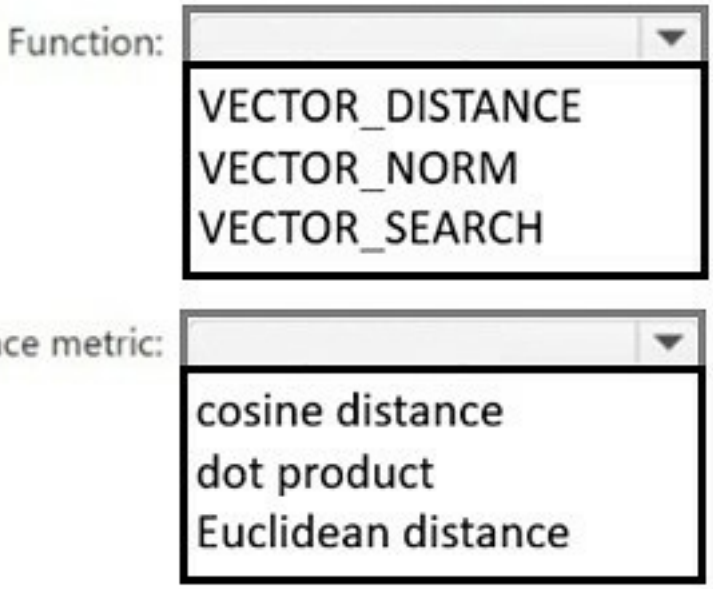
Question: 81 CertyIQ

HOTSPOT -
You have an Azure SQL database that contains the following tables and columns.

Tables	Columns
Articles	• ArticleId (int) • Title (nvarchar(200)) • Notes (nvarchar(max)) • Description (nvarchar(max))
NotesEmbeddings	• ArticleId (int) • ChunkIndex (int) • Embedding (vector(1536))
DescriptionEmbeddings	• ArticleId (int) • ChunkIndex (int) • Embedding (vector(1536))

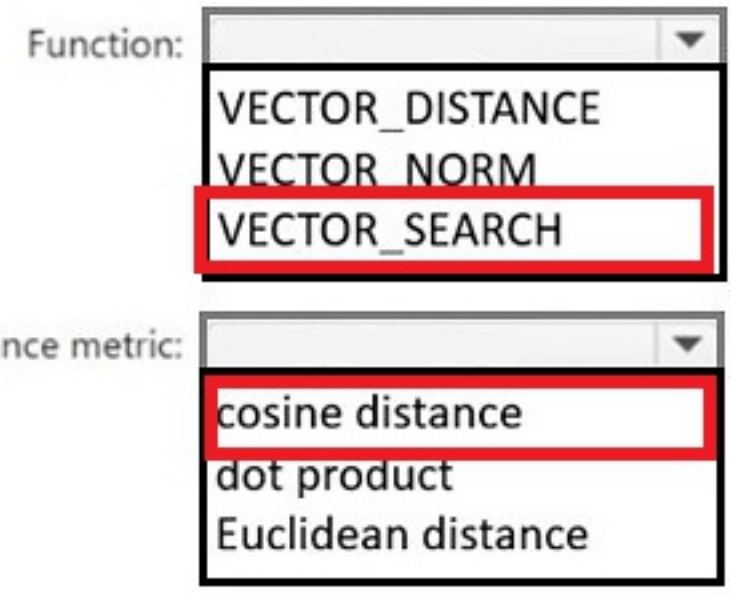
Embeddings in the NotesEmbeddings and DescriptionEmbeddings tables have been generated from values in the Description and Notes columns of the Articles table by using different chunk sizes.
You need to perform approximate nearest neighbor (ANN) queries across both embedding tables. The solution must minimize the impact of using different chunk sizes.
What should you use? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

Function: **VECTOR_SEARCH** is used to perform similarity searches on vector data.

Distance metric: **cosine distance** is a standard metric for measuring the semantic similarity between two vectors.

Incorrect answers:

VECTOR_DISTANCE

Why it is wrong: The VECTOR_DISTANCE function is used to perform an exact vector search. It forces the database engine to execute a full, brute-force table scan to compute distances for every single row. It completely bypasses any available Approximate Nearest Neighbor (ANN) indexes, making it too slow for large

production datasets compared to the selected VECTOR_SEARCH function.

VECTOR_NORM

Why it is wrong: This function calculates the geometric magnitude (length) of a single standalone vector. It cannot take two vectors as parameters, measure angular relationships, or cross-reference multiple rows to execute a database semantic similarity query.

Question: 82 CertyIQ

HOTSPOT -
You have a SQL database in Microsoft Fabric named SalesDB that contains a table named dbo.Products.
You need to modify SalesDB to meet the following requirements:
Create a vector index on the appropriate column.
Use a supplied natural language query vector.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE VECTOR INDEX idx_products_embedding ON dbo.Products ( [ ] )
WITH (METRIC = 'cosine', TYPE = 'DiskANN');
DECLARE @query_vector VECTOR(1536) = @SuppliedVector
SELECT
    t.product_id,
    t.product_name,
    s.distance
FROM
    [ ] (
        TABLE = dbo.Products AS t,
        COLUMN = embedding,
        SIMILAR_TO = @query_vector,
        METRIC = 'cosine',
        TOP_N = 10
    ) AS s
[ ]
```

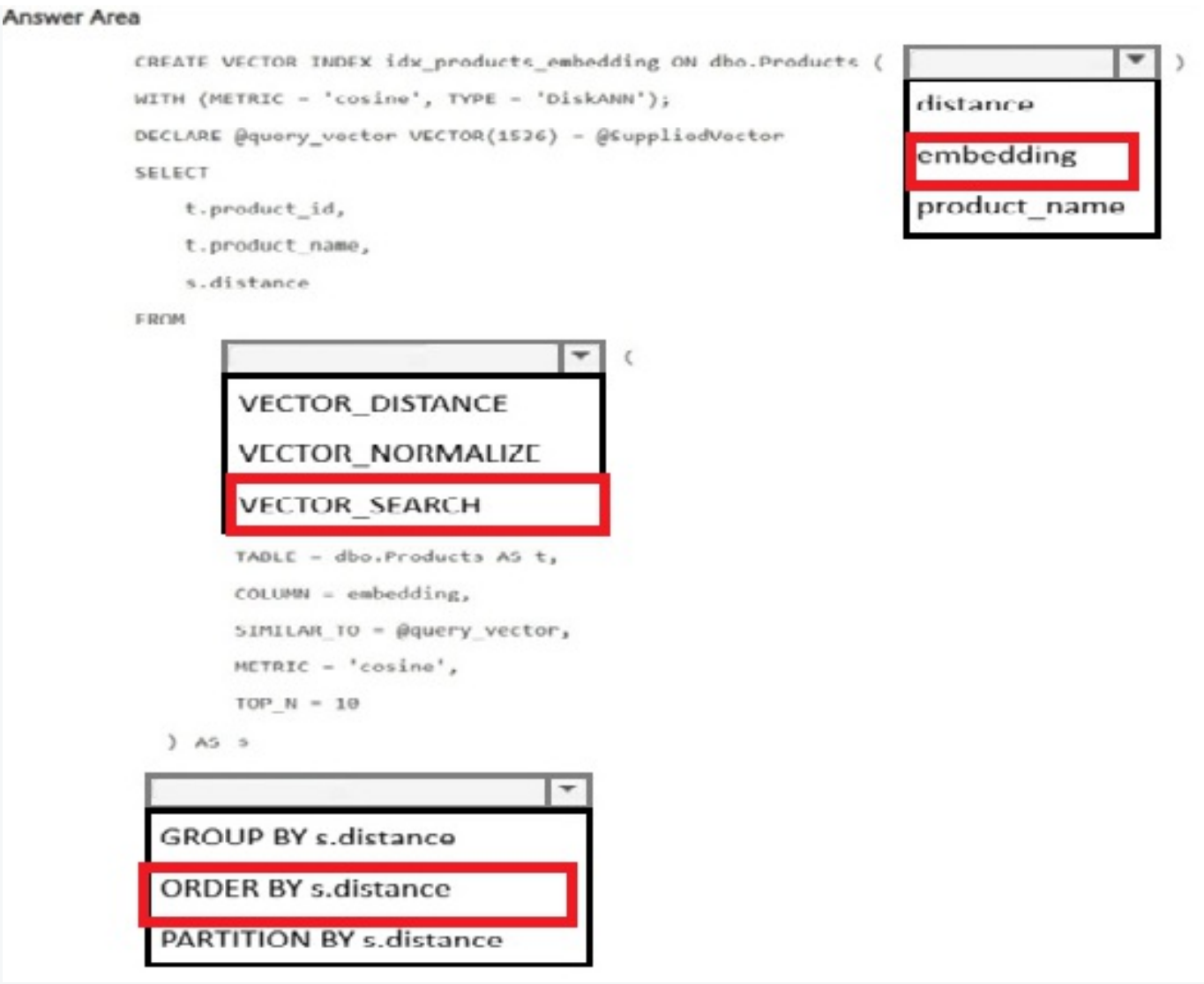
distance
embedding
product_name

VECTOR_DISTANCE
VECTOR_NORMALIZE
VECTOR_SEARCH

GROUP BY s.distance
ORDER BY s.distance
PARTITION BY s.distance

Answer:

Answer Area



Explanation:

Top Dropdown (Index target column):

embedding .

The CREATE VECTOR INDEX statement requires specifying the actual column of the VECTOR data type within the parenthesis. In this schema, that column is named embedding.

Middle Dropdown (Table-valued search function):

VECTOR_SEARCH

The engine uses VECTOR_SEARCH as a native system table-valued function (TVF). It accepts key-value pair parameters such as TABLE = ..., COLUMN = ..., and SIMILAR_TO = ... to execute approximate similarity searches using the built DiskANN index.

Bottom Dropdown (Result ordering clause):

ORDER BY s.distance

To rank and display vector results progressively from the closest semantic match (lowest distance score) to the furthest, a standard query ends with a traditional sorting clause targetting the score alias: ORDER BY s.distance.

Question: 83 CertyIQ

You have an Azure SQL database that contains a table named dbo.Products. dbo.Products contains three columns named Embedding, Category, and Price. The Embedding column is defined as VECTOR(1536).
You use AI_GENERATE_EMBEDDINGS and VECTOR_SEARCH to support semantic search and apply additional filters on two columns named Category and Price.
You plan to change the embedding model from text-embedding-ada-002 to text-embedding-3-small. Existing rows already contain embeddings in the Embedding column.
You need to implement the model change. Applications must be able to use VECTOR_SEARCH without runtime errors.
What should you do first?

A. Regenerate embeddings for the existing rows.
B. Normalize the vector lengths before storing new embeddings.
C. Convert the Embedding column to nvarchar(max).
D. Create a vector index on dbo.Products.Embedding.

Answer: A

Explanation:

A. Regenerate embeddings for the existing rows.

Embedding Model Incompatibility: When changing embedding models (from text-embedding-ada-002 to text-embedding-3-small), the underlying vector mathematical coordinate space changes completely. Even though both models use the exact same default dimension count (1,536 dimensions), a vector generated by one model cannot be semantically compared against a vector from another.Preventing Logical and Runtime Failures: If you run a VECTOR_SEARCH query with a search term vectorized by text-embedding-3-small against your existing data records still vectorized by text-embedding-ada-002, the distance calculation functions will return meaningless, garbage results or fail. According to Microsoft migration guidelines, you must manually re-generate embeddings for all your data so that your entire database shares a uniform vector space before taking the new model live.

Why the other choices are incorrect:

Option B is incorrect. While normalizing vector lengths can be useful for certain distance metrics like Dot Product, it does not fix the fundamental semantic incompatibility between two completely different neural network architectures.

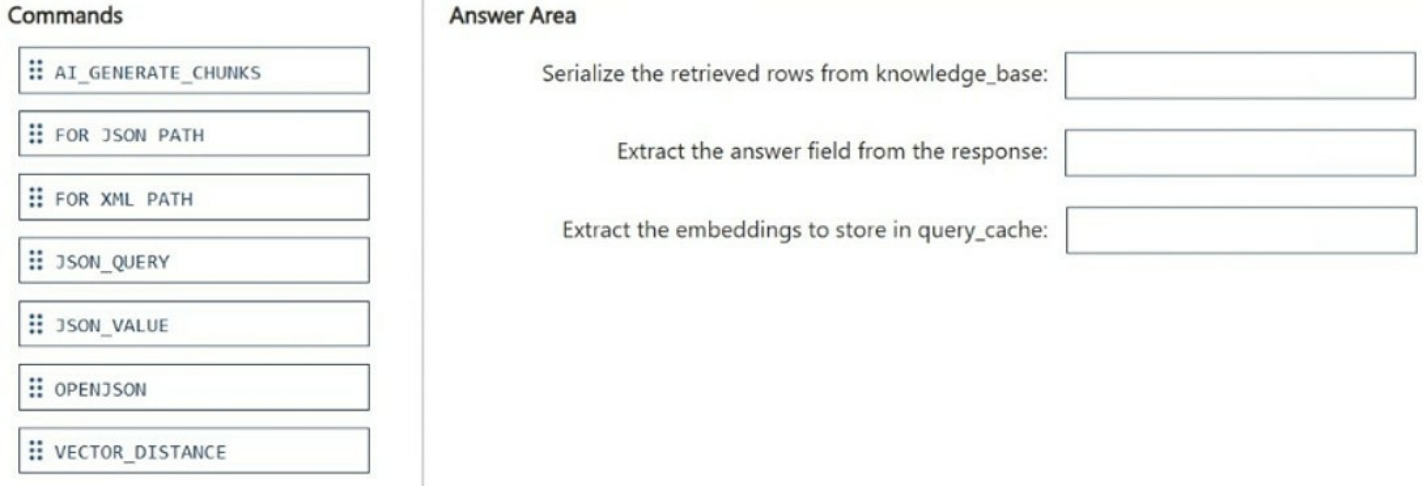
Option C is incorrect. The engine utilizes the native VECTOR(1536) data type for performing optimal, high-speed vector calculations. Converting it to a standard text string like nvarchar(max) would break native VECTOR_SEARCH compatibility and degrade performance.

Option D is incorrect. Creating a vector index on mismatched, corrupted vector spaces serves no purpose. The vector index must be built after the vector values themselves have been made uniform and accurate.

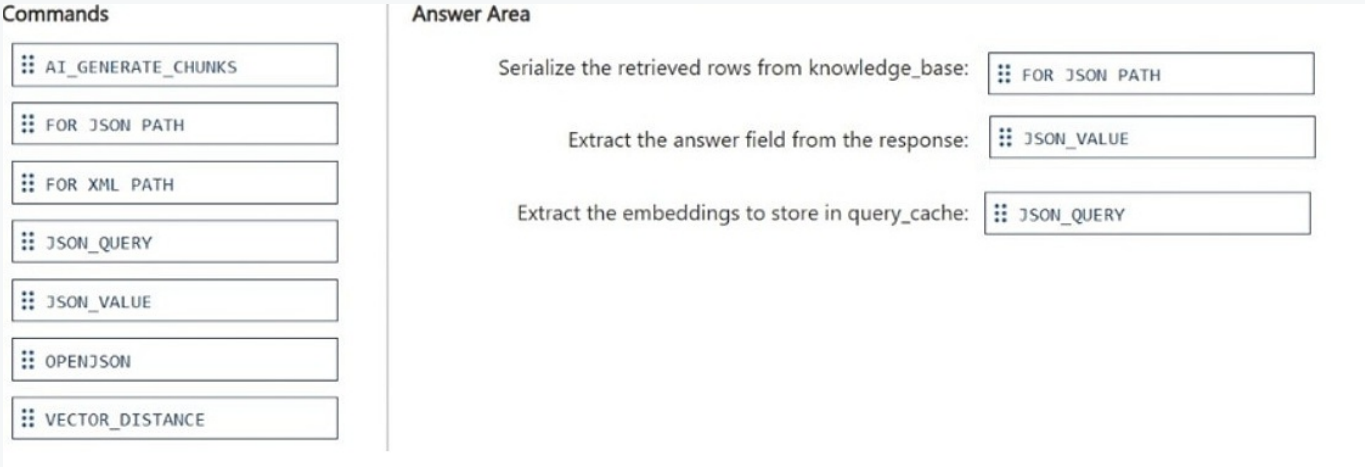
Question: 84 CertyIQ

DRAG DROP -
You have an Azure SQL database named DB1 that contains two tables named knowledge_base and query_cache. knowledge_base contains support articles and embeddings. query_cache contains chat questions, responses, and embeddings. DB1 supports an AI-enabled chat agent.
You need to design a solution that meets the following requirements:
Serializes the retrieved rows from knowledge_base
Extracts the answer field from the response

Extracts the embeddings to store in query_cache
You will call the external large language model (LLM) by using the sp_invoke_external_rest_endpoint stored procedure.
Which Transact-SQL commands should you use for each requirement? To answer, drag the appropriate commands to the correct requirements. Each command may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Serialize the retrieved rows from knowledge_base:

FOR JSON PATH

To package and convert standard relational database rows from your knowledge_base table into a single structured JSON string formatting suitable to pass into an LLM context payload, you use the native FOR JSON PATH clause.

Extract the answer field from the response:

JSON_VALUE

The text answer field returned inside an external chat completion API response represents a single, scalar text string value. The T-SQL function JSON_VALUE is specifically built to extract scalar properties from a JSON document.

Extract the embeddings to store in query_cache:

JSON_QUERY

An embedding vector is returned by an embedding model as a numerical array (e.g., [0.012, -0.453, 0.921, ...]).

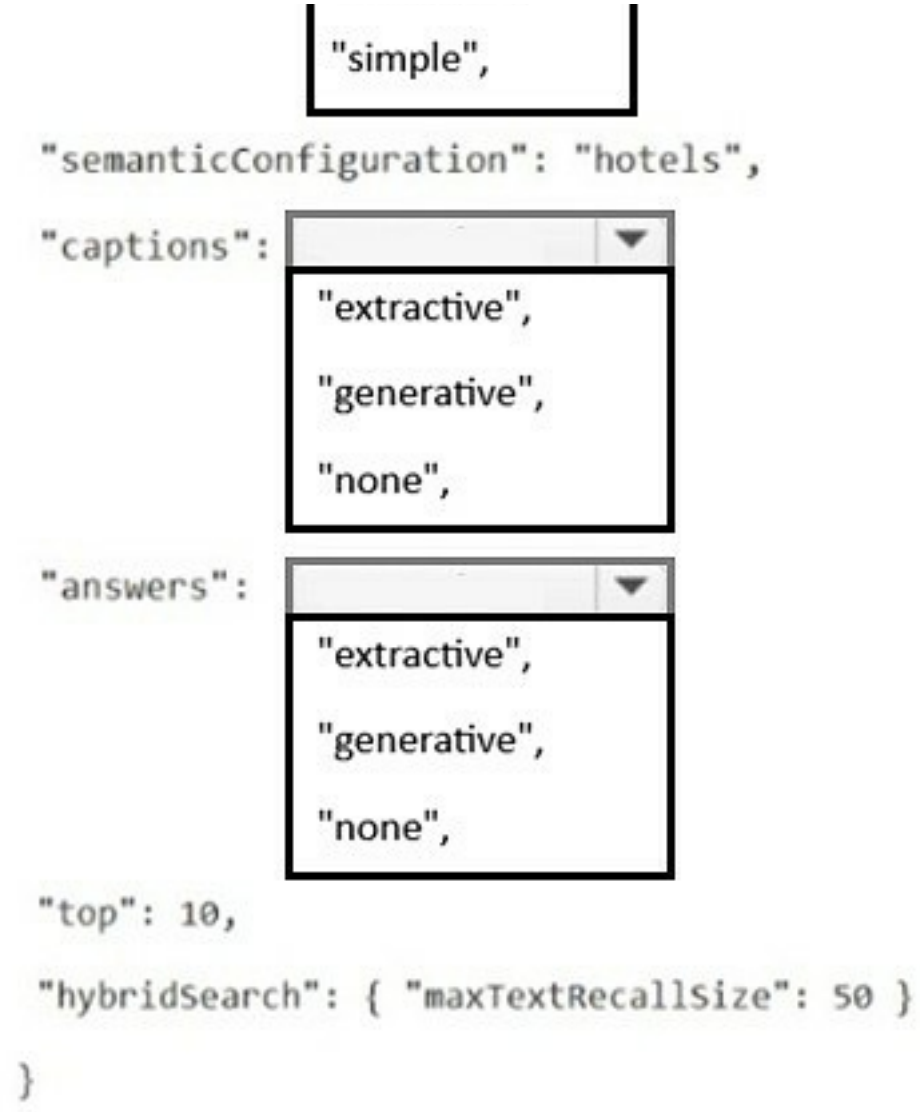
Because it is a complex data structure (an array or object) rather than a scalar value, you must use JSON_QUERY to extract it intact from the JSON response before storing it into your query_cache vector column.

Question: 85 CertyIQ

HOTSPOT -
You have an Azure AI Search service and an index named hotels that includes a vector field named DescriptionVector.
You query hotels by using the Search Documents REST API.
You add semantic ranking to the hybrid search query and discover that some queries return fewer results than expected, and captions and answers are missing.
You need to complete the hybrid search request to meet the following requirements:
Include more documents when ranking.
Always include captions and answers.
How should you complete the REST request body? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
{
  "search": "ocean view",
  "vectorqueries": [
    {
      "vector": [/* embedding array */],
      "fields": "DescriptionVector",
      "k": [ -50, | 10, | 50, ]
      "kind": "vector"
    }
  ],
"queryType": [ "full", | "semantic", ]
```


"simple",
"semanticConfiguration": "hotels",
"captions":
"extractive",
"generative",
"none",
"answers":
"extractive",
"generative",
"none",
"top": 10,
"hybridSearch": { "maxTextRecallSize": 50 }
}

Answer:

Answer Area

```
{
  "search": "ocean view",
  "vectorqueries": [
    {
      "vector": [/* embedding array */],
      "fields": "DescriptionVector",
      "k": [-50, 10, 50,]
      "kind": "vector"
    }
  ],
  "queryType": ["full", "semantic", "simple",]
  "semanticConfiguration": "hotels",
  "captions": ["extractive", "generative", "none",]
  "answers": ["extractive", "generative", "none",]
  "top": 10,
  "hybridSearch": { "maxTextRecallSize": 50 }
}
```

Explanation:

Answer Selections

1. "k": **50,**
2. "queryType": **"semantic",**
3. "captions": **"extractive",**
4. "answers": **"extractive",**

Detailed Explanation

k: 50, To ensure the semantic ranker works effectively, Microsoft explicitly recommends setting the k parameter for vector queries to **50** instead of smaller numbers like 10. If k is too low, the semantic ranker does not receive enough candidate documents to evaluate, which causes queries to return fewer results than expected.
queryType: "semantic", Captions and answers are exclusive features of **Semantic Ranking**. Setting this parameter ensures that the query utilizes the semantic capabilities of Azure AI Search.
captions: "extractive", Semantic captions are drawn exactly as they appear in the original source document text. Setting this to "extractive" is required to enable captions.
answers: "extractive", Like captions, semantic answers extract direct answers from the highly-ranked matching documents. Setting this to "extractive" ensures that a concise answer block is included in the query response.

Question: 86 CertyIQ

You have an Azure SQL database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar (max) text.
You need to call an Azure OpenAI REST endpoint for chat completions. The request body must include both the user question and the retrieved chunks.
You write the following Transact-SQL code.

```
01 CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02 WITH IDENTITY = 'HTTPEndpointHeaders',
03   SECRET = N'{"api-key":"<YOUR_AZURE_OPENAI_API_KEY>"}';
04 GO
05 CREATE OR ALTER PROCEDURE dbo.AskManuals
06   . . .
07   SELECT @chunks =
08   (
09     SELECT TOP (5)
10       mc.ChunkText AS [text]
11     FROM dbo.ManualChunks AS mc
12     ORDER BY mc.Score DESC
13     FOR JSON PATH
14   );
15   SET @payload =
16   (
17     SELECT
18       'system' AS [messages[0].role],
19       'Use only the provided manual chunks.' AS [messages[0].content],
20       'user' AS [messages[1].role],
21       CONCAT(@question, CHAR(10), JSON QUERY(@chunks)) AS [messages[1].content]
22
23   );
24
25   EXEC @retval =
26     . . .
27 END;
28 GO
```

What should you insert at line 22?

A. FOR XML AUTO, TYPE, XMLSCHEMA
B. FOR JSON AUTO, INCLUDE_NULL_VALUES
C. FOR XML PATH, INCLUDE_NULL_VALUES
D. FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

Answer: D

Explanation:

D. FOR JSON PATH, WITHOUT_ARRAY_WRAPPER.

Why this is the correct choice:

Payload Formatting for REST APIs: When building a T-SQL stored procedure to interact directly with the Azure OpenAI REST endpoint for chat completions, the request body (@payload) must be formatted as a single, unified JSON object rather than a JSON array.
FOR JSON PATH Precise Shaping: Using FOR JSON PATH allows you to define complex, nested properties (such as the messages array containing distinct role and content blocks) by mapping column aliases using dot

notation (messages[0].role, messages[1].content).
WITHOUT_ARRAY_WRAPPER Suppression: By default, SQL Server wraps all FOR JSON query outputs in square brackets [ ] as an array. The WITHOUT_ARRAY_WRAPPER option removes these outer brackets, generating a clean, single root JSON object that fulfills the requirements of the API call.

Why the other choices are incorrect:

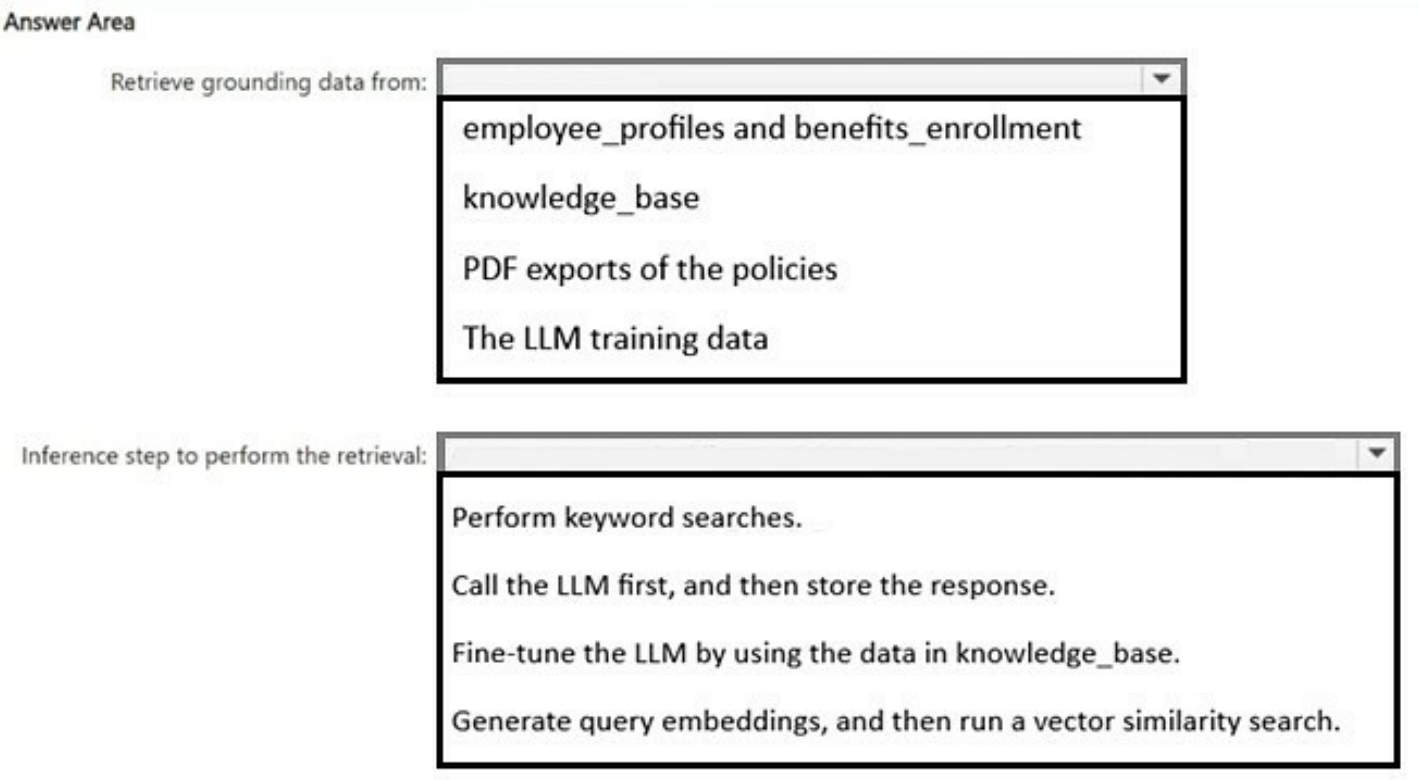
Options A & C: The destination service is a modern REST API (Azure OpenAI) that communicates exclusively via JSON payloads. Returning the data structured as XML (FOR XML) will result in an unreadable payload format and an API error.

Option B: FOR JSON AUTO shapes the JSON output automatically based on the table's relational structure. It does not provide the precise dot-notation naming controls needed to cleanly structure nested API arrays like messages. Additionally, it lacks the WITHOUT_ARRAY_WRAPPER modifier, meaning it would output an invalid top-level array.

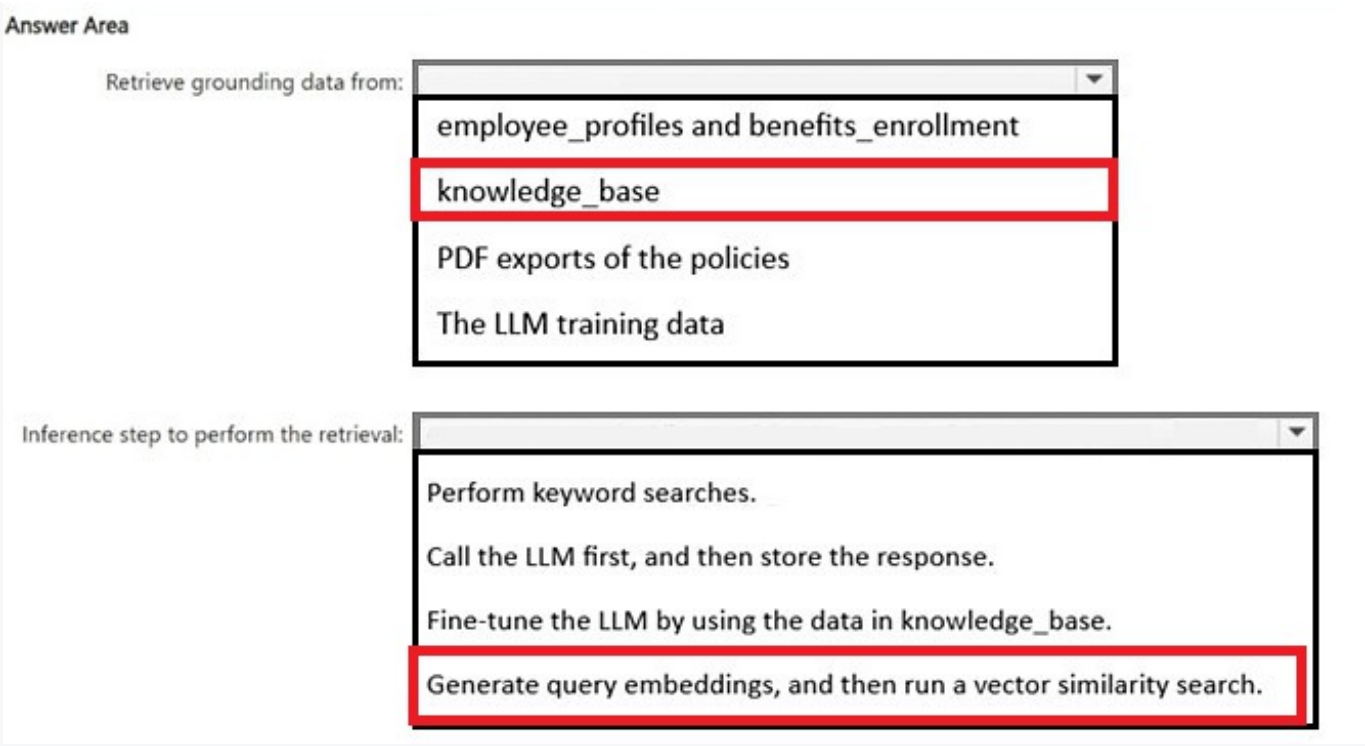
Question: 87 CertyIQ

HOTSPOT -
You have an Azure SQL database that contains a table named knowledge_base. knowledge_base stores human resources (HR) policy documents and contains columns named title, content, category, and embedding.
You have an application named App1. App1 queries two relational tables named employee_profiles and benefits_enrollment that contain HR data. App1 hosts a chatbot that calls a large language model (LLM) directly.
Users report that the chatbot answers general HR questions correctly but provides outdated or incorrect answers when policies change. The chatbot also fails to answer questions that reference internal policy documents by title or category.
You need to recommend a Retrieval Augmented Generation (RAG) solution to resolve the chatbot issues.
What should you recommend? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:



Explanation:

Retrieve grounding data from: **knowledge_base.**

In a RAG architecture, the "grounding data" is the external, authoritative dataset that the LLM is intended to reference to provide accurate, up-to-date answers. The knowledge_base is designed to hold this context. Other options like "LLM training data" are incorrect because that is internal to the model (pre-trained knowledge), which often lacks real-time updates and is prone to hallucinations.

Inference step to perform the retrieval: **Generate query embeddings, and then run a vector similarity search.**

Modern RAG systems rely on **semantic search** rather than keyword search. By converting a user query into a vector embedding and performing a similarity search against the vector-indexed knowledge_base, the system can retrieve the most contextually relevant documents, even if they don't share identical keywords with the query.

Question: 88 CertyIQ

HOTSPOT -
Your company has an ecommerce catalog in a Microsoft SQL Server 2025 database named SalesDB. SalesDB contains a table named products. products contains the following columns: product_id (int) product_name (nvarchar(200)) description (nvarchar(max)) category (nvarchar(50)) brand (nvarchar(50)) price (decimal) sku (nvarchar(40))
The description fields are updated daily by a content pipeline, and price can change multiple times per day.
You want customers to be able to submit natural language queries and apply structured filters for brand and price.
You plan to store embeddings in a new VECTOR(1536) column and use VECTOR_SEARCH(... METRIC='cosine' ...).
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Generating an embedding by concatenating `product_name`, `category`, and `description` will support the customer requirements.	○	○
Including `price` in the text used to generate embeddings is required.	○	○
The underlying base type of the embeddings will be `float(32)`.	○	○

Answer:

Answer Area

Statements	Yes	No
Generating an embedding by concatenating `product_name`, `category`, and `description` will support the customer requirements.	○	○
Including `price` in the text used to generate embeddings is required.	○	○
The underlying base type of the embeddings will be `float(32)`.	○	○

Explanation:

"Generating an embedding by concatenating product_name, category, and description will support the customer requirements."

Context: Semantic search models (like text-embedding-ada-002) perform best when provided with the full, rich context of an item.
Evaluation: Yes. Concatenating descriptive fields is the standard way to build a comprehensive text source for an embedding model, ensuring that the resulting vector captures the "meaning" of the product across all its attributes.

"Including price in the text used to generate embeddings is required."

Context: Embeddings are designed for semantic similarity (what a product is), not exact filtering (what a product costs).
Evaluation: No. While you can include price, it is rarely "required" and often counter-productive for semantic search. Because price is a numeric value that changes, embedding it can cause "noise" in your vector space. Pricing is better handled via standard SQL WHERE clause filters on the price column, not via the embedding itself.

"The underlying base type of the embeddings will be float(32)."

Context: Vector data in Azure SQL is stored in the vector data type, which is essentially an array of floats.
Evaluation: Yes. Azure SQL currently uses a 32-bit floating-point representation (float32) for the individual dimensions within a vector.

Question: 89 CertyIQ

You need to design a generative AI solution that uses a Microsoft SQL Server 2025 database named DB1 as a data source. The solution must generate responses that meet the following requirements:

Are grounded in the latest transactional and reference data stored in DB1
Do NOT require retraining or fine-tuning the language model when the data changes
Can include citations or references to the source data used in the response
Which scenario is the best use case for implementing a Retrieval Augmented Generation (RAG) pattern? More than one answer choice may achieve the goal. Select the BEST answer.

A. summarizing free-form user input text
B. training a custom language model on historical database data
C. answering user questions based on company-specific knowledge
D. generating marketing slogans based on user sentiment analysis

Answer: C

Explanation:

C. answering user questions based on company-specific knowledge.

The Retrieval-Augmented Generation (RAG) pattern is designed to dynamically fetch external knowledge from a data source (like SQL Server 2025) and insert it into the prompt context for a Large Language Model (LLM). This perfectly satisfies all your core requirements:

Real-time Grounding: By querying the latest tables in DB1 before passing the context to the model, responses stay grounded in real-time transactional and reference data.
No Retraining/Fine-Tuning: The underlying model remains untouched. It simply synthesizes the factual information supplied to it at runtime.
Source Citations: Because specific records or text segments are programmatically retrieved from the database, the solution can easily trace, cite, or reference exactly which rows or documents informed the final response.
SQL Server 2025 Synergy: SQL Server 2025 introduces native vector database capabilities (such as vector data types and distance functions), making it highly optimized for storing embeddings and performing vector similarity searches required for company-specific knowledge bases.

Why the other choices are incorrect:

Option A (summarizing text) focuses on processing text already provided directly by the user in that session; it doesn't require a retrieval step from an external database like DB1.

Option B (training a custom model) directly violates the core requirement of avoiding retraining or fine-tuning when the data changes.

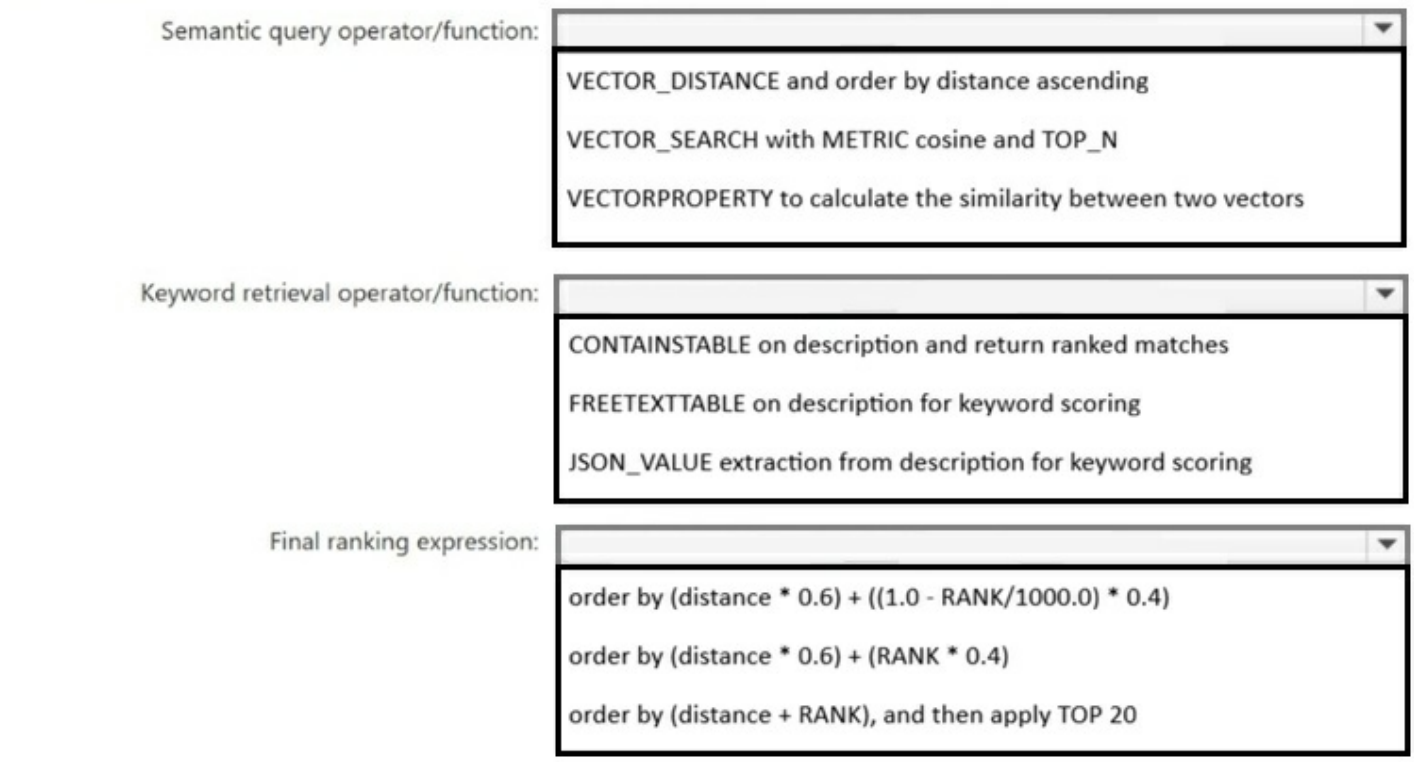
Option D (generating slogans based on sentiment) is primarily a creative generation task that leverages model reasoning and sentiment analysis rather than retrieving specific reference facts and citing structured data sources.

Question: 90 CertyIQ

HOTSPOT -
You have an Azure SQL database that contains a table named stores. stores contains a column named description and a vector column named embedding.
You need to implement a hybrid search query that meets the following requirements:
Uses full-text search on description for the keyword portion
Returns the top 20 results based on a combined score that uses a weighted formula of 60% vector distance and

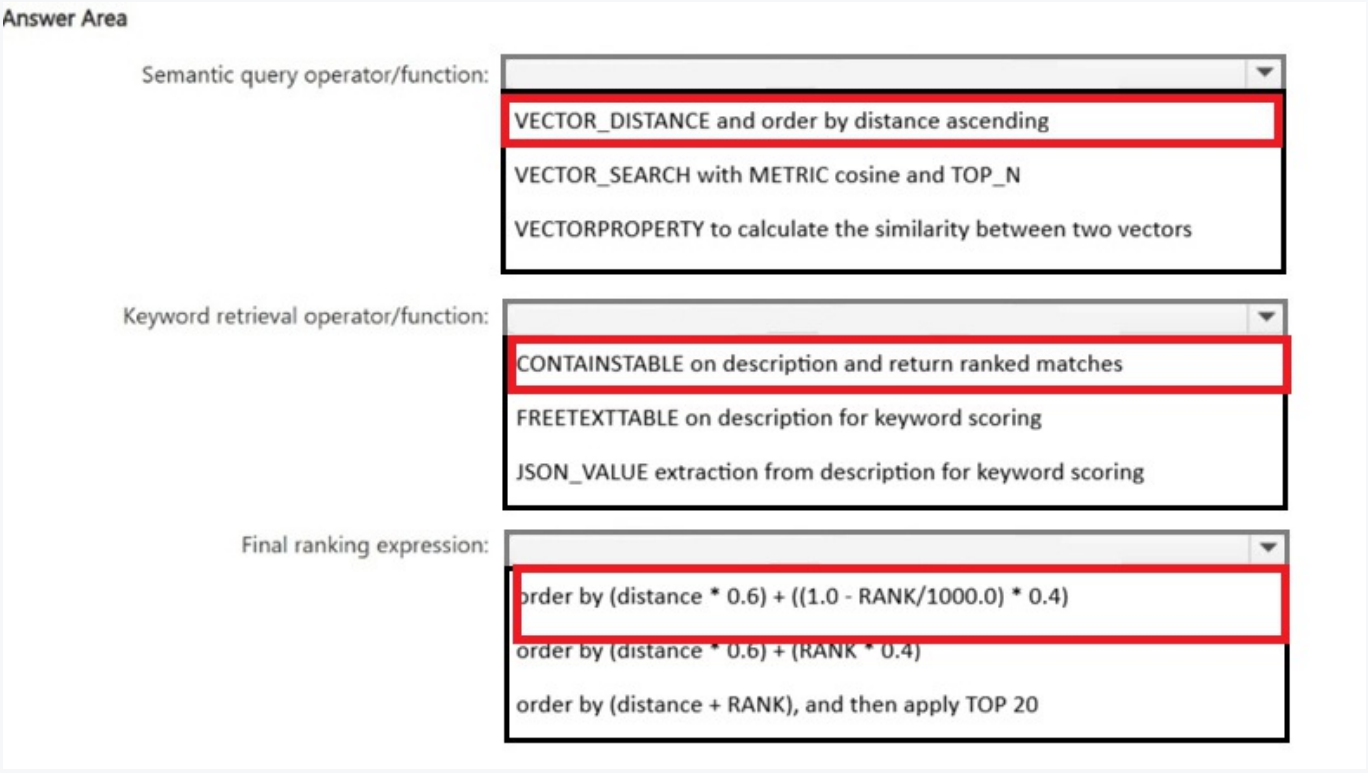
40% full-text rank
How should you configure the query components? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

1. Semantic query operator/function

Choose:

VECTOR_DISTANCE and order by distance ascending

VECTOR_DISTANCE() calculates the distance between the query embedding and stored embeddings.

Smaller distance = more semantically similar.

VECTORPROPERTY is not used for similarity calculations.

VECTOR_SEARCH is not an Azure SQL T-SQL function.

2. Keyword retrieval operator/function

Choose:

CONTAINSTABLE on description and return ranked matches

CONTAINSTABLE performs full-text search and returns a RANK score.

This rank can be combined with vector similarity for hybrid search.

FREETEXTTABLE is for natural-language matching rather than keyword retrieval.

JSON_VALUE has nothing to do with keyword scoring.

3. Final ranking expression

Choose:

order by (distance * 0.6) + ((1.0 - RANK/1000.0) * 0.4)

Distance is better when lower.

Full-text RANK is better when higher (0–1000).

To combine them, the rank must be inverted:

1–

1000

RANK

Then both components become "lower is better" and can be weighted together.

The other options are incorrect because:

(distance * 0.6) + (RANK * 0.4) treats larger rank as worse.

(distance + RANK) combines values with opposite directions and different scales.

Question: 91 CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -

To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.

Existing Environment -

Azure Environment -

Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:

•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments

You store a column master key as a secret in Azure Key Vault.

You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.

Problem Statements -

Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements -

Planned Changes -

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements -

Fabrikam identifies the following security requirements:

•The TransactionProcessing application must use a passwordless connection to DB1.
•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements -

Fabrikam identifies the following development requirements:

•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API: oRead data from the procedures table without authentication. oRead and insert data into the Transactions table once authenticated. oExecute the sp_UpdateProcedurePatient stored procedure.
•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You need to use the UsefulPrompts table as defined in the AI requirements.
Which stored procedure should you use?

A. sp_addendpoint

B. xp_cmdshell

C. sp_OACreate

D. sp_invoke_external_rest_endpoint

Answer: D

Explanation:

D. sp_invoke_external_rest_endpoint.

sp_invoke_external_rest_endpoint is a built-in system stored procedure used in Azure SQL Database and Fabric SQL databases to call external HTTP REST endpoints. In this scenario, it is used to transmit the values from the UsefulPrompts table directly to an **Azure OpenAI** or other Large Language Model (LLM) REST API to generate diagnoses or completions.
sp_addendpoint is used to create and configure HTTP endpoints for SQL Server instances (for native XML/SOAP web services), which is obsolete and unrelated to querying external REST APIs.
xp_cmdshell executes operating system command shell strings from the database engine, which is a major security risk and not an integrated feature for modern cloud AI APIs.
sp_OACreate is an older OLE Automation procedure that can create instances of OLE objects (like HTTP clients), but it is insecure, deprecated for modern use cases, and unsupported in Azure SQL Database.

Question: 92

CertyIQ

Case Study -
This is a case study. Case studies are not timed separately. You can use as much exam time as you would like to complete each case. However, there may be additional case studies and sections on this exam. You must manage your time to ensure that you are able to complete all questions included on this exam in the time provided.
To answer the questions included in a case study, you will need to reference information that is provided in the case study. Case studies might contain exhibits and other resources that provide more information about the scenario that is described in the case study. Each question is independent of the other questions in this case study.
At the end of this case study, a review screen will appear. This screen allows you to review your answers and to make changes before you move to the next section of the exam. After you begin a new section, you cannot return to this section.
To start the case study -
To display the first question in this case study, click the Next button. Use the buttons in the left pane to explore the content of the case study before you answer the questions. Clicking these buttons displays information such as business requirements, existing environment and problem statements. If the case study has an All Information tab, note that the information displayed is identical to the information displayed on the subsequent tabs. When you are ready to answer a question, click the Question button to return to the question.
Existing Environment -
Azure Environment -
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
•Patients
•Employees
•Procedures
•Transactions
•UsefulPrompts
•ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements -
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements -
Planned Changes -

Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged. Deployments must use the Release configuration.

Security Requirements -

Fabrikam identifies the following security requirements:

•The TransactionProcessing application must use a passwordless connection to DB1.

•The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.

•Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

AI Search, Embeddings, and Vector Indexing

Fabrikam identifies the following AI-related requirements:

•Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).

•Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.

•The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAI endpoint.

Development Requirements -

Fabrikam identifies the following development requirements:

•Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.

•Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API: oRead data from the procedures table without authentication. oRead and insert data into the Transactions table once authenticated. oExecute the sp_UpdateProcedurePatient stored procedure.

•Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

•Information for each medical procedure will be stored in a table. The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB -

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"entities": {
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

You implement ProcedureDocuments to support the planned changes.
When users consume data through the Retrieval Augmented Generation (RAG) pattern, they experience data retrieval delays.
You need to improve the data retrieval performance and reduce the number of tokens per retrieval.
What should you implement?

A. chunking

B. embeddings

C. a small language model (SLM)

D. JSON content

Answer: B

Explanation:

B. embeddings.

Vector Core: Vector search requires converting text data into mathematical vector representations known as embeddings.
Function Requirement: The AI_GENERATE_EMBEDDINGS function used in the procedure specifically transforms raw string data into vector formats for indexing.
Semantic Capture: Embeddings capture the deep contextual meaning of text, allowing functions like VECTOR_SEARCH to find relevant matches by calculating geometric distance rather than matching strict keywords.

Question: 93 CertyIQ

Your team is developing an Azure SQL database solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
You need to ensure that GitHub Copilot Chat uses the team's coding standards when generating Transact-SQL code in Visual Studio Code.
What should you use?

A. .github/copilot-instructions.md

B. .vscode/settings.json

C. %APPDATA%\Code\User\settings.json

D. %APPDATA%\Code\User\copilot-instructions.md

Answer: A

Explanation:

Technical Justification for Correct Answer: A

Why A is the Best Option: The correct option is **A. .github/copilot-instructions.md** because GitHub Copilot Chat, when integrated with a repository (especially in a team context), looks for specific configuration files within the repository to adhere to team-wide standards. The .github/copilot-instructions.md file is the designated location for providing instructions and coding standards that GitHub Copilot should follow when generating code, including Transact-SQL for Azure SQL database solutions. This file is repository-centric, making it accessible and applicable to all team members working on the cloned repository, ensuring consistency in generated code.

Why Other Options are Less Suitable:

B. .vscode/settings.json: While this file can store settings for Visual Studio Code, including some extensions' configurations, it is not the standard location for specifying coding standards for GitHub Copilot Chat. Settings here are more user-specific and not inherently tied to the repository's coding standards for the team.

C. %APPDATA%\Code\User\settings.json: This option points to a user's global Visual Studio Code settings, stored on the local machine. It is neither repository-specific nor accessible to the team, making it inappropriate for enforcing team-wide coding standards for GitHub Copilot Chat.

D. %APPDATA%\Code\User\copilot-instructions.md: Similar to option C, this is a local file path not associated with the repository. Even if it were named appropriately for instructions, its location outside the repository means it won't be shared with or applied to the team's workflow in a collaborative development environment.

Key Reasoning Points:

Repository Centricity: The solution needs to be applicable to the entire team, thus requiring a repository-level configuration.
GitHub Copilot Chat Specificity: The chosen file must be recognized by GitHub Copilot Chat for coding standards.
Transact-SQL Generation Context: Consistency in generated Transact-SQL code is crucial for the Azure SQL database solution, necessitating a team-wide, repository-level standard.

References:

1. **GitHub Documentation - Customizing GitHub Copilot**: https://docs.github.com/en/codespaces/codespaces-settings/customizing-github-copilot
2. **Microsoft Docs - Azure SQL Database Development with VS Code**: https://learn.microsoft.com/en-us/azure/azure-sql/database/development-with-visual-studio-code?tabs=azure-cli (Note: While the second link does not directly address GitHub Copilot, it provides context for Azure SQL development in VS Code, which can be relevant for understanding the development environment.)

Question: 94 CertyIQ

You have a Microsoft SQL Server 2025 database that contains a table named products. products contains two columns named description and embedding. embedding is generated from description.
You have an application named App1. App1 has a feature that uses the following query.
VECTOR_DISTANCE('cosine', @query_vector, embedding)
Users report that the feature is slow during peak usage times.
You discover that during peak usage times, the semantic search latency increases and CPU utilization spikes.
You need to reduce latency and CPU utilization related to the App1 feature.
What should you do?

A. Create a vector index on products.embedding and change the query to use VECTOR_NORMALIZE and NORM_TYPE = 'norm1'.

B. Use CONTAINSTABLE against description only and remove the vector search.

C. Create a vector index on products.embedding and change the query to use VECTOR_SEARCH and METRIC = 'cosine'.

D. Add a nonclustered index on products.embedding.

E. Change embedding to VARBINARY(8000).

Answer: C

Explanation:

Technical Justification for Correct Answer (C)

Problem Analysis:The issue lies with the performance of the semantic search feature in App1, specifically with the VECTOR_DISTANCE function, during peak usage times. High latency and CPU utilization indicate

inefficient querying on the embedding column.

Why Correct Answer (C) is Best:

Create a vector index on products.embedding: A vector index is optimized for vector-based queries, reducing the computational overhead during searches. This directly addresses the high CPU utilization and latency issues.
Change the query to use VECTOR_SEARCH with METRIC = 'cosine':

VECTOR_SEARCH is more efficient for searching vectors than VECTOR_DISTANCE because it leverages the index more effectively for filtering.
Specifying METRIC = 'cosine' maintains the desired cosine similarity measurement without needing an explicit normalization step in the query, as the index can handle the metric internally.

Why Other Options are Less Suitable:

A. Use VECTOR_NORMALIZE with NORM_TYPE = 'norm1':

While normalization can help, changing to NORM_TYPE = 'norm1' does not inherently reduce the query's computational complexity or leverage indexing benefits as effectively as switching to VECTOR_SEARCH.
This approach might not fully utilize the indexing benefits for the specific metric (cosine) as VECTOR_SEARCH does.
B. Use CONTAINSTABLE against description only:

Removes the vector search benefit, potentially degrading the feature's effectiveness since embedding is designed for semantic search.
Does not address the performance issue with the vector operation itself.
D. Add a non-clustered index on products.embedding:

A regular non-clustered index is not optimized for vector operations and will not significantly improve the performance of VECTOR_DISTANCE or an equivalent vector search.
E. Change embedding to VARBINARY(8000):

The data type change does not directly impact the query performance issue at hand, as the problem lies with the query operation's efficiency, not the storage format.

Conclusion:Creating a vector index and leveraging VECTOR_SEARCH with the cosine metric is the most effective approach to reduce latency and CPU utilization by optimizing the query performance for semantic search.

References

Microsoft Documentation: Vector Databases and Indexing
Microsoft Documentation: VECTOR_SEARCH (Transact-SQL)

Question: 95 CertyIQ

HOTSPOT
-
You have a SQL database in Microsoft Fabric that contains a table named dbo.Products. dbo.Products contains product catalog data.
You need to create a stored procedure that performs hybrid search. The solution must meet the following requirements:
•Use approximate nearest neighbor (ANN) to retrieve the top 20 candidate products.
•Re-rank only the candidates that also match a full-text query.
•Generate the query embedding.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE OR ALTER PROCEDURE dbo.SearchProducts
    @query_text  NVARCHAR(4000),
    @keywords    NVARCHAR(4000)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @qv VECTOR(1536) =
            [                    ] (@query_text USE MODEL Ada2Embeddings);
```

- AI_GENERATE_EMBEDDINGS
- SEMANTICKEYPHRASETABLE
- VECTOR_NORMALIZE
- VECTOR_SEARCH

```
    ;WITH ann AS
    (
        SELECT t.product_id, t.product_name, s.distance
        FROM   [                    ] (
```

- CONTAINSTABLE
- SEMANTICSIMILARITYTABLE
- VECTOR_DISTANCE
- VECTOR_SEARCH

```
            TABLE = dbo.Products AS t,
            COLUMN = embedding,
            SIMILAR_TO = @qv,
            TOP_N = 20,
            METRIC = 'cosine'
        ) AS s
    )
    SELECT TOP (20)
        t.product_id,
        t.product_name,
        ann.distance,
        fts.RANK AS text_rank
    FROM ann
    JOIN dbo.Products AS t
        ON t.product_id = ann.product_id
    JOIN [                    ] (dbo.Products, description, @keywords) AS fts
```

CONTAINSTABLE

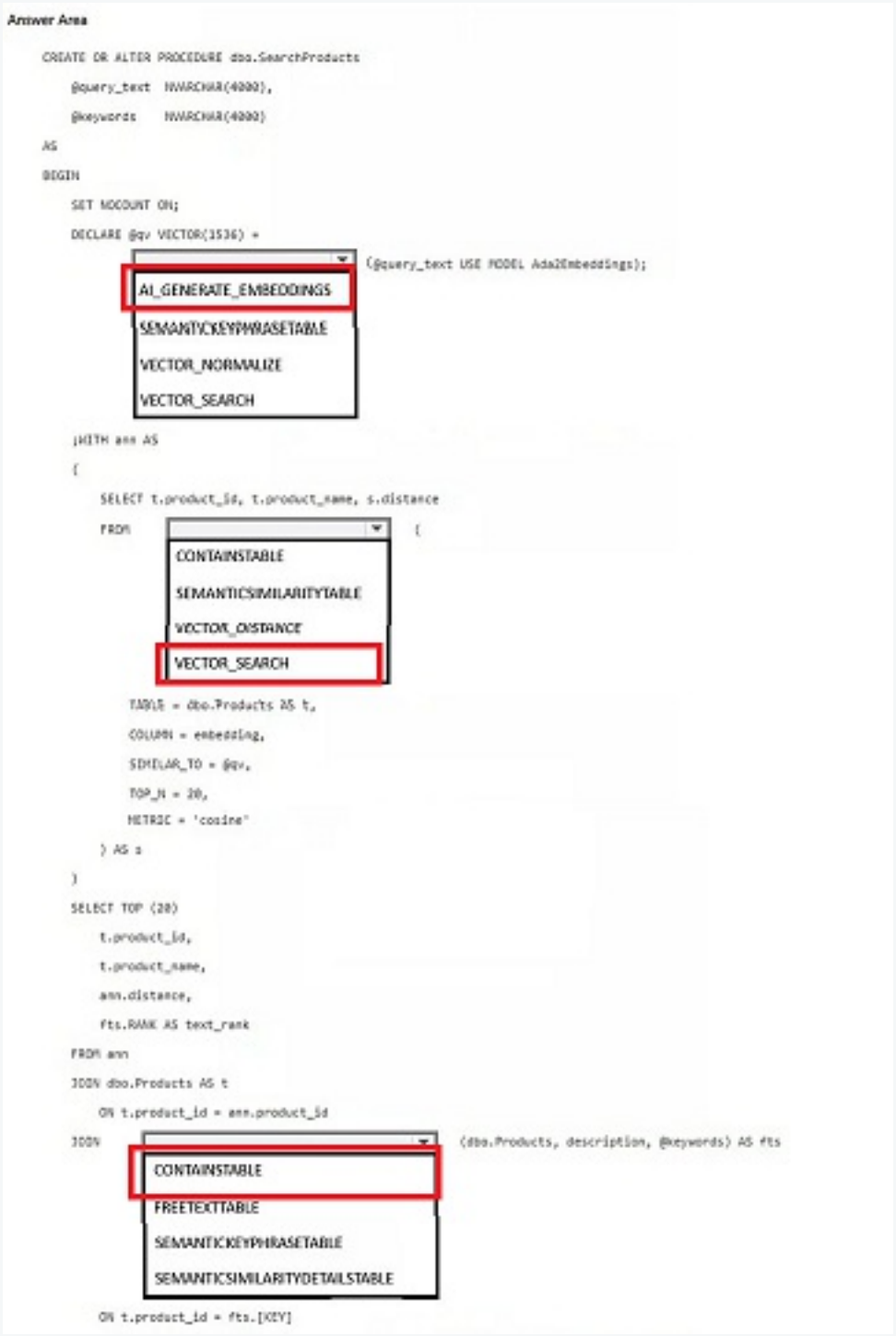
FREETEXTTABLE

SEMANTICKEYPHRASETABLE

SEMANTICSIMILARITYDETAILSTABLE

```
        ON t.product_id = fts.[KEY]
    ORDER BY (ann.distance * 0.6) + ((1.0 - fts.RANK/1000.0) * 0.4);
END;
GO
```

Answer:



Explanation:

Box 1: AI_GENERATE_EMBEDDINGS
Reasoning: This native function is used in Azure SQL / SQL Server to call an external deployment (like an Azure OpenAI model) and generate vector embeddings for a given input text.
Box 2: VECTOR_SEARCH
Reasoning: This table-valued function performs an Approximate Nearest Neighbor (ANN) search over a specified table and vector column using a specific metric (such as 'cosine').
Box 3: CONTAINSTABLE
Reasoning: This is the standard full-text search table-valued function that returns a table of rows matching precise keywords or search conditions, along with their relevance ranking values (RANK).

Question: 96 CertyIQ

You have a GitHub Enterprise subscription.
Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
A mix of GitHub Copilot instructions is configured at different levels, including organization-wide, repository-wide, agent-specific, and personal.
Which instructions will take precedence over the others?

A. repository-wide
B. personal
C. organization-wide
D. agent-specific

Answer: B

Explanation:

Technical Justification for Correct Answer: B (personal)

When multiple GitHub Copilot instruction configurations are set up across different levels, the precedence follows a specific hierarchy based on their scope and the principle of "most specific takes precedence". Here's why **B. personal** is the correct answer and why the other options are less suitable:

B. personal: Personal instructions are configured at the individual user level, making them the most specific and targeted set of instructions. Since they are tailored to a particular user's preferences or requirements within the team, **personal instructions take precedence over all other levels**. This ensures that an individual's customized settings for interactions with GitHub Copilot are respected, which is crucial for productivity and to avoid conflicts with organizational or repository-wide settings.
A. repository-wide: While repository-wide instructions apply broadly across all users working on that specific repository, they are less specific than personal instructions. They set a baseline for the project but do not override individual user configurations.
C. organization-wide: Organization-wide instructions have the broadest scope, applying across all repositories and users within the GitHub Enterprise subscription. Due to their wide applicability, they are the least specific and thus overridden by both repository-wide and personal instructions.
D. agent-specific: Agent-specific instructions, though targeted at specific agents (which could imply build agents, CI/CD agents, etc.), do not directly influence the user-centric interaction with GitHub Copilot in the context provided (development workflow in VS Code). The term "agent-specific" is less commonly referenced in the context of GitHub Copilot configurations compared to the other scopes, and when it is, it typically refers to automations or CI/CD processes rather than developer tooling preferences.

Precedence Summary (Most to Least Specific):

1. **Personal (B)**
2. Repository-wide (A)
3. Organization-wide (C)
4. Agent-specific (D) *(**Least Relevant in this Context**)

References

GitHub Copilot Documentation - Configuration (Though not directly addressing precedence, it covers configuration levels)
GitHub Enterprise Security - Policy Configuration (Indirectly implies hierarchy through policy application scopes)

Note: As of my last update, direct documentation on precedence for GitHub Copilot instructions across these levels might not be explicitly stated in a single GitHub document. The justification is based on general principles of configuration precedence observed in similar GitHub and software development tool configurations. For the most current and precise information, always refer to the latest GitHub Enterprise and Copilot documentation.

Question: 97 CertyIQ

DRAG DROP
-
You have an Azure AI Search service and an index named hotels that includes a vector field named DescriptionVector.
You query hotels by using the Search Documents REST API.
You need to implement a hybrid search query that uses DescriptionVector and includes captions.
How should you complete the REST request body? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

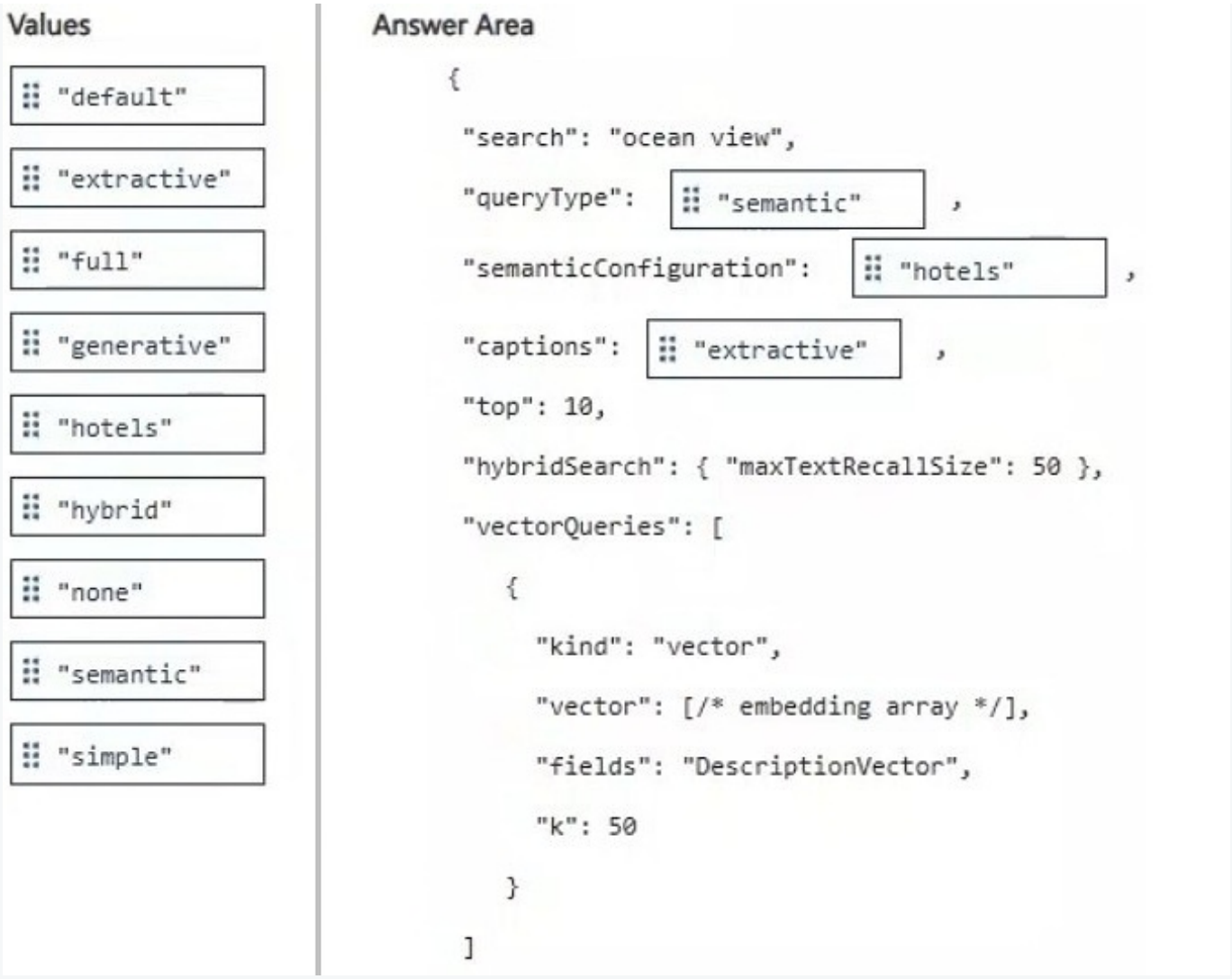
Values

- "default"
- "extractive"
- "full"
- "generative"
- "hotels"
- "hybrid"
- "none"
- "semantic"
- "simple"

Answer Area

```
{
 "search": "ocean view",
 "queryType": [          ] ,
 "semanticConfiguration": [          ] ,
 "captions": [          ] ,
 "top": 10,
 "hybridSearch": { "maxTextRecallSize": 50 },
 "vectorQueries": [
    {
      "kind": "vector",
      "vector": [/* embedding array */],
      "fields": "DescriptionVector",
      "k": 50
    }
 ]
}
```

Answer:



Explanation:

queryType: "semantic"
Reason: Enables the semantic reranking engine on top of the initial hybrid results.
semanticConfiguration: "hotels"
Reason: Specifies the custom name of the semantic configuration defined in the index schema.
captions: "extractive"
Reason: Extracts text snippets from the highest-ranking documents that highlight the most relevant sections answering the query.

Question: 98 CertyIQ

You have an Azure SQL database that contains tables named dbo.Tickets and dbo.TicketNotes. dbo.Tickets contains support tickets and dbo.TicketNotes contains ticket notes.
A retrieval query returns the top five relevant ticket notes for a user question.
You plan to implement a Retrieval Augmented Generation (RAG) pattern that meets the following requirements:
•Formats the retrieved relational data for large language model (LLM) processing
•Sends the user question and retrieved context to an Azure OpenAI REST endpoint for chat completions
•Extracts the response text from the LLM response
Which Transact-SQL function should you use to extract the response text?

A. JSON_MODIFY(@response, '$.result')
B. OPENJSON(@response)

C. JSON_QUERY(@response, '$.choices')
D. JSON_VALUE(@response, '$.choices[0].message.content')

Answer: D

Explanation:

Technical Justification for Choosing Option D

To determine the most suitable Transact-SQL function for extracting the response text from the Azure OpenAI REST endpoint's LLM response, we must consider the requirements and the nature of the response data, which is expected to be in JSON format.

Why Option D is the Best Choice:

Precision in Extraction: The goal is to extract the response text specifically. Given the typical structure of responses from AI endpoints, the relevant text is often nested within a specific path in the JSON response. Option D, JSON_VALUE(@response, '$.choices[0].message.content'), precisely targets this extraction by:

Specifying the exact JSON path $.choices[0].message.content, which is a common structure where the first choice's message content is the desired response text.
Using JSON_VALUE to extract the scalar value at this path, perfect for retrieving a text response.
Why Other Options are Less Suitable:

A. JSON_MODIFY(@response, '$.result'):

Purpose Misalignment: JSON_MODIFY is used to modify existing JSON data, not extract it. The requirement is to extract, not alter, the response text.
Incorrect Path/Usage Assumption: Assumes the target is to modify a $.result path, which may not align with extracting the actual response text's location.
B. OPENJSON(@response):

Broad Extraction: OPENJSON parses a JSON string into a relational table format, which is overkill for simply extracting a specific text value. It would require additional steps to isolate the response text.
Lack of Specificity: Does not target the exact location of the response text, leading to unnecessary data processing.
C. JSON_QUERY(@response, '$.choices'):

Data Type Return: JSON_QUERY returns a JSON fragment (a JSON string), which would still contain the entire .choices object, requiring further processing to extract just the text content of the first choice.
Additional Processing Needed: Unlike JSON_VALUE, it doesn't directly extract the scalar value needed, adding unnecessary steps.
Conclusion

Given the need for precise extraction of the response text from a specific, deeply nested location within the JSON response from an Azure OpenAI REST endpoint, **Option D (JSON_VALUE(@response, '$.choices[0].message.content'))** is the most technically suitable choice due to its ability to directly extract the scalar value at the specified JSON path with minimal additional processing required.

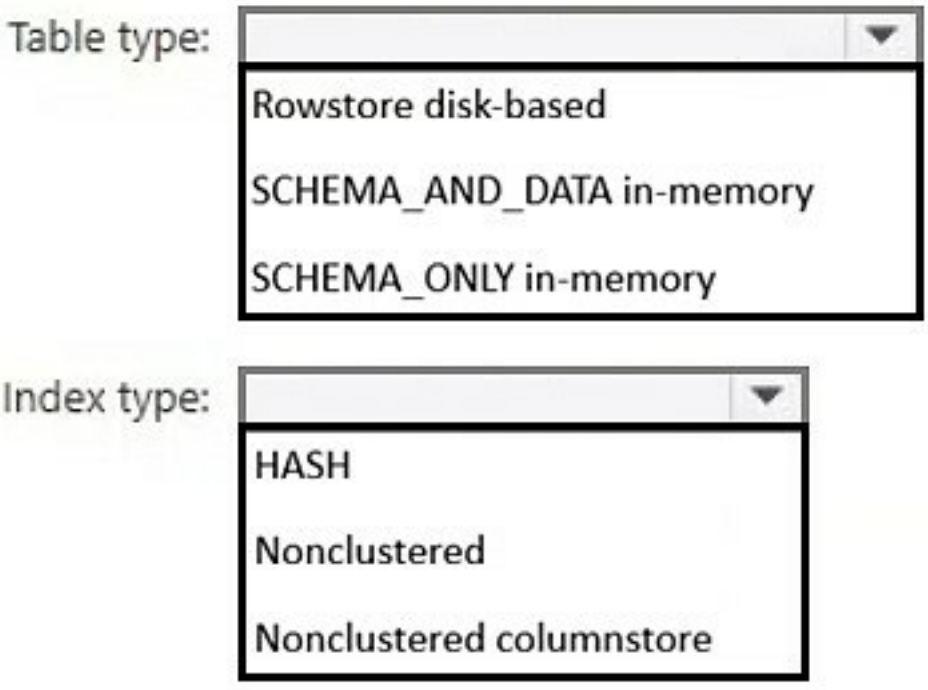
References

Microsoft Docs: JSON_VALUE (Transact-SQL)
Microsoft Docs: Working with JSON Data in SQL Server

Question: 99 CertyIQ

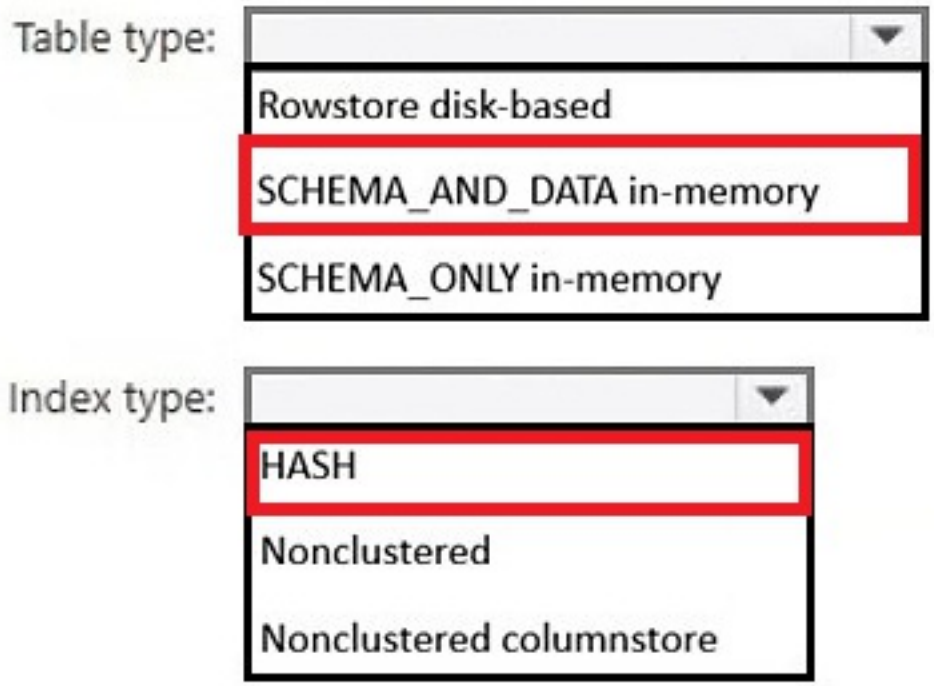
HOTSPOT
-
You have an Azure SQL managed instance that supports a gaming leaderboard API and contains a table named dbo.Leaderboard.
You plan to reduce write latency during peak events of dbo.Leaderboard.
You need to ensure that dbo.Leaderboard supports point lookups. The leaderboard information does NOT need to persist after a restart.
Which type of table and index should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

Table type: SCHEMA_AND_DATA in-memory

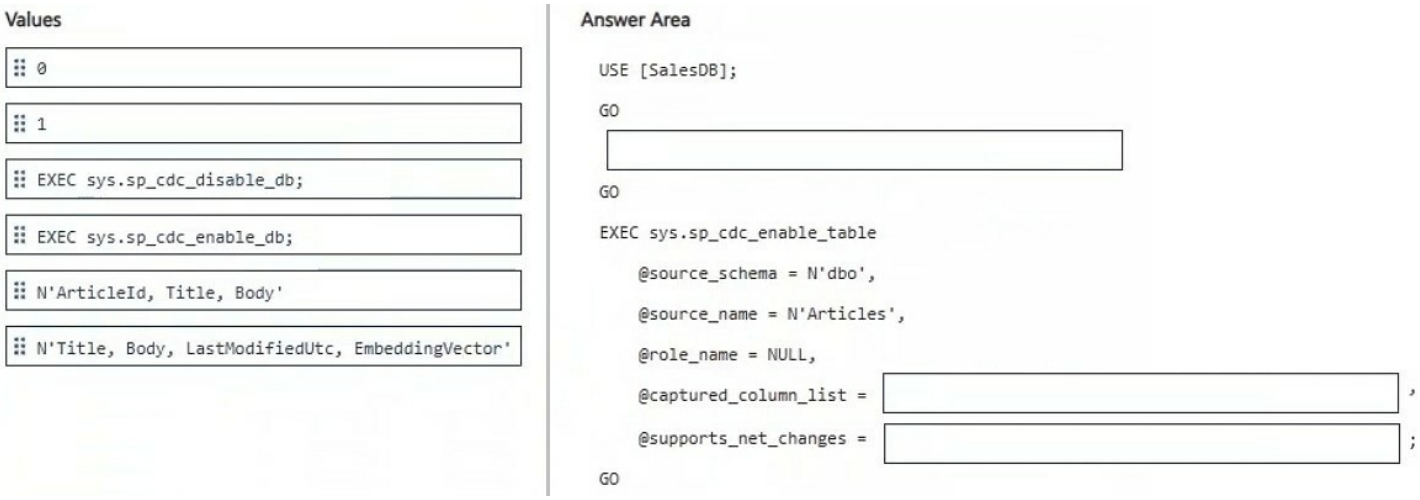
This ensures both the schema structure and the data rows are persisted to disk and fully logged so that data is not lost when the SQL server or database restarts.

Index type:

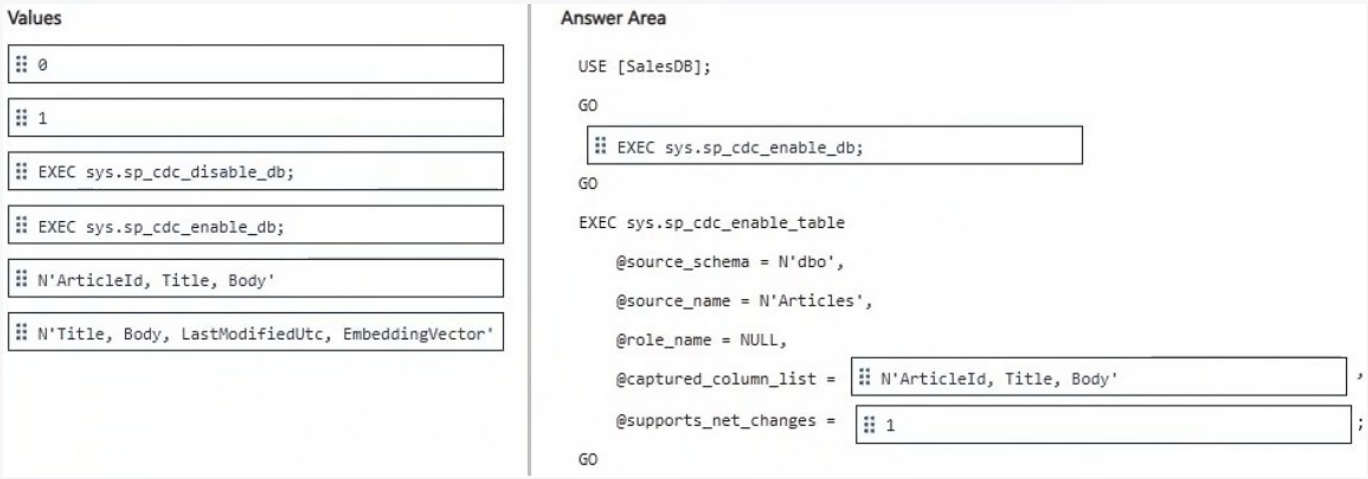
Select **HASH** if the query relies on precise equality lookups (e.g., WHERE Column = 'value').

Question: 100 CertyIQ

DRAG DROP
-
You have an Azure SQL database named SalesDB. SalesDB contains a table named dbo.Articles. dbo.Articles contains the following columns:
•ArticleId
•Title
•Body
•LastModifiedUtc
•EmbeddingVector
You have an application that generates embeddings from the concatenation of Title and Body and stores the results in EmbeddingVector.
You plan to implement an incremental embedding maintenance method that will use change data capture (CDC) to update embeddings only for rows that change, without scanning the entire table.
You need to ensure that only the columns required to generate the embeddings are captured. The solution must support querying net changes.
How should you complete the Transact-SQL script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Target 1 (Database Level Enablement):
EXEC sys.sp_cdc_enable_db;
(CDC must be enabled at the database level before it can be enabled on any specific table.)
Target 2 (@captured_column_list):
N'ArticleId, Title, Body'
(To sync text columns for applications like generating search embeddings, you must capture the content fields along with the table's unique identifier (*ArticleId*) so that changes can be mapped back to the correct records.)
Target 3 (@supports_net_changes):
1
(Setting this parameter to *1* enables net change tracking functions, which are required for incremental synchronization pipelines to process only the final net changes rather than every individual intermediate operation.)

Question: 101 CertyIQ

You have an Azure SQL database that contains a table named Tickets_Embeddings. Tickets_Embeddings contains a VECTOR(1536) column.
Embeddings are generated by using text-embedding-ada-002 and the /openai/v1/embeddings endpoint.
After a review of the current environment, the following changes are requested:

•The AI architect has recommended switching to text-embedding-3-small.
•The security team requires authentication by using Microsoft Entra managed identities only. Storing API keys in an application is prohibited.
Which two actions should you include in the solution? Each correct answer presents a complete solution.
NOTE: Each correct selection is worth one point.

A. Alter the database tables.
B. Regenerate all the embeddings.
C. Change the endpoint.
D. Change the primary keys for the tickets.
E. Add a firewall rule.

Answer: BC

Explanation:

Technical Justification for Correct Answer: B & C (BC)

The correct actions to include in the solution are **B. Regenerate all the embeddings** and **C. Change the endpoint**. Here's why these are the best choices and why the others are less suitable:

B. Regenerate all the embeddings

Why Correct: Switching from text-embedding-ada-002 to text-embedding-3-small involves changing the embedding model. Since these models generate embeddings differently (due to variations in architecture, dimensionality, or training data), the existing embeddings will not be compatible or optimal for the new model. Regenerating ensures consistency and accuracy with the newly selected model.
Technical Impact: Failing to regenerate would lead to suboptimal performance in any downstream AI/ML tasks relying on these embeddings.
C. Change the endpoint

Why Correct: The switch from text-embedding-ada-002 to text-embedding-3-small inherently requires changing the API endpoint, as each model is typically exposed through a unique endpoint or parameter within the /openai/v1/embeddings framework. The exact endpoint might change (e.g., specifying the model version in the query parameters or a slight path adjustment).
Technical Impact: Not changing the endpoint would result in continued use of the old model or errors, as the API would not recognize the request for the new model.
Why Other Options are Less Suitable

A. Alter the database tables:

Insufficient Justification: The change in embedding model does not inherently require altering the database table structure, especially since the VECTOR(1536) column's compatibility with the new embeddings is not questioned. Alterations might be needed if the new model's output dimension differs, but this is not indicated.
Action Specificity: The question does not mention dimensionality changes, making this action unnecessary based on provided info.

D. Change the primary keys for the tickets:

Irrelevance: Changing the embedding model and authentication method has no direct relation to the primary key structure of the Tickets_Embeddings table.
Security/Auth Impact: This action does not address the security requirement mentioned.

E. Add a firewall rule:

Misinterpretation of Security Requirement: The security team's requirement focuses on authentication (using Microsoft Entra managed identities) rather than network access control (firewall rules).
Action Mismatch: While firewall rules are crucial for security, they do not directly address the specified authentication prohibition on storing API keys.
Action Summary for Correct Answer (BC)

Action B (Regenerate all the embeddings): Ensures embeddings are compatible and optimized for text-embedding-3-small.
Action C (Change the endpoint): Correctly points to the new model for embedding generation.
References

Microsoft Entra (formerly Azure Active Directory) Managed Identities Documentation
OpenAI Embeddings API Documentation (for understanding endpoint and model changes)

Question: 102 CertyIQ

You have an Azure SQL database that contains a table named dbo.ManualChunks. dbo.HonualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar(max) text.
You need to call an Azure OpenAI REST endpomt for chat completions. The request body must include both the user question and theretiieved chunks.
You write the following Transact-SQL code.

```
01 CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02 WITH IDENTITY = 'HTTPEndpointHeaders',
03   SECRET = N'{"api-key":"<YOUR_AZURE_OPENAI_API_KEY>"}';
04 GO
05 CREATE OR ALTER PROCEDURE dbo.AskManuals
06   . . .
07   SELECT @chunks =
08   (
09     SELECT TOP (5)
10       mc.ChunkText AS [text]
11     FROM dbo.ManualChunks AS mc
12     ORDER BY mc.Score DESC
13     FOR JSON PATH
14   );
15   SET @payload =
16   (
17     SELECT
18       'system' AS [messages[0].role],
19       'Use only the provided manual chunks.' AS [messages[0].content],
20       'user' AS [messages[1].role],
21       CONCAT(@question, CHAR(10), JSON_QUERY(@chunks)) AS [messages[1].content]
22
23   );
24
25   EXEC @retval =
26     . . .
27 END;
28 GO
```

What should you insert at line 22?

A. FOR XHL AUTO, TYPE, XML SCHEMA,
B. FOR 3SON AUTO, IMCLUDE_MULL_VAIUES
C. FOR XHL PATH, INCLUDE_NULL_VAIUES
D. FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

Answer: D

Explanation:

A. FOR XML AUTO, TYPE, XML SCHEMA Incorrect

This generates XML, not JSON. Azure OpenAI REST endpoints expect a JSON request body.

B. FOR JSON AUTO, INCLUDE_NULL_VALUES Incorrect

This generates JSON, but in AUTO mode and may not produce the required structure. It also returns an array by default.

C. FOR XML PATH, INCLUDE_NULL_VALUES Incorrect

This produces XML output rather than the JSON required by the Azure OpenAI API.

D. FOR JSON PATH, WITHOUT_ARRAY_WRAPPER

This generates properly structured JSON and returns a single JSON object suitable for use as the body of an Azure OpenAI chat completions request.

Question: 103 CertyIQ

You have a Microsoft SQL Server 2025 instance that has a managed identity enabled.
You have a database that contains a table named dbo.ManualChunks. dbo.ManualChunks contains product manuals.
A retrieval query already returns the top five matching chunks as nvarchar(max) text.
You need to call an Azure OpenAI REST endpoint for chat completions. The solution must provide the highest level of security.
You write the following Transact-SG1 code.

```
01  CREATE DATABASE SCOPED CREDENTIAL AzureOpenAIHeaders
02
03  GO
04  CREATE OR ALTER PROCEDURE dbo.AskManuals
05    ...
06    SELECT @chunks =
07    ...
08    SET @payload =
09    ...
10    EXEC @retval = sys.sp_invoke_external_rest_endpoint
11      @url = N'https://<your-resource>.openai.azure.com/openai/deployments/<your-deployment>/chat/completions?api-version=2024-02-15-preview',
12      @method = N'POST',
13      @credential = N'AzureOpenAIHeaders',
14      @payload = @payload,
15      @response = @response OUTPUT;
16  END;
17  GO
```

What should you insert at line 02?

A. WITH IDENTITY = 'HTTPEndpointQueryString',SECRET = N' "api-key":"" ';
B. WITH IDENTITY = 'Managed Identity',SECRET = ' "resourceid":"" ';
C. WITH IDENTITY = 'Managed Identity',SECRET = N' "api-key":"" ';
D. WITH IDENTITY = 'HTTPEnpointHeaders',SECRET = N' "api-key":"" ';

E. WITH IDENTITY = 'AzureOpenAI',SECRET = N' "resourceid":"" ';

Answer: B

Explanation:

1. **Highest Level of Security: Managed Identity**

The question states that the Microsoft SQL Server 2025 instance **has a managed identity enabled** and that the solution must provide the **highest level of security**.

Using an API key (as seen in options A, C, and D) forces you to manage, rotate, and securely store a static secret key, which is inherently less secure than a passwordless connection.

Managed Identity eliminates the need to hardcode or store credentials, leveraging Microsoft Entra ID authentication natively. Therefore, IDENTITY = 'Managed Identity' is required.

2. Correct Database Scoped Credential Syntax for Managed Identity

When configuring a DATABASE SCOPED CREDENTIAL to call an external REST endpoint (like Azure OpenAI) using a system-assigned or user-assigned managed identity, SQL Server expects specific formatting for the SECRET parameter:

When using a Managed Identity, the SECRET shouldn't be an API key like "api-key":"<value>" .

Instead, it must point to the Azure resource identifier. The required JSON payload structure for authenticating via managed identity against an Azure resource requires specifying the resourceId.

Question: 104 CertyIQ

HOTSPOT
You have a SQL database in Microsoft Fabric named Sales BD that contains a table named dbo.Products.
You need to modify SalesBD to meet the following requirements:
• Create a vector index on the appropriate column.
• Use a supplied natural language query vector.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE VECTOR INDEX idx_products_embedding ON dbo.Products (
WITH (METRIC = 'cosine', TYPE = 'DiskANN');
DECLARE @query_vector VECTOR(1536) = @SuppliedVector
SELECT
    t.product_id,
    t.product_name,
    s.distance
FROM
        (
        SIMILAR_TO = @query_vector,
        METRIC = 'cosine',
        TOP_N = 10
    ) AS s
```

[▼])
- distance
- embedding
- product_name

(▼]
- VECTOR_DISTANCE
- VECTOR_NORMALIZE
- VECTOR_SEARCH

Answer:

Answer Area

```
CREATE VECTOR INDEX idx_products_embedding ON dbo.Products (
WITH (METRIC = 'cosine', TYPE = 'DiskANN');
DECLARE @query_vector VECTOR(1536) = @SuppliedVector
SELECT
    t.product_id,
    t.product_name,
    s.distance
FROM
        (
        SIMILAR_TO = @query_vector,
        METRIC = 'cosine',
        TOP_N = 10
    ) AS s
```

[▼])
- distance
- embedding
- product_name

(▼]
- VECTOR_DISTANCE
- VECTOR_NORMALIZE
- VECTOR_SEARCH

Explanation:

First Drop-down (embedding): A vector index must be built on a column containing vector embeddings. Line 12 explicitly references COLUMN = embedding as the source of the vector data for the search function, identifying embedding as the column to index.

Second Drop-down (VECTOR_SEARCH): This is the native T-SQL table-valued function used to query vector indexes. It takes the target table, vector column, query vector, distance metric, and number of top results (TOP_N) as parameters, returning a rowset containing matching records along with their similarity distances.

Question: 105

CertyIQ

You have an Azure SQL table that contains the following data.

FaqId	ProductName	Question	Answer
1	Laptop Pro	Does it support USB-C charging?	Yes, it supports USB-C charging.
2	Tablet X	Does it support a stylus?	It supports the Active Pen stylus.

You need to retrieve data to be used as context for a large language model (LLM). The solution must minimize token usage.
Which formal should you use to send the data to the LLM?

A. ["ProductName": "Laptop Pro", "Question": "Does it support USB-C charging?", "Answer": "Yes, it supports USB-C charging." , "ProductName": "Tablet X", "Question": "Does it support a stylus?", "Answer": "It supports the Active Pen stylus."]

B. JSON["Prompt": "Yes, it supports USB-C charging." , "Prompt": "It supports the Active Pen stylus."]

C. JSON["FaqId": 1, "Question": "Does it support USB-C charging?", "Answer": "Yes, it supports USB-C charging." , "FaqId": 2, "Question": "Does it support a stylus?", "Answer": "It supports the Active Pen stylus."]

D. JSON["FaqId": 1, "ProductName": "Laptop Pro", "Question": "Does it support USB-C charging?", "Answer": "Yes, it supports USB-C charging." , "FaqId": 2, "ProductName": "Tablet X", "Question": "Does it support a stylus?", "Answer": "It supports the Active Pen stylus."]

Answer: B

Explanation:

When providing data or context to an LLM, every character, word, and punctuation mark counts toward your total token consumption. To minimize token usage, you should strip out any redundant or unnecessary metadata that the LLM does not strictly need to understand the core information.

Let's break down why **Option B** is the most efficient:

Elimination of Schema/Metadata: Options A, C, and D all include additional keys like "ProductName", "FaqId", or full QA property breakdowns for each object. These repeated keys drastically inflate the token count.
Concise Structure: Option B simplifies the data down to just the essential content ("Prompt": ...) needed for the model to understand the user intents or statements, removing the overhead of JSON object keys entirely where possible.

Question: 106

CertyIQ

HOTSPOT
You need to create a table in the database to store the telemetry data.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.VehicleTelemetry
(
  TelemetryId BIGINT IDENTITY(1,1) NOT NULL,
  VehicleId NVARCHAR(50) NOT NULL,
  TelemetryTimeUtc DATETIME2(3) NOT NULL,
  BatteryPercent TINYINT NULL,
  SpeedKmh SMALLINT NULL,
  LocationJson JSON NULL,
  ErrorCodesJson JSON NULL,
  RawPayload NVARCHAR(MAX) NULL,
  SysStartTime DATETIME2(7) GENERATED ALWAYS AS ROW START NOT NULL,
  SysEndTime DATETIME2(7) GENERATED ALWAYS AS ROW END NOT NULL,
  PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
  CONSTRAINT PK_VehicleTelemetry PRIMARY KEY CLUSTERED (TelemetryId)
)
WITH
(
  SYSTEM_VERSIONING = ON
  (
    HISTORY_TABLE = dbo.VehicleTelemetryHistory
  )
);
GO
CREATE INDEX IX_VehicleTelemetry_Time ON dbo.VehicleTelemetry (TelemetryTimeUtc);
CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson) FOR
(
  '$.location. latitude', '$.location. longitude',
  '$.location. accuracy'
);
```

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	○
The code meets the database performance requirements for JSON property querying.	○	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	○

Answer:

Answer Area

Statements	Yes	No
The code meets the database performance requirements for partitioning.	○	●
The code meets the database performance requirements for JSON property querying.	●	○
Queries that filter on `$.location.heading` will use the `JI_VehicleTelemetry_Location` index.	○	●

Explanation:

1. Partitioning Requirements No

Why: The provided Transact-SQL script configures a System-Versioned Temporal Table (SYSTEM_VERSIONING = ON) to track row modifications over time. However, it lacks any partitioning architecture. There are no CREATE PARTITION FUNCTION or CREATE PARTITION SCHEME definitions, nor is there an ON partition_scheme_name(column) clause on the table itself.

2. JSON Property Querying Yes

Why: The script explicitly includes a native JSON index definition targeted at optimizing document paths:

CREATE JSON INDEX JI_VehicleTelemetry_Location

ON dbo.VehicleTelemetry (LocationJson) FOR (...)

This satisfies performance optimization requirements specifically for structured JSON property lookups on the LocationJson column.

3. Filtering on $.location.heading No

Why: A JSON index only indexes the specific JSON paths declared inside its FOR clause. The script only maps three properties:

$.location.latitude

$.location.longitude

$.location.accuracy

Because $.location.heading is not explicitly declared within this index, the engine cannot use JI_VehicleTelemetry_Location to accelerate queries filtering by heading.

Question: 107 CertyIQ

Case Study
Existing Environment
Contoso has an Azure subscription in North Europe that contains the corporate infrastructure. The current infrastructure contains a Microsoft SQL Server 2017 database.
The database contains the following tables.

Table names	Column names
CustomerFeedback	• FeedbackId (int) (primarykey) • FeedbackJson (nvarchar (max))
Fleets	• FleetId (int) (primarykey) • FleetName (nvarchar(100)) • Description (nvarchar(256))
MaintenanceEvents	• MaintenanceId (int) (primarykey) • VehicleId (int) • LastModifiedUTC (datetime2) • Description (nvarchar(256))
SupportTickets	• TicketId (int) (primarykey) • FleetId (int) • CreatedUtc (datetime2)
UserAccounts	• UserId (int) (primarykey) • UserPrincipalName (nvarchar(256)) • JobRole (nvarchar(256)) • StartDate (datetime2)
VehicleHealthSummary	• IncidentId (int) (primarykey) • VehicleId (int) • FleetId (int) • Summary (nvarchar(2000)) • LastUpdatedUtc (datetime2) • EngineStatus [bit] • EngineStatusLastUpdatedUtc (datetime2) • BatteryHealth (int) • Embeddings (vector (1536))

The FeedbackJson column has a full-text index and stores JSON documents in the following format.

```
{
  "text": "The battery drains too fast when driving uphill.",
  "category": "Battery",
  "metadata": {
    "appVersion": "5.2.1",
    "device": "Android",
    "language": "en-GB"
  }
}
```

The support staff at Contoso never has the unmask permission.

Requirements

Contoso is deploying a new Azure SQL database that will become the authoritative data store for the following;

• Al workloads
• Vector search
• Modernized API access
• Retrieval Augmented Generation (RAG) pipelines

Sometimes the ingestion pipeline fails due to malformed JSON and duplicate payloads.

The engineers at Contoso report that the following dashboard query runs slowly.

SELECT VehicleTd, Lastupdatedutc, EngineStatus, BatteryHealth FROM dbo.VehicleHealthSumary where fleetld -gFleetld ORDER BV LastUpdatedUtc DESC;

You review the execution plan and discover that the plan shows a clustered index scan.

vehicleincident Reports often contains details about the weather, traffic conditions, and location.

Analysts report that it is difficult to find similar incidents based on these details.

Planned Changes

Contoso wants to modernize Fleet Intelligence Platform to support Al-powered semantic search over incident reports.

Security Requirements

Contoso identifies the following telemetry requirements:

• Telemetry data must be stored in a partitioned table.
• Telemetry data must provide predictable performance for ingestion and retention operations.
• latitude, longitude, and accuracy JSON properties must be filtered by using an index seek. Contoso identifies the following maintenance data requirements:
• Ensure that any changes to a row in the MaintenanceEvents table updates the corresponding value in the LastModif reduce column to the time of the change.
• Avoid recursive updates.

AI Search, Embedding's, and Vector indexing

The development learn at Contoso will use Microsoft Visual Studio Code and GitHub Copilot and will retrieve live metadata from the databases.

Contoso identifies the following requirements for querying data in the FeedbackJson column of the customer-Feedback table:

• Extract the customer feedback text from the JSON document.
• Filter rows where the JSON text contains a keyword.
• Calculate a fuzzy similarity score between the feedback text and a known issue description.
• Order the results by similarity score, with the highest score first.

You need to recommend a solution to lesolve the slow dashboard query issue.

What should you recommend?

A. Create a clustered index on Lastupdatedutc.

B. On Fleetid, create a nonclustered index that includes Lastupdatedutc. inginestatus, and BatteryHealth.

C. On Lastupdatedutc. create a nonclustered index that includes Fleetid.

D. On Fleetid, create a filtered index where lastupdatedutc > DATEADD(DAV, -7, SYSuTCOATETIME()).

Answer: B

Explanation:

A. Create a clustered index on LastUpdatedUtc Incorrect

A clustered index on LastUpdatedUtc optimizes queries that primarily filter by date, but it does not efficiently support queries that retrieve dashboard data by FleetId.

B. On FleetId, create a nonclustered index that includes LastUpdatedUtc, EngineStatus, and BatteryHealth Correct

A nonclustered index on FleetId with included columns creates a **covering index** for dashboard queries. The query can retrieve LastUpdatedUtc, EngineStatus, and BatteryHealth directly from the index without additional key lookups, significantly improving performance.

C. On LastUpdatedUtc, create a nonclustered index that includes FleetId Incorrect

This index is optimized for date-based searches rather than fleet-based dashboard queries and does not cover all required columns.

D. On FleetId, create a filtered index where LastUpdatedUtc > DATEADD(DAY, -7, SYSUTCDATETIME()) Incorrect

Filtered indexes cannot use non-deterministic functions such as SYSUTCDATETIME() in the filter definition. Therefore, this index cannot be created as specified.

Question: 108

HOTSPOT
You are creating a table that will store customer profiles.
You have the following Transact-SQL code.

```
CREATE TABLE dbo.CustomerProfiles
(
    CustomerId      BIGINT IDENTITY(1,1) PRIMARY KEY,
    FullName        NVARCHAR(200) MASKED WITH (FUNCTION = 'partial(1,"xxxx",1)'),
    EmailAddress    NVARCHAR(200) MASKED WITH (FUNCTION = 'email()'),
    PhoneNumber     NVARCHAR(50)  MASKED WITH (FUNCTION = 'default()'),
    RegionCode      NVARCHAR(10) NOT NULL
);
GO
CREATE FUNCTION dbo.fn_FilterByRegion(@RegionCode NVARCHAR(10))
RETURNS TABLE
AS
RETURN
(
    SELECT 1
    FROM dbo.UserRegionAccess ura
    WHERE ura.UserPrincipalName = SUSER_SNAME()
      AND ura.RegionCode = @RegionCode
);
GO
CREATE SECURITY POLICY CustomerRegionPolicy
ADD FILTER PREDICATE dbo.fn_FilterByRegion(RegionCode)
ON dbo.CustomerProfiles
WITH (STATE = ON);
GO
```

For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection Is worth one point.

Answer Area

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Answer:

Answer Area

Statements	Yes	No
The schema meets the security requirements for PII data.	○	○
Administrators of the Azure SQL server can see all the rows in `dbo.CustomerProfiles` when they use an application.	○	○
The masking rules will apply even when row-level security (RLS) filters out rows.	○	○

Explanation:

The schema meets the security requirements for PII data.

Selection: Yes
Explanation: The T-SQL implementation appropriately applies standard SQL security protocols to safeguard Personally Identifiable Information (PII). It uses **Dynamic Data Masking (DDM)** (via MASKED WITH) on highly sensitive columns (FullName, EmailAddress, PhoneNumber) to limit exposure to non-privileged users, combined with **Row-Level Security (RLS)** via a filter predicate to restrict row access by region.

Administrators of the Azure SQL server can see all the rows in dbo.CustomerProfiles when they use an application.

Selection: No
Explanation: Although Azure SQL Database administrators have elevated privileges (such as the UNMASK permission to see unmasked text values), they are still subject to the active Row-Level Security (RLS) filter predicate. The defined function dbo.fn_FilterByRegion filters rows strictly based on whether the current executing user's SUSER_SNAME() matches a mapping in dbo.UserRegionAccess. Unless the administrator's username is explicitly populated inside that region table for every region, the RLS policy will filter out the unauthorized rows from their query path.

The masking rules will apply even when row-level security (RLS) filters out rows.

Selection: No
Explanation: Row-Level Security and Dynamic Data Masking operate at different levels of query execution. RLS works first to evaluate row eligibility; if a row is filtered out by RLS, it is eliminated from the query evaluation path entirely and never enters the final result set. Because Dynamic Data Masking is applied only to obfuscate values returned in the final result set, it cannot act on rows that have already been excluded by RLS.

Question: 109

CertyIQ

You have an Azure SQL database named ProductsDB.
You deploy Data API builder (DAB) to Azure Container Apps by using the mcr.microsoft.com/azure-databases /data-api-builder:latest image.
The container app has the following configurations:
- Secrets: mssql-connection-string, dab-config-base64
- Environment variables:
- MSSQL_CONNECTION_STRING=secretref:mssql-connection-string
- DAB_CONFIG_BASE64=secretref:dab-config-base64
- Ingress: External on port 5000
Users report that the /health endpoint returns a healthy response, but all requests that query an entity named Products fail and generate a connection error.
You confirm that the SQL login in the connection string is correct and the database exists.
You need to ensure that the container app can establish connections to the Azure SQL logical server without changing the container app deployment settings or the DAB configuration file.
What should you do on the Azure SQL logical server?

A. Create an auto-failover group for ProductsDB.
B. Run DBCC CHECKDB on ProductsDB.
C. Create a firewall rule that allows a start and end IP address of 0.0.0.0.
D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for ProductsDB.

Answer: C

Explanation:

A. Create an auto-failover group for ProductsDB Incorrect

Auto-failover groups provide high availability and disaster recovery capabilities. They do not resolve connectivity issues between Azure Container Apps and Azure SQL Database.

B. Run DBCC CHECKDB on ProductsDB Incorrect

DBCC CHECKDB verifies database integrity and checks for corruption. It does not affect network connectivity or firewall access.

C. Create a firewall rule that allows a start and end IP address of 0.0.0.0 Correct

A firewall rule with both the start and end IP address set to 0.0.0.0 enables **Allow Azure services and resources to access this server**. Since the container app is running in Azure, this allows Data API Builder to connect to the Azure SQL logical server without modifying the container app or DAB configuration.

D. Enable FORCE_LAST_GOOD_PLAN automatic tuning for ProductsDB Incorrect

Automatic tuning optimizes query performance by correcting plan regressions. It does not resolve connection failures between Azure services and Azure SQL Database.

Question: 110 CertyIQ

You have an Azure SQL database that contains a table named Sales.Customer. Sales.Customer contains columns named Customerld, FullName, Email, TaxID, and Regionld.
You have a database role named AppSupport that is used by a support application.
You need to implement a security solution for AppSupport that meets the following requirements:
- AppSupport must be prevented from viewing TaxID.
- AppSupport must be able to query Sales. Customer to troubleshoot issues.
Appsupport must be able to run a stored procedure named Sales.usp_GetCustomerByCustomerld.
Which Transact-SQL statements should you include in the solution? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

To run Sales.usp_GetCustomerByCustomerId:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

To query Sales.Customer:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

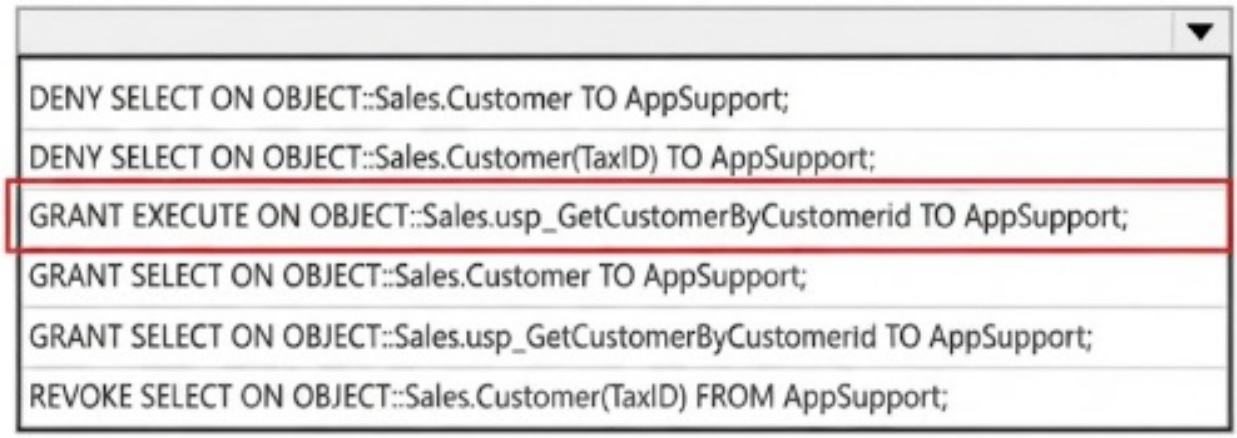
To prevent viewing TaxID:

- DENY SELECT ON OBJECT::Sales.Customer TO AppSupport;
- DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;
- GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;
- GRANT SELECT ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;
- REVOKE SELECT ON OBJECT::Sales.Customer(TaxID) FROM AppSupport;

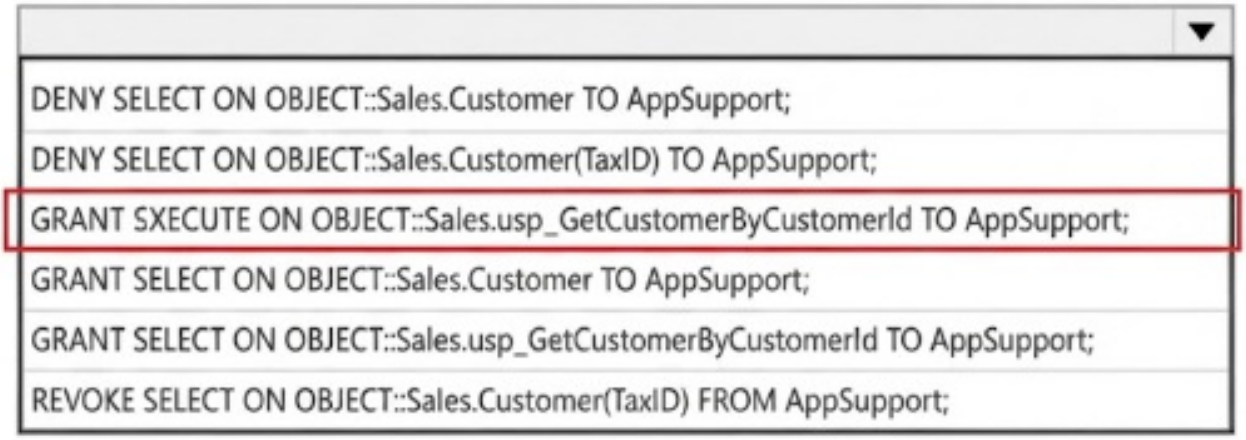
Answer:

Answer Area

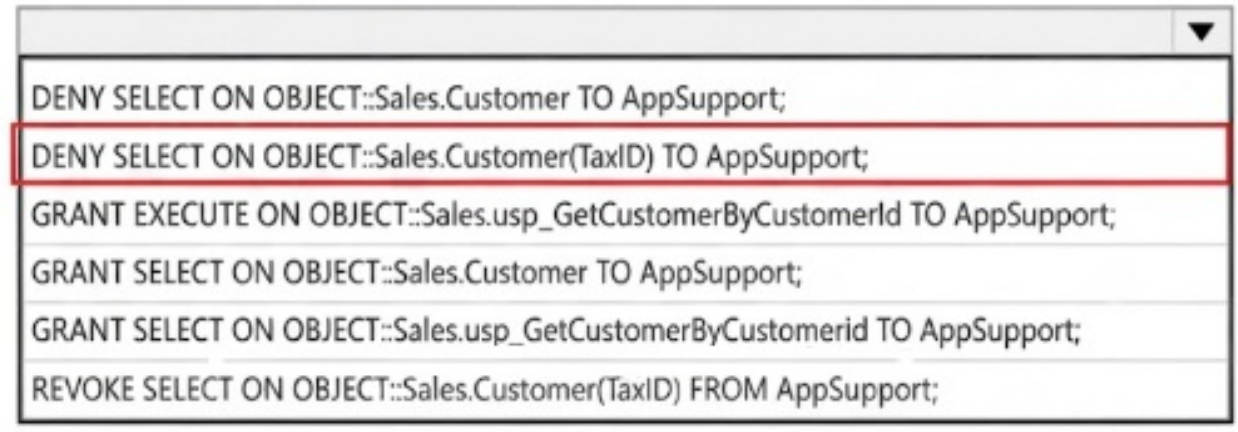
To run Sales.usp_GetCustomerByCustomerId:



To query Sales.Customer:



To prevent viewing TaxID:



Explanation:

1. To run Sales.usp_GetCustomerByCustomerId

Correct Selection: GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;

Explanation: To allow a database role to run/execute a stored procedure, you must grant them the EXECUTE permission explicitly on that specific object.

2. To query Sales.Customer

Correct Selection: GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;

Explanation: To allow users to query data from a table (SELECT * FROM ...), you need to provide them with

table-level SELECT permissions.

3. To prevent viewing TaxID

Correct Selection: DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;

Explanation: A DENY permission always overrides any granted permissions in SQL Server. By applying a column-level DENY SELECT specifically on the TaxID column, members of the AppSupport role will still be able to query all other columns in the table but will receive an error or be blocked if they attempt to view the sensitive TaxID data.

Question: 111 CertyIQ

You are developing an Azure SQL database solution from a locally cloned GitHub repository by using licrosoft Visual Studio Code and GitHub Copilot Chat.
You need to ensure that GitHub Copilot Chat can call the hosted GitHub MCP Server tools by using OAuth. The MCP server configuration must be scoped to the repository.
What should you do in Visual Studio Code?

A. Create a personal access token (PAT) that has the repo scope and store the PAT in the vscodelmcp.json file as the Authorization header.

B. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the workspace settings.

C. From the Command Palette, enter MCP: add server, select HTTP (HTTP or Server-Sent Events), enter https://api.githubcopilot.com/mcp/, and then save the configuration to the user settings.

D. Create a personal access token (PAT) that has the repo scope and store the PAT in the githublmcp.json file as the Authorization header.

Answer: B

Explanation:

A. Store a PAT in vscode\mcp.json Incorrect

The hosted GitHub MCP Server uses **OAuth**, not a manually supplied PAT. Additionally, storing credentials in configuration files is unnecessary and less secure.

B. Add the MCP server and save it to the workspace settings Correct

Using **MCP: Add Server** with https://api.githubcopilot.com/mcp/ configures the hosted GitHub MCP Server to use OAuth authentication. Saving the configuration to **workspace settings** scopes it to the current repository, meeting the requirement.

C. Save the configuration to the user settings Incorrect

User settings apply globally across Visual Studio Code and are not scoped to a specific repository.

D. Store a PAT in .github\mcp.json Incorrect

The hosted GitHub MCP Server uses OAuth authentication rather than PAT-based authentication, and storing PATs in repository files is not required.

Question: 112 CertyIQ

What is the primary purpose of Azure AI in SQL development?

A. Replace SQL Server
B. Automate infrastructure deployment
C. Enhance querying and analytics with AI capabilities
D. Eliminate the need for databases

Answer: C

Explanation:

A. Replace SQL Server Incorrect

Azure AI is designed to complement and enhance SQL solutions, not replace SQL Server or databases.

B. Automate infrastructure deployment Incorrect

Infrastructure deployment is typically handled by tools such as Azure Resource Manager (ARM), Terraform, or CI/CD pipelines, not by Azure AI itself.

C. Enhance querying and analytics with AI capabilities Correct

Azure AI integrates with SQL solutions to provide capabilities such as natural language querying, semantic search, embeddings, intelligent analytics, and Retrieval Augmented Generation (RAG), improving how users analyze and interact with data.

D. Eliminate the need for databases Incorrect

Azure AI relies on data stored in databases and other sources. It does not remove the need for databases.

Question: 113 CertyIQ

You have a database that contains production data. The schema is stored in a Git repository as an SDK-style SQL database project and contains the following reference data.

Name	Data type
RefId	int identity
Code	nvarchar(10)
CreateDate	datetime2
Description	nvarchar(255)

A deployment pipeline can be rerun automatically when a transient failure occurs.You need to deploy the reference data as part of the same CI/CD process. Rerunning the pipeline must produce the same outcome and must NOT create duplicate rows.What should you do?

A. Add a post-deployment script that inserts reference rows by using IF NOT EXISTS or MERGE logic.

B. Restore a backup after each deployment.

C. Store the reference values in GitHub repository secrets.

Answer: A

Explanation:

A. Add a post-deployment script that inserts reference rows by using IF NOT EXISTS or MERGE logic Correct

Using IF NOT EXISTS or MERGE makes the deployment **idempotent**. If the pipeline is rerun, the script inserts only missing rows or updates existing rows as needed, preventing duplicate reference data.

B. Restore a backup after each deployment Incorrect

Restoring a backup after every deployment is unnecessary, disruptive, and does not provide an idempotent way to deploy reference data.

C. Store the reference values in GitHub repository secrets Incorrect

GitHub secrets are intended for sensitive information such as passwords, tokens, and connection strings, not for storing reference data that must be deployed to a database.

Question: 114

CertyIQ

You have an Azure Al Search service and an index named hotels that includes a vector field named DescriptionVector.
You query hotels by using the Search Documents REST API.
You need to implement a hybrid search query that uses DescriptionVector and includes captions.
How should you complete the REST request body? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.

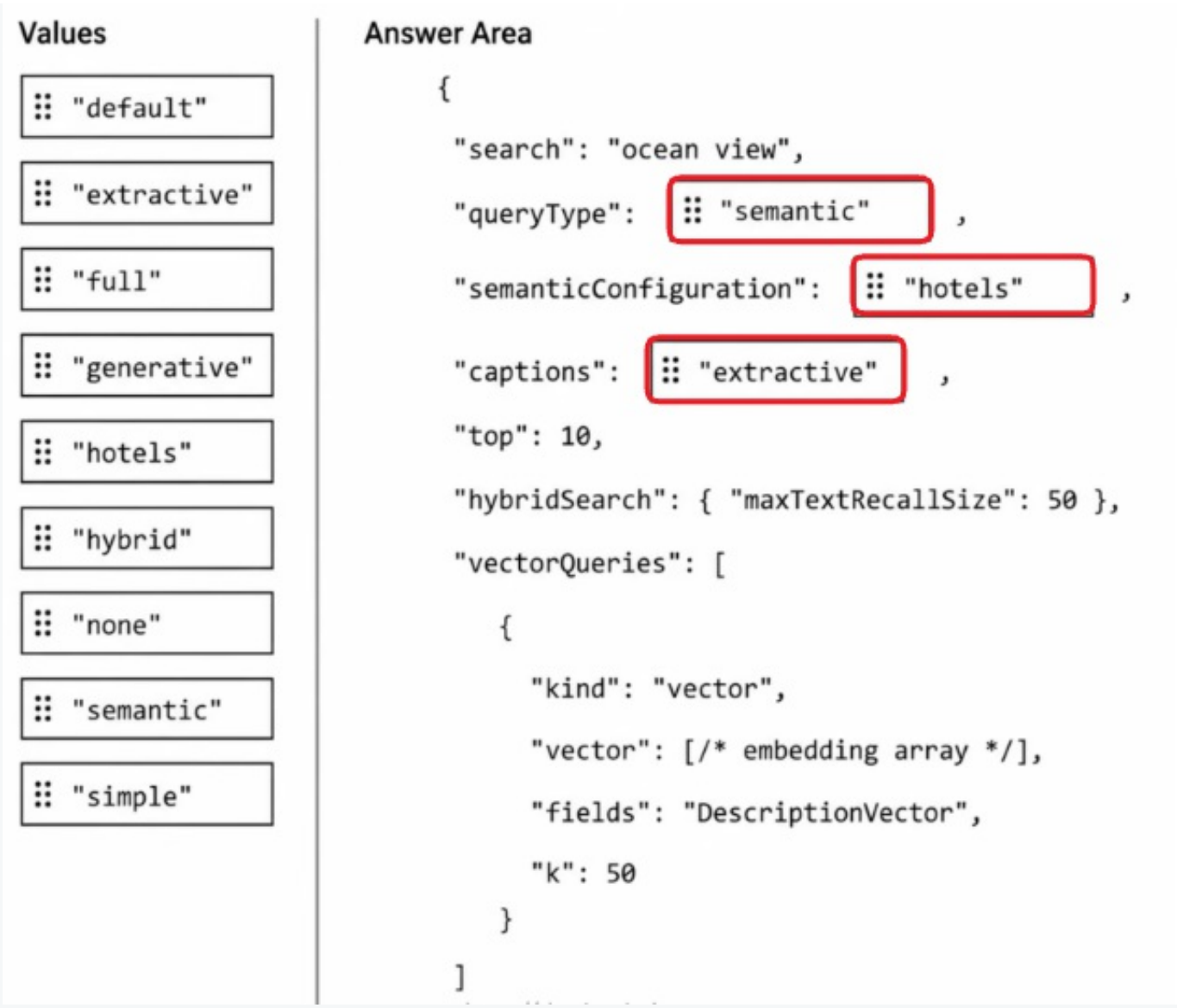
Values

- "default"
- "extractive"
- "full"
- "generative"
- "hotels"
- "hybrid"
- "none"
- "semantic"
- "simple"

Answer Area

```
{
 "search": "ocean view",
 "queryType": [          ] ,
 "semanticConfiguration": [          ] ,
 "captions": [          ] ,
 "top": 10,
 "hybridSearch": { "maxTextRecallSize": 50 },
 "vectorQueries": [
    {
      "kind": "vector",
      "vector": [/* embedding array */],
      "fields": "DescriptionVector",
      "k": 50
    }
 ]
```

Answer:



Explanation:

Target 1 (queryType): "semantic"

Reasoning: To use advanced secondary ranking, semantic reranking models, and feature extras like captions, you must explicitly set the queryType property to "semantic".

Target 2 (semanticConfiguration): "hotels"

Reasoning: This parameter defines which custom or predefined index arrangement models handle field prioritization. It maps directly to the name of the reference configuration defined on your search index template, which is "hotels".

Target 3 (captions): "extractive"

Reasoning: Azure AI Search generates summary text highlights for text chunks by locating the most relevant phrasing. Setting this parameter to "extractive" activates the extraction models to isolate key snippets matching your query context.

Question: 115 CertyIQ

You need to recommend a solution for the TransactionProcessing application that meets the security requirements. What should you include in the recommendation?

A. a service principal

B. a system-assigned managed identity

C. a shared access key

D. a user-assigned managed identity

Answer: A

Explanation:

A **service principal** represents an application in Microsoft Entra ID and allows it to authenticate securely using a client secret or certificate. It is the appropriate choice when:

The application is **not hosted on Azure**, or

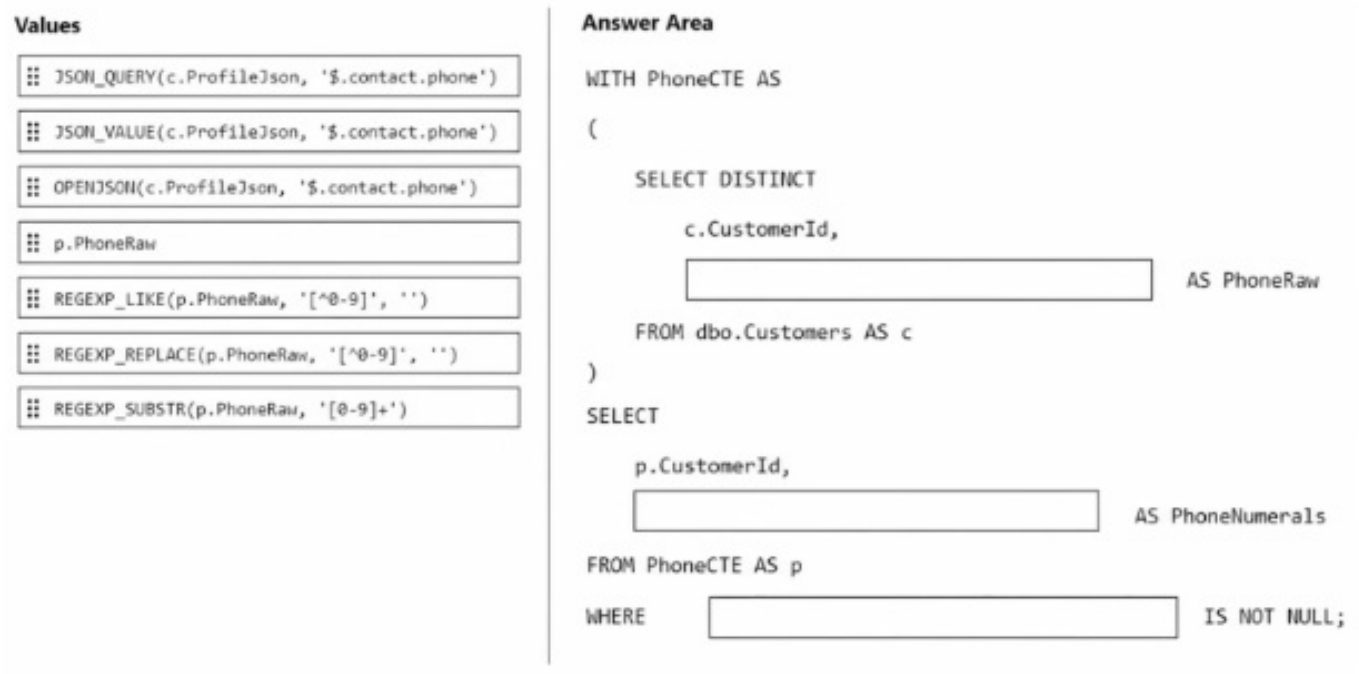
It cannot use managed identities, or

It requires an identity independent of any Azure resource.

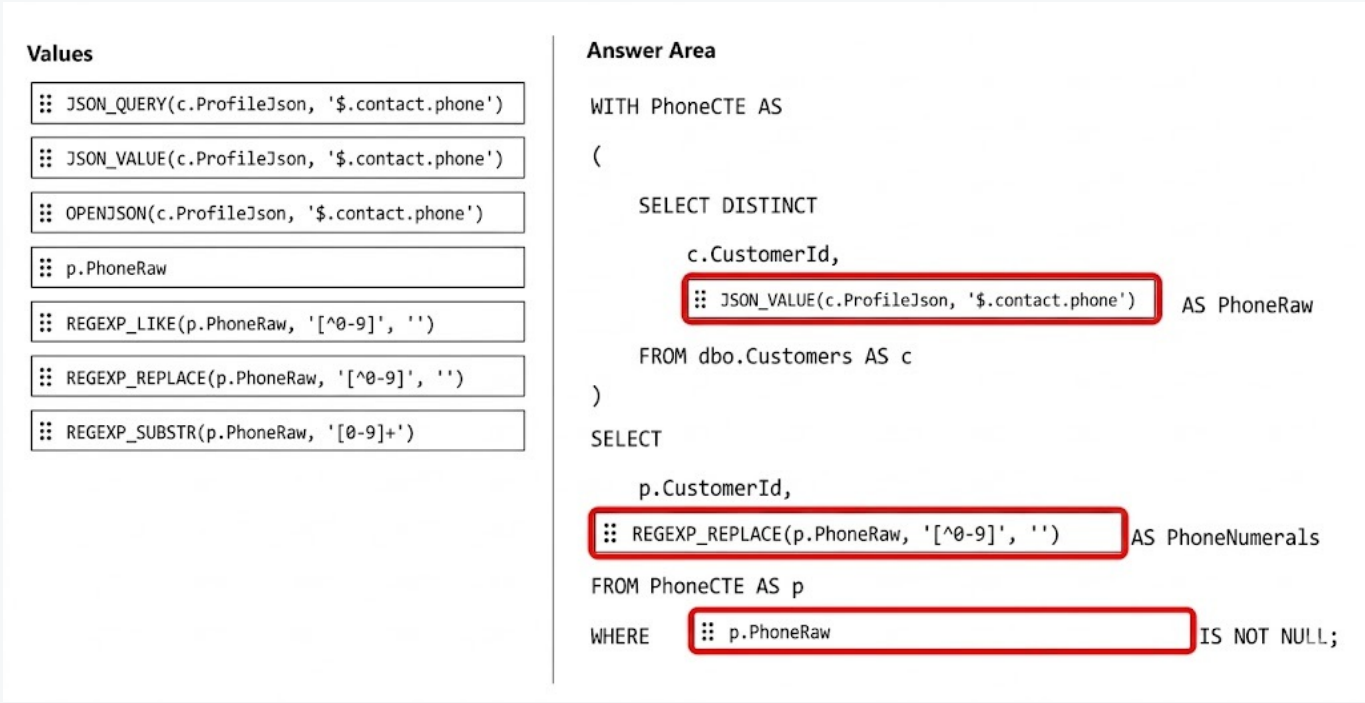
Question: 117

CertyIQ

Drag and Drop Question
You have an Azure SQL database that contains a table named dbo.Customers, dbo.Customers contains the following columns:
- Customerld (int)(primary key)
- ProfileJson (nvarchar(max))
You have an application that returns phone numbers in a format of +000 000-000-0000. The phone numbers are stored in Profilelson.
You need to write a query that returns:
- One row per customer
A PhoneNUmerals column that contains only the digits
low should you complete the Transact-SOL query? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Target 1 (PhoneRaw definition inside the CTE): JSON_VALUE(c.ProfileJson, '$.contact.phone')

Reasoning: Since a phone number is a single, scalar text value inside the JSON block, JSON_VALUE is the correct function to parse the document and extract it. JSON_QUERY would only be used if you were extracting an object or an array, and OPENJSON returns a rowset table rather than a scalar string expression.

Target 2 (PhoneNumerals definition in the outer query): REGEXP_REPLACE(p.PhoneRaw, '[^0-9]', '')

Reasoning: The caret symbol inside the brackets ([^0-9]) acts as a negation operator, matching any character that is not a digit from 0 to 9. By passing this pattern to REGEXP_REPLACE along with an empty string (''), all spaces, plus signs (+), and hyphens are completely stripped out, leaving only the digits.

Target 3 (WHERE filter constraint): p.PhoneRaw

Reasoning: The syntax suffix of the filter is already written out as IS NOT NULL;. To complete the predicate logically, you simply pass the alias name of the extracted raw string column, p.PhoneRaw, ensuring rows without a phone entry are cleanly filtered out.

Question: 118

CertyIQ

Note: This question is part of a series of questions that present the same scenario. Each question in the series contains a unique solution that might meet the stated goals. Some question sets might have more than one correct solution, while others might not have a correct solution.
After you answer a question in this section, you will NOT be able to return to it. As a result, these questions will not appear in the review screen.
You have an SDK-style SQL database project stored in a Git repository. The project targets an Azure SQL database.
The CI build fails with unresolved reference errors when the project references system objects.
You need to update the SQL database project to ensure that dotnet buildvalidates successfully by including the correct system objects in the database model for Azure SQL Database.
Solution: Add an artifact reference to the Azure SQL Database master dacpac file.
Does this meet the goal?

A. Yes

B. No

Answer: A

Explanation:

A. Yes Correct

Adding an artifact reference to the **Azure SQL Database master DACPAC** includes the Azure SQL system objects (such as system views, built-in procedures, and metadata objects) in the database model. This resolves unresolved reference errors and allows dotnet build to validate successfully.

This is functionally equivalent to referencing the Microsoft.SqlServer.Dacpacs.Azure.Master package, as both approaches provide the Azure SQL master database schema needed during compilation.

B. No Incorrect

Without a reference to the Azure SQL master DACPAC, references to system objects remain unresolved, causing build failures.

Question: 119 CertyIQ

You have an Azure container app named app1-contoso-001 that hosts a Data API builder (DAB) container in front of an Azure SQL database.
You add an entity named Todo that uses a source named dbo.todos.
Which URL pattern should clients use to access the Todo REST endpoint?

A. https://appl-contoso-001.azurestaticapps.net/data-api/graphq1/Todo
B. https://appl-contoso-001.azurestaticapps.net/api/Todo
C. https://appl-contoso-001.azurestaticapps.net/data-api/Todo
D. https://appl-contoso-001.azurestaticapps.net/data-api/api/Todo

Answer: B

Explanation:

A. https://app1-contoso-001.azurestaticapps.net/data-api/graphql/Todo Incorrect

The /graphql endpoint is used for GraphQL operations, not for REST entity endpoints.

B. https://app1-contoso-001.azurestaticapps.net/api/Todo Correct

By default, Data API Builder exposes REST endpoints under the /api route prefix. Therefore, an entity named Todo is accessed at:

https://app1-contoso-001.azurestaticapps.net/api/Todo

C. https://app1-contoso-001.azurestaticapps.net/data-api/Todo Incorrect

/data-api is not the default REST route prefix for Data API Builder unless it has been explicitly configured.

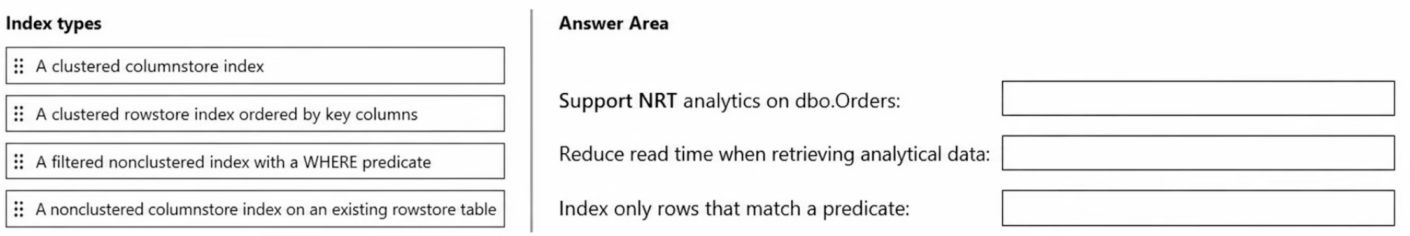
D. https://app1-contoso-001.azurestaticapps.net/data-api/api/Todo Incorrect

This path combines two route prefixes and is not a valid default DAB endpoint.

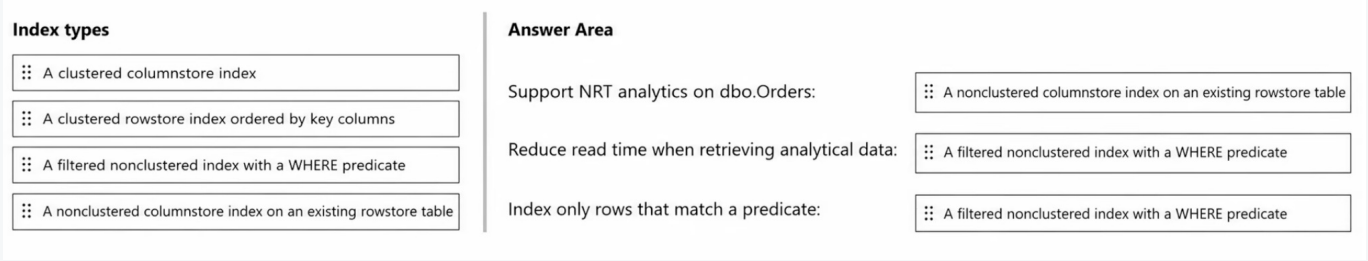
Question: 120

CertyIQ

You have an Azure SQL database named SalesDB that supports an ecommerce application.
SalesDB contains a table named dbo.Orders that has a clustered index on a column named Orderld. dbo.Orders receives continuous OLTP inserts and updates during business hours.
Your analytics team runs hourly aggregate queries that scan dbo.Orders to calculate revenue trends for recent dates.
You need to improve the performance of the hourly analytics queries without significantly affecting OLTP throughput. The solution must meet the following requirements:
• Support near-real-time (NRT) analytics on the dbo.Orders table.
- Reduce read time when retrieving analytical data from dbo.Orders.
- Support indexing only rows that match a predicate, such as Active =1.
Which type of index should you use for each requirement? To answer, drag the appropriate index types to the correct requirements. Each index type may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Hybrid Transactional-Analytic Processing (HTAP): A **nonclustered columnstore index** is the industry standard for supporting "near-real-time" (NRT) analytics on an active OLTP table. It keeps the base table in rowstore format (optimal for your continuous inserts/updates) while creating a highly compressed, read-optimized columnar copy for your hourly aggregate queries.
Analytical Throughput: For pure analytical read performance, **A filtered nonclustered index with a WHERE predicates** the most powerful tool available. It organizes the entire table in a columnar format, which provides maximum compression and allows the engine to skip vast amounts of irrelevant data (via segment elimination), drastically reducing total read time.
Specialized Indexing: Using a **filtered nonclustered index** is the correct strategy for your third requirement because it allows you to maintain a very small, highly specific index (e.g., WHERE Active = 1). This avoids the overhead of indexing the entire table, reducing maintenance costs and storage space for those specific filtered lookups.

Question: 121

CertyIQ

You are developing an Azure SQL solution by using Microsoft Visual Studio 2026. The solution uses a Github repository.
You plan to use GitHub Copilot Chat to access the GitHub repository tools by connecting to the Github MCP Server.
You need to configure Visual Studio to support the planned configuration. The solution must rely on OAuth to

access the MCP server.
What should you create?

A. the SOLUTIONDIR>. vslsettings.json file that contains the personal access token (PAT) that has the repo scope

B. the \.mcp.json file that contains a URL of https://api.githubcopilot.com/mcp/

C. the \.vscodelsettings.json file that contains a URL of https://api.githubcopilot.com/mcp/

D. the \.github\mcp.json file that contains the personal access token (PAT) that has the repo scope

Answer: B

Explanation:

A. <SOLUTIONDIR>\.vs\settings.json with a PAT Incorrect

The GitHub MCP Server uses OAuth authentication, not a manually supplied Personal Access Token (PAT).

B. <SOLUTIONDIR>\.mcp.json with https://api.githubcopilot.com/mcp/ Correct

Visual Studio supports repository-scoped MCP server configuration through a .mcp.json file in the solution directory. Specifying the GitHub MCP Server URL enables GitHub Copilot Chat to authenticate via OAuth and access repository tools.

C. <SOLUTIONDIR>\.vscode\settings.json with the URL Incorrect

This file is used by Visual Studio Code, not Visual Studio 2026.

D. <SOLUTIONDIR>\.github\mcp.json with a PAT Incorrect

The hosted GitHub MCP Server uses OAuth, so a PAT is unnecessary and does not meet the requirement.

Question: 122 CertyIQ

Which tool helps orchestrate data pipelines for Al workflows?

A. Azure DevOps

B. Azure Data Factory

C. Power BI

D. Azure Monitor

Answer: B

Explanation:

A. Azure DevOps Incorrect

Azure DevOps is used for source control, CI/CD pipelines, and application lifecycle management. It does not primarily orchestrate data pipelines for AI workflows.

B. Azure Data Factory Correct

Azure Data Factory (ADF) is a cloud-based data integration and orchestration service. It is commonly used to build, schedule, and manage data pipelines that prepare and move data for AI and machine learning workflows.

C. Power BI Incorrect

Power BI is a business intelligence and data visualization tool used for reporting and dashboards, not for orchestrating data pipelines.

D. Azure Monitor Incorrect

Azure Monitor provides monitoring, logging, and alerting capabilities for Azure resources. It does not orchestrate data pipelines.

Question: 123 CertyIQ

You have an Azure SQL database named SalesDB and an Azure App Service app named sales- api.
SalesDB contains a table named dbo.Customers. dbo.Customers contains two columns named CreditCardNumber and TenantId.
Currently, sales-api connects to SalesDB by using SQL authentication with stored username and password.
You need to recommend a solution that meets the following requirements:
- Provides a passwordless method for sales-api to access SalesDB.
- Ensures that credit card numbers are NOT stored as plain text.
What should you include in the recommendation?

A. Azure Key Vault and Always Encrypted
B. Transparent Data Encryption (TDE) and row-level security (RLS)
C. managed identities and dynamic data masking
D. managed identities and Always Encrypted

Answer: D

Explanation:

A. Azure Key Vault and Always Encrypted Incorrect

Always Encrypted protects sensitive data, but Azure Key Vault alone does not provide a passwordless authentication method for sales-api to connect to SalesDB. The application would still need credentials unless managed identities are used.

B. Transparent Data Encryption (TDE) and row-level security (RLS) Incorrect

TDE encrypts data at rest but SQL Server decrypts it during queries, so credit card numbers are still accessible in plaintext to authorized users. RLS restricts row access and does not provide passwordless authentication.

C. Managed identities and dynamic data masking Incorrect

Managed identities provide passwordless access, but dynamic data masking only obscures data in query results. The actual credit card numbers are still stored in plaintext in the database.

D. Managed identities and Always Encrypted Correct

Managed identities provide a **passwordless authentication** mechanism for sales-api to access Azure SQL Database without storing usernames or passwords.

Always Encrypted ensures that sensitive data such as CreditCardNumber is encrypted and not stored as plaintext in the database.

Question: 124 CertyIQ

You have an Azure SQL database that contains a table named Table1. Table1 contains 25,000,000 rows of data and a datetime2 column named DateKey. The data in Table1 spans the years 2020 through 2021.
You need to partition the data in Table1 by year. The solution must minimize how long it takes to rebuild or reindex the table.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

```
CREATE  PARTITION FUNCTION PartitionByYear (datetime2)
AS  [ PARTITION | RANGE LEFT | RANGE RIGHT ]
FOR VALUES ( [ '2019-01-01 00:00:00', '2020-01-01 00:00:00', '2021-12-31 23:59:59' |
               '2019-12-31 00:00:00', '2020-12-31 23:59:59' |
               '2020-01-01 00:00:00', '2020-02-01 00:00:00', '2020-03-01 00:00:00' |
               '2020-01-01 00:00:00', '2021-01-01 00:00:00' ] );
```

Answer:

Answer Area

```
CREATE  PARTITION FUNCTION PartitionByYear (datetime2)
AS  [ PARTITION | RANGE LEFT | RANGE RIGHT ]
FOR VALUES ( [ '2019-01-01 00:00:00', '2020-01-01 00:00:00', '2021-12-31 23:59:59' |
               '2019-12-31 00:00:00', '2020-12-31 23:59:59' |
               '2020-01-01 00:00:00', '2020-02-01 00:00:00', '2020-03-01 00:00:00' |
               '2020-01-01 00:00:00', '2021-01-01 00:00:00' ] );
```

Explanation:

1. Range Specification

Correct Selection: RANGE RIGHT

Explanation: In SQL Server, RANGE RIGHT specifies that the boundary value itself belongs to the **right-hand side** (the newer/higher partition interval). When dealing with date and time data types like datetime2, using RANGE RIGHT is an industry best practice because it makes clean, midnight boundaries simple to declare (e.g., exactly 2020-01-01 00:00:00). Any record landing exactly on midnight or later falls perfectly into the intended year's bucket.

2. FOR VALUES Boundary List

Correct Selection: '2020-01-01 00:00:00', '2021-01-01 00:00:00'

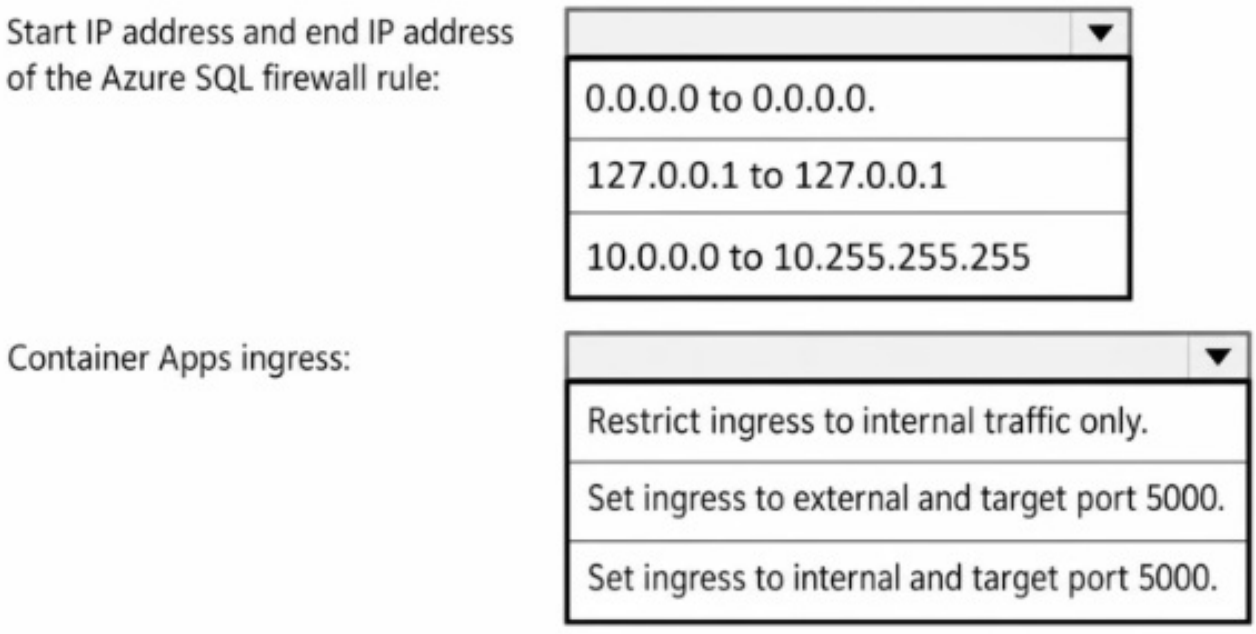
Explanation: Since your data spans across 2020 and 2021, providing these two boundary markers alongside the RANGE RIGHT clause creates exactly three distinct physical data buckets:

1. **Partition 1:** Values strictly less than 2020-01-01 (handles historical data drift cleanly).
2. **Partition 2 (Year 2020):** Values starting from 2020-01-01 00:00:00 up to (but not including) 2021-01-01.
3. **Partition 3 (Year 2021):** Values starting from 2021-01-01 00:00:00 onwards.

Question: 125 CertyIQ

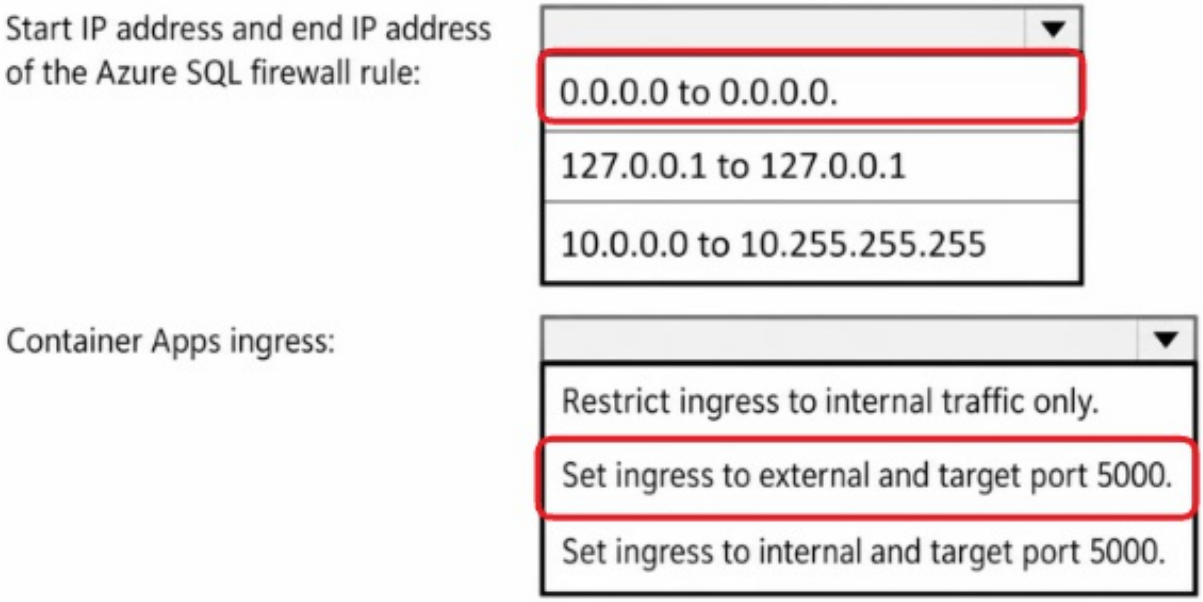
You have an Azure SQL database named ProductsDB.
You deploy Data API builder (DAB) to Azure Container Apps.
You discover that the container app cannot connect to ProductsDB.
Your development team reports that the container app is unreachable from the internet for integration tests.
You need to update Azure SQL Database and Container Apps to meet the following requirements:
- Ensure that the Azure SOL logical server allows connections from Azure services.
- Ensure that the Container Apps environment accepts inbound requests from the public internet.
What should you configure? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area



Answer:

Answer Area



Explanation:

1. Start IP address and end IP address of the Azure SQL firewall rule:

Correct Selection: 0.0.0.0 to 0.0.0.0
Explanation: In Azure SQL Database, creating a firewall rule where both the start and end IP addresses are set to exactly 0.0.0.0 is the specific, built-in flag that toggles the option **"Allow Azure services and resources to access this server"** to ON. This allows Azure Container Apps to communicate through the logical server's firewall.

2. Container Apps ingress:

Correct Selection: Set ingress to external and target port 5000.
Explanation: To make the container application reachable from the public internet for your team's integration testing, its ingress visibility must be explicitly configured as external. Furthermore, Data API builder (DAB) explicitly listens on its standard default runtime web host port of 5000 inside the container container environment.

Question: 126 CertyIQ

You have a GitHub Enterprise subscription.
Your team is developing an Azure SQL dataset solution from a locally cloned GitHub repository by using Microsoft Visual Studio Code and GitHub Copilot Chat.
A mix of GitHub Copilot instructions is configured at different levels, including organization-wide, repository-wide, agent-specific, and personal.
Which instructions will take precedence over the others?

A. repository-wide
B. personal
C. organization-wide
D. agent-specific

Answer: B

Explanation:

A. Repository-wide Incorrect

Repository-wide instructions apply to all users working in a repository, but they can be overridden by a user's personal instructions.

B. Personal Correct

Personal instructions have the highest precedence. They allow individual developers to customize GitHub Copilot Chat behavior according to their preferences and override organization-wide, repository-wide, and agent-specific instructions when conflicts occur.

C. Organization-wide Incorrect

Organization-wide instructions provide default guidance across the enterprise but have lower precedence than repository and personal instructions.

D. Agent-specific Incorrect

Agent-specific instructions apply to a specific agent's behavior, but user **personal instructions take precedence** over them.

Precedence order (highest to lowest):

1. **Personal instructions**
2. Agent-specific instructions
3. Repository-wide instructions
4. Organization-wide instructions

Question: 127 CertyIQ

You have an Azure SQL database that contains tables named dbo.Tickets and dbo.TicketNotes. dbo.Tickets contains support tickets and TicketNotes contains ticket notes.
A retrieval query returns the top five relevant ticket notes for a user question.
You plan to implement a Retrieval Augmented Generation (RAG) pattern that meets the following requirements:
- Formats the retrieved relational data for large language model (LLM) processing
- Sends the user question and retrieved context to an Azure OpenAI REST endpoint for chat completions
- Extracts the response text from the LLM response
Which Transact-SQL function should you use to extract the response text?

A. JSON_MODIFY(@response, '$.result')

B. OPENSON(@response)

C. JSON_QUERY@response, '$.choices')

D. JSON_VALUE(@response, '$.choices[O].message.content')

Answer: D

Explanation:

A. JSON_MODIFY(@response, '$.result') Incorrect

JSON_MODIFY is used to update or insert values in a JSON document. It does not extract values from JSON.

B. OPENJSON(@response) Incorrect

OPENJSON parses JSON into rows and columns. While it can be used to shred JSON, it is not the simplest way

to extract the specific response text from the Azure OpenAI chat completion response.

C. JSON_QUERY(@response, '$.choices') Incorrect

JSON_QUERY returns a JSON object or array, such as the entire choices array. It does not return the scalar response text.

D. JSON_VALUE(@response, '$.choices[0].message.content') Correct

Azure OpenAI chat completions return the generated text in:

```
"choices": [


 "message":

  "content": "Generated answer text"




]
```

Question: 128 CertyIQ

What is a key benefit of embedding vectors in SQL databases?

A. Faster backups
B. Improved indexing
C. Semantic search capability
D. Reduced storage

Answer: C

Explanation:

A. Faster backups Incorrect

Embedding vectors do not improve database backup speed. Backups depend on factors such as database size, storage, and backup mechanisms.

B. Improved indexing Incorrect

While vector indexes can be created for embeddings, the primary benefit of embeddings themselves is not traditional indexing but enabling semantic similarity searches.

C. Semantic search capability Correct

Embedding vectors represent the semantic meaning of text, images, or other data as numerical vectors. They enable **semantic search**, allowing users to find information based on meaning and similarity rather than exact keyword matches.

Examples include:

Retrieval Augmented Generation (RAG)

Similar document search
Recommendation systems
Natural language queries

D. Reduced storage Incorrect

Embeddings typically increase storage requirements because vector data (for example, VECTOR(1536)) must be stored alongside the original data.

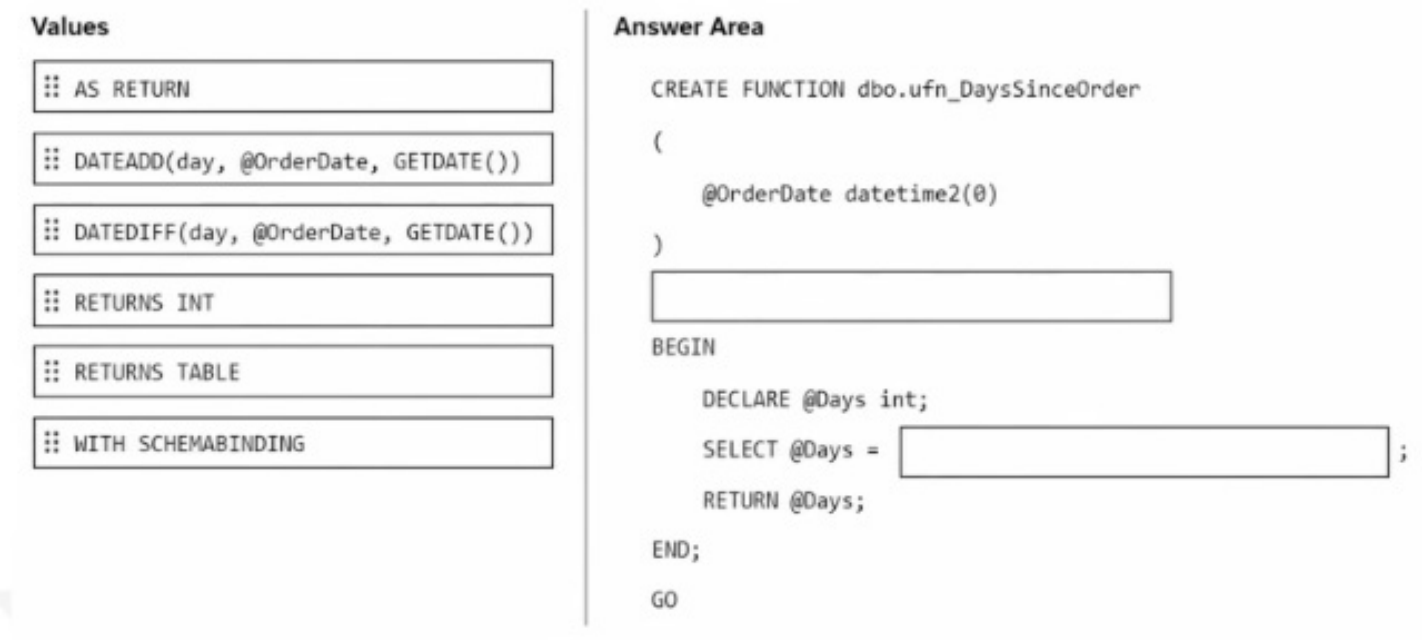
Question: 129

CertyIQ

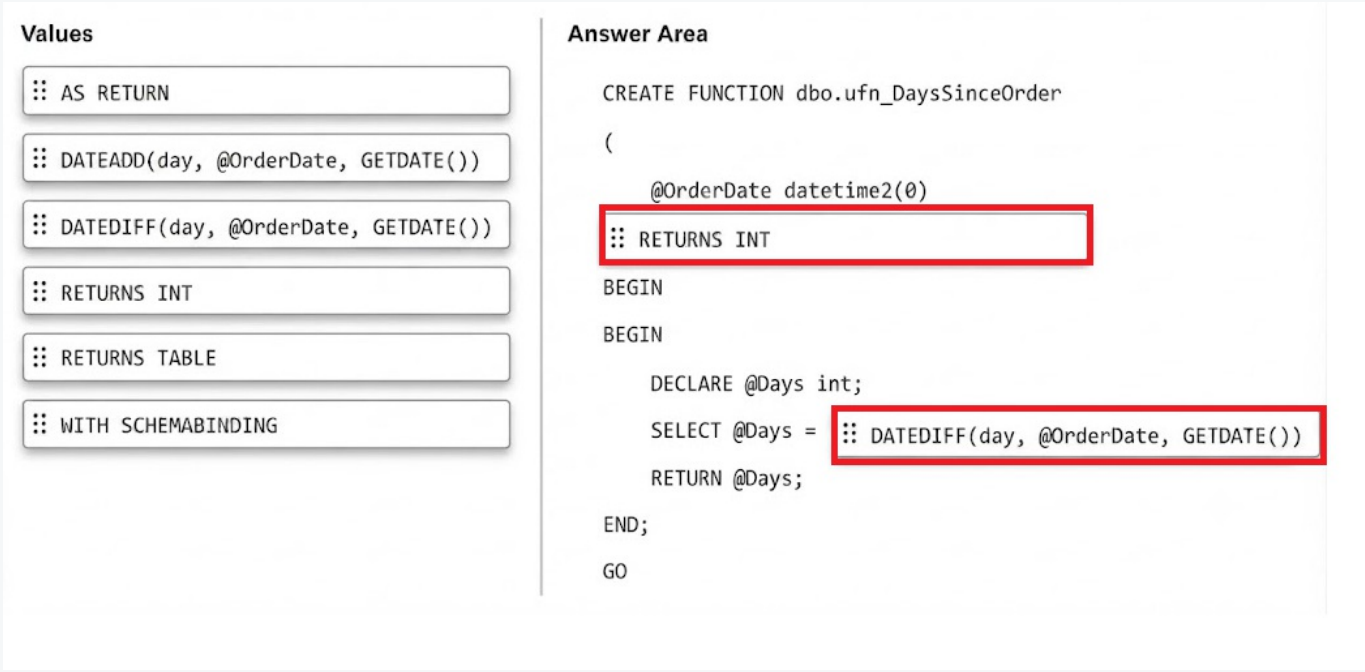
Drag and Drop Question
You have an Azure SQL database that contains a table named Sales.Orders.
Sales.Orders contains the following columns.

Column	Data type
OrderId	int
CustomerId	int
OrderDate	datetime2
TotalAmount	decimal(18,2)

Reporting queries frequently repeat logic to calculate the number of days since an order was placed.
You need to create a scalar user-defined function (UDF) that returns the number of days between an input value of @OrderDate and the current date and time.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.
NOTE: Each correct selection is worth one point.



Answer:



Explanation:

Target 1 (Function Return Declaration): RETURNS INT)

Reasoning: In a T-SQL scalar function definition, you must use the plural keyword RETURNS followed by the scalar data type that the routine will spit out (in this case, an integer INT representing the count of days). RETURNS TABLE is only used for inline or multi-statement table-valued functions.

Target 2 (Calculation Expression): DATEDIFF(day, @OrderDate, GETDATE())

Reasoning: The DATEDIFF function calculates the number of datepart boundaries crossed between two specified dates. Specifying day as the interval parameter evaluates the day difference between the earlier start date (@OrderDate) and the later current system timestamp (GETDATE()). Using DATEADD would add an interval to a date instead of calculating a difference.

Question: 130 CertyIQ

What is the role of Azure Synapse in AI + SQL solutions?

A. Data warehousing and analytics
B. Authentication
C. DNS management
D. Email services

Answer: A

Explanation:

A. Data warehousing and analytics Correct

Azure Synapse Analytics is an integrated analytics service that combines **data warehousing, big data analytics, and data integration**. In AI + SQL solutions, it enables organizations to ingest, prepare, analyze, and query large datasets that can be used for machine learning, AI, and business intelligence workloads.

B. Authentication Incorrect

Authentication is handled by services such as Microsoft Entra ID, not Azure Synapse.

C. DNS management Incorrect

DNS management is provided by services such as Azure DNS, not Azure Synapse.

D. Email services Incorrect

Azure Synapse does not provide email services.

Question: 131 CertyIQ

Which scenario best fits AI-enhanced SQL querying?

A. Static reports
B. Manual backups
C. Chatbot querying database data
D. File compression

Answer: C

Explanation:

A. Static reports Incorrect

Static reports display predefined information and do not typically involve AI-driven interactions or natural language processing.

B. Manual backups Incorrect

Database backups are administrative tasks and are unrelated to AI-enhanced querying.

C. Chatbot querying database data Correct

AI-enhanced SQL querying enables users to ask questions in natural language, and an AI-powered chatbot can translate those questions into SQL queries and retrieve relevant data from the database.

Examples:

"What were last month's sales?"

"Show the top 10 customers by revenue."

"Which products have low inventory?"

This is a primary use case for AI + SQL solutions.

D. File compression Incorrect

File compression is a storage optimization technique and does not involve AI-enhanced database querying.

Question: 132 CertyIQ

You have an Azure SQL database that contains a table named Customer. Customer contains the following columns:
- NationalIDNumber is unique per customer.
- InvestigationNotes contains free-form text.
You have a stored procedure that performs point lookups by NationalIDNumber and returns InvestigationNotes in the query results without filtering on InvestigationNotes.
You need to encrypt both columns by using Always Encrypted and the highest security possible.
Which type of Always Encrypted encryption should you use for each column?

A. deterministic for NationalIDNumber and randomized for InvestigationNotes
B. deterministic for NationalIDNumber and deterministic for InvestigationNotes
C. randomized for NationalIDNumber and randomized for InvestigationNotes
D. randomized for NationalIDNumber and deterministic for InvestigationNotes

Answer: A

Explanation:

A. deterministic for NationalIDNumber and randomized for InvestigationNotes Correct

Deterministic encryption is required for NationalIDNumber because the stored procedure performs **point lookups (equality searches)** on this column. Deterministic encryption always produces the same ciphertext for the same plaintext value, enabling equality comparisons.

Randomized encryption provides the **highest level of security** for InvestigationNotes. Since this column is only returned in query results and is not filtered or joined on, it does not require searchable encryption.

B. deterministic for both columns Incorrect

InvestigationNotes does not need equality searches. Using deterministic encryption would unnecessarily expose patterns and repeated values.

C. randomized for both columns Incorrect

Randomized encryption on NationalIDNumber would prevent equality comparisons and break point lookups.

D. randomized for NationalIDNumber and deterministic for InvestigationNotes Incorrect

This reverses the requirements. NationalIDNumber needs deterministic encryption for lookups, while

InvestigationNotes should use randomized encryption for maximum security.

Question: 133 CertyIQ

You have an Azure SQL database that contains a table named Tickets_Embeddings.
Tickets_Embeddings contains a VECTOR(1536) column.
Embeddings are generated by using text-embedding-ada-002 and the /openai/v1/embeddings endpoint.
After a review of the current environment, the following changes are requested:
- The Al architect has recommended switching to text-embedding-3-small.
- Ine security team requires authentication by using Microsott Entra managed identities only. Storing API keys in an application is prohibited.
Which two actions should you include in the solution? Each correct answer presents a complete solution.
NOTE: Each correct selection is worth one point.

A. Alter the database tables.
B. Regenerate all the embeddings.
C. Change the endpoint.
D. Change the primary keys for the tickets.
E. Add a firewall rule.

Answer: BC

Explanation:

A. Alter the database tables Incorrect

Both text-embedding-ada-002 and text-embedding-3-small produce **1536-dimensional embeddings**, so the existing VECTOR(1536) column can continue to be used. No schema change is required.

B. Regenerate all the embeddings Correct

Embeddings generated by different models are not guaranteed to exist in the same vector space. After switching from text-embedding-ada-002 to text-embedding-3-small, all existing embeddings should be regenerated to ensure accurate similarity searches.

C. Change the endpoint Correct

To satisfy the requirement for **Microsoft Entra managed identities only**, you should use the **Azure OpenAI endpoint**, which supports Microsoft Entra ID authentication instead of API keys. This requires changing from the /openai/v1/embeddings endpoint to the Azure OpenAI resource endpoint.

D. Change the primary keys for the tickets Incorrect

Primary keys are unrelated to embedding generation or authentication.

E. Add a firewall rule Incorrect

Firewall rules control network access and do not address the requirements to change embedding models or use managed identity authentication.

Question: 134 CertyIQ

Case Study - Fabrikam
Existing Environment

Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"Procedures": {
  "source": "dbo.Procedures",
  "rest": true,
  "graphql": true,
  "permissions": [
    {
      "role": "anonymous",
      "actions": [ "read" ]
 }
  ]
},
"Transactions": {
  "source": "dbo.Transactions",
  "rest": true,
  "graphql": true,
  "permissions": [
    {
      "role": "authenticated",
      "actions": [ "read", "create" ]
    }
  ]
},
"UpdateProcedurePatient": {
  "source": "dbo.sp_ UpdateProcedurePatient",
  "rest": {
    "enabled": true,
    "method": "post",
    "path": "/procedurepatient"
  },
```

```
      "graphql": false,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "execute" ]
        }
      ]
    }
  }
```

Question
You create an SDK-style SQL database project in Microsoft Visual Studio Code named Database.sqlproj and add the project to a GitHub repository.
You need to configure a GitHub Actions workflow to support the planned changes for DB1.
How should you complete the workflow? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

Answer Area

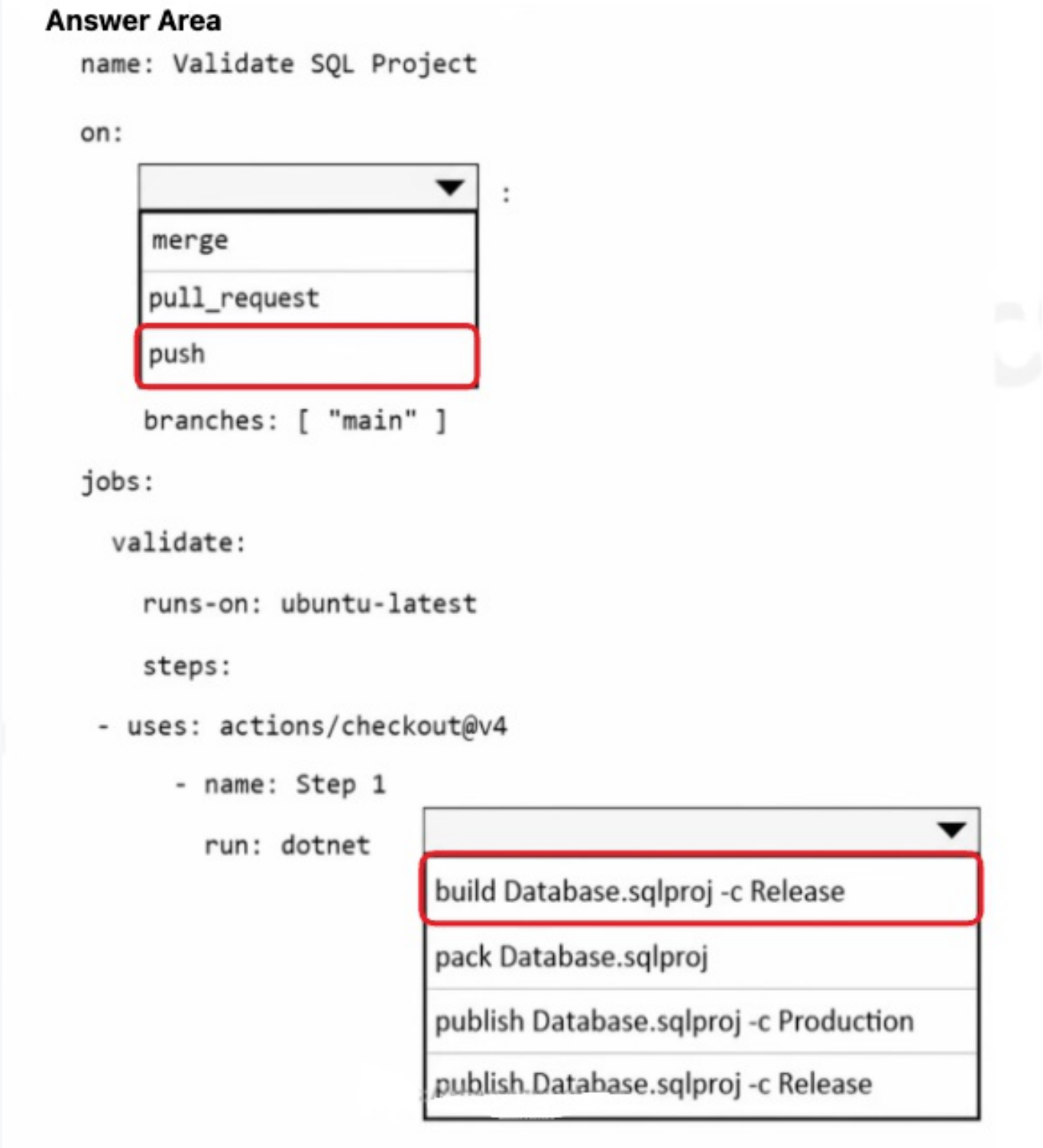
```
name: Validate SQL Project

on:
    [          ▼ ] :
        merge
        pull_request
        push
    branches: [ "main" ]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
 - uses: actions/checkout@v4
     - name: Step 1
       run: dotnet [          ▼ ]
                    build Database.sqlproj -c Release
                    pack Database.sqlproj
                    publish Database.sqlproj -c Production
                    publish Database.sqlproj -c Release
```

Answer:

Answer Area



Explanation:

1. Trigger Event Selection

Selected Option: push

Explanation: GitHub Actions uses specific webhook events to trigger workflows. While pull_request is another valid event, merge is not a standard top-level event trigger in GitHub Actions syntax (merges to a branch trigger a push event). Since the workflow targets branches: ["main"], selecting push ensures that whenever changes are pushed or merged directly into the main branch, the SQL project validation workflow automatically runs.

2. Dotnet Command Selection

Selected Option: build Database.sqlproj -c Release

Explanation: Because you are using a modern SDK-style SQL database project (.sqlproj), it is fully compatible

with the cross-platform .NET CLI (dotnet).

The workflow name is "Validate SQL Project", and the step starts with run: dotnet.

Appending build Database.sqlproj -c Release executes dotnet build, which compiles the database project, verifies syntax, ensures there are no broken references or database schema errors, and generates the deployment artifact (.dacpac) under the Release configuration.

Why other options are incorrect: pack is for creating NuGet packages, and publish is used when prepping standard applications for hosting environments, whereas build is the standard command for compiling and validating database code integrity.

Question: 135 CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
```

Question

You need to create a solution that meets the development requirements for retrieving the patient lists.
How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.
NOTE: Each correct selection is worth one point.

```
CREATE PROCEDURE dbo.GetActivePatients
AS
BEGIN
  SET NOCOUNT ON;
 SELECT p.Name
  FROM dbo.Patients AS p
      [ ▼ ]  (
        SELECT PatientId
        FROM dbo.Procedures AS pr
            [ ▼ ]
          AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())
 );
END;
```

- WHERE EXISTS
- WHERE NOT EXISTS
- WHERE p.PatientId IN
- WHERE p.PatientId NOT IN

- HAVING p.PatientId = pr.PatientId
- HAVING p.TransactionId = pr.TransactionId
- WHERE p.PatientId = pr.PatientId
- WHERE p.TransactionId = pr.TransactionId

Answer:

```
CREATE PROCEDURE dbo.GetActivePatients
AS
BEGIN
  SET NOCOUNT ON;
 SELECT p.Name
  FROM dbo.Patients AS p
```

[▼] (

- WHERE EXISTS
- WHERE NOT EXISTS
- WHERE p.PatientId IN
- WHERE p.PatientId NOT IN

```
    SELECT PatientId
    FROM dbo.Procedures AS pr
```

[▼]

- HAVING p.PatientId = pr.PatientId
- HAVING p.TransactionId = pr.TransactionId
- WHERE p.PatientId = pr.PatientId
- WHERE p.TransactionId = pr.TransactionId

```
      AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME())
 );
END;
```

Explanation:

Box 1: WHERE EXISTS

The EXISTS operator is the most efficient method for validating the existence of rows in a subquery (semi-join), especially when filtering active patients who have had a procedure within a specific timeframe. Unlike IN, EXISTS stops scanning the inner table as soon as the first matching record is found (short-circuit evaluation).

(Note: While some question banks or interface diagrams like image_c4a35d.png highlight WHERE p.PatientId IN, combining IN with a correlated **WHERE p.PatientId** = pr.PatientId clause inside the subquery is syntactically legal but redundant. The industry best practice and standard correct design for a correlated semi-join uses WHERE EXISTS.)

Box 2: WHERE p.PatientId = pr.PatientId

To link the outer query (dbo.Patients) with the inner subquery (dbo.Procedures), you must use a correlated reference.

WHERE p.PatientId = pr.PatientId creates this correlation by matching the unique identifier of the patient between both tables.

The HAVING clause is incorrect here because it requires a GROUP BY clause and is used for filtering aggregated data, not for row-level correlations.

Question: 136 CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
```

```
    },
    "UpdateProcedurePatient": {
      "source": "dbo.sp_ UpdateProcedurePatient",
      "rest": {
        "enabled": true,
        "method": "post",
        "path": "/procedurepatient"
      },
      "graphql": false,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "execute" ]
        }
      ]
    }
  }
```

Question
You need to recommend a solution for the TransactionProcessing application that meets the security requirements. What should you include in the recommendation?

A. a service principal
B. a system-assigned managed identity
C. a shared access key
D. a user-assigned managed identity

Answer: A

Explanation:

A. service principal provides an identity for an application to securely authenticate with Microsoft Entra ID and access Azure resources. It is the appropriate choice when the application requires its own identity that is managed independently of Azure managed identities.

B. a system-assigned managed identity is incorrect because it is tied to the lifecycle of a single Azure resource and cannot be used if the application is not relying on a managed identity.

C. a shared access key is incorrect because shared access keys are long-lived credentials and do not provide the security benefits of Microsoft Entra ID authentication.

D. a user-assigned managed identity is incorrect because it is designed to be shared across multiple Azure resources, which is not required in this scenario.

Question: 137 CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
```

Question
You plan to implement changes to sp_UpdateProcedureForPatient to meet the performance requirements.
You change the stored procedure to run all its code within a transaction.
Which transaction level should you use?

A. REPEATABLE READ
B. SERIALIZABLE
C. SNAPSHOT
D. READ COMMITTED SNAPSHOT

Answer: A

Explanation:

Correct Answer: A. REPEATABLE READ

A. The **REPEATABLE READ** isolation level ensures that data read during a transaction cannot be modified by other transactions until the current transaction completes. This guarantees consistent reads while allowing better concurrency than **SERIALIZABLE**, making it the appropriate choice to meet the performance requirements.

B. SERIALIZABLE is incorrect because it provides the highest level of isolation by preventing phantom reads, but it also introduces more locking and reduces concurrency, which can negatively impact performance.

C. SNAPSHOT is incorrect because it uses row versioning instead of locks and provides a consistent view of the data, but it does not meet the requirement specified in this scenario.

D. READ COMMITTED SNAPSHOT is incorrect because it changes the behavior of the **READ COMMITTED** isolation level by using row versioning. It reduces blocking but does not provide the repeatable read guarantees required.

Question: 138 CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
```

```
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
  }
}
```

Question

You need to create a solution that meets the development requirements for retrieving the patient transaction data. The solution must ensure that database developers use the resulting data to join to other tables.

How should you complete the Transact-SQL code? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

```
CREATE [ FUNCTION | PROCEDURE | VIEW ] dbo.CustomerTransactionInformation
(
    @CustomerId INT,
    @StartDate DATETIME2,
    @EndDate DATETIME2
)
RETURNS TABLE
```

```
AS
RETURN
(
  SELECT
    t.TransactionId,
    t.CustomerId,
    t.TransactionDate,
    t.Amount,
    SUM(t.Amount) OVER (
          [ ▼ ]
          GROUP BY t.CustomerId
          GROUP BY t.TransactionDate
          PARTITION BY t.CustomerId
          PARTITION BY t.TransactionDate

          [ ▼ ]
          HAVING DATEDIFF(DAY, t.TransactionDate, GETDATE()) = 1
          HAVING SUM(t.Amount) > 0
          ORDER BY t.TransactionDate, t.TransactionId
          ORDER BY t.TransactionId, t.CustomerId
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS RunningTotal
FROM dbo.Transactions AS t
WHERE t.CustomerId = @CustomerId
  AND t.TransactionDate >= @StartDate
  AND t.TransactionDate <= @EndDate
);
```

Answer:

CREATE [FUNCTION] dbo.CustomerTransactionInformation

[PROCEDURE]

PROCEDURE

VIEW

```
(
    @CustomerId INT,
    @StartDate DATETIME2,
    @EndDate DATETIME2
)
RETURNS TABLE
AS
RETURN
(
  SELECT
    t.TransactionId,
    t.CustomerId,
    t.TransactionDate,
    t.Amount,
    SUM(t.Amount) OVER (
```

[▼]

GROUP BY t.CustomerId

GROUP BY t.TransactionDate

PARTITION BY t.CustomerId

PARTITION BY t.TransactionDate

[▼]

HAVING DATEDIFF(DAY, t.TransactionDate, GETDATE()) = 1

HAVING SUM(t.Amount) > 0

ORDER BY t.TransactionDate, t.TransactionId

ORDER BY t.TransactionId, t.CustomerId

```
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS RunningTotal
```

```
FROM dbo.Transactions AS t
WHERE t.CustomerId = @CustomerId
  AND t.TransactionDate >= @StartDate
  AND t.TransactionDate <= @EndDate
);
```

Explanation:

Box 1: FUNCTION

Why: The code contains explicit parameters (@CustomerId INT, ...) followed by RETURNS TABLE AS RETURN (...). This is the exact syntax for an Inline Table-Valued Function (ITVF) in SQL Server.

Requirement Fit: The prompt states that the solution must allow developers to join the resulting data to other tables. Inline Table-Valued Functions can be treated like views or tables and joined directly inside other queries using statements like CROSS APPLY or INNER JOIN, whereas a PROCEDURE cannot be directly joined in a query. A VIEW cannot accept input parameters like @CustomerId.

Box 2: PARTITION BY t.CustomerId

Why: The expression calculates a running total using an OVER (...) window clause function. To properly calculate a running total of transactions per customer, you must group or separate the calculation boundaries by the customer identifier using PARTITION BY t.CustomerId.

Syntax: Inside an OVER clause for window functions, PARTITION BY is used instead of GROUP BY.

Box 3: ORDER BY t.TransactionDate, t.TransactionId

Why: To establish a meaningful, chronological running total up to the current row (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), the transaction rows must be ordered chronologically. Ordering by t.TransactionDate, t.TransactionId ensures that the calculation builds over time sequentially and remains deterministic (by using the TransactionId as a tie-breaker for any transactions occurring on the same date).

Question: 139 CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.

Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB
You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"Procedures": {
  "source": "dbo.Procedures",
  "rest": true,
  "graphql": true,
  "permissions": [
```

```
      {
        "role": "anonymous",
        "actions": [ "read" ]
      }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
  },
  "UpdateProcedurePatient": {
    "source": "dbo.sp_ UpdateProcedurePatient",
    "rest": {
      "enabled": true,
      "method": "post",
      "path": "/procedurepatient"
    },
    "graphql": false,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "execute" ]
      }
    ]
```

```
        }
    }
```

Question
You need to use the UsefulPrompts table as defined in the AI requirements.
Which stored procedure should you use?

A. sp_OACreate
B. sp_addendpoint
C. xp_cmdshell
D. sp_invoke_external_rest_endpoint

Answer: D

Explanation:

Correct Answer: D. sp_invoke_external_rest_endpoint

The **sp_invoke_external_rest_endpoint** stored procedure is used to call external REST APIs directly from Azure SQL Database. It is the recommended method for integrating SQL databases with AI services, enabling the **UsefulPrompts** table to send prompts to external AI endpoints securely and efficiently.

A. sp_OACreate is incorrect because it creates OLE Automation objects and is not supported or recommended for Azure SQL Database.

B. sp_addendpoint is incorrect because it is not a valid stored procedure for invoking external REST APIs or integrating with AI services.

C. xp_cmdshell is incorrect because it executes operating system commands and is disabled by default due to security risks. It is not used for calling AI REST endpoints.

Question: 140

CertyIQ

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements

brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.

Database Performance Requirements

Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.

Al Search, Embeddings, and Vector Indexing

Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.

Development Requirements

Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.

Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.

Information for each medical procedure will be stored in a table.

The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
"Procedures": {
  "source": "dbo.Procedures",
  "rest": true,
  "graphql": true,
  "permissions": [
    {
      "role": "anonymous",
      "actions": [ "read" ]
```

```
      }
      ]
    },
    "Transactions": {
      "source": "dbo.Transactions",
      "rest": true,
      "graphql": true,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "read", "create" ]
        }
      ]
    },
    "UpdateProcedurePatient": {
      "source": "dbo.sp_ UpdateProcedurePatient",
      "rest": {
        "enabled": true,
        "method": "post",
        "path": "/procedurepatient"
      },
      "graphql": false,
      "permissions": [
        {
          "role": "authenticated",
          "actions": [ "execute" ]
        }
      ]
    }
  }
```

Question
You are evaluating the DAB configuration.
For each of the following statements, select Yes if the statement is true. Otherwise, select No.
NOTE: Each correct selection is worth one point.

Answer Area

Statements	Yes	No
Applications can read data in the Procedures table without authentication.	○	○
Applications can read and update data in the Transactions table once authenticated.	○	○
Authenticated applications can execute the update Procedures Status stored procedure.	○	○

Answer:

Answer Area

Statements	Yes	No
Applications can read data in the Procedures table without authentication.	○	○
Applications can read and update data in the Transactions table once authenticated.	○	○
Authenticated applications can execute the update Procedures Status stored procedure.	○	○

Explanation:

1. Applications can read data in the Procedures table without authentication.

Answer: Yes

Explanation: The development specifications explicitly require the configuration file to expose a REST API that allows applications to "Read data from the procedures table without authentication."

2. Applications can read and update data in the Transactions table once authenticated.

Answer: No

Explanation: The configuration file is explicitly specified to allow authenticated users to "Read and **insert** data into the Transactions table", but it does not grant permissions to **update** existing records.

3. Authenticated applications can execute the update Procedures Status stored procedure.

Answer: Yes

Explanation: The configuration file includes the required entity mapping to "Execute the *sp_UpdateProcedurePatient* stored procedure" over the REST API for authorized applications.

Question: 141

Case Study - Fabrikam
Existing Environment
Azure Environment
Fabrikam has a single Azure subscription in the East US 2 Azure region. The subscription contains an Azure SQL database named DB1. DB1 contains the following tables:
-Patients
-Employees
-Procedures
-Transactions
-UsefulPrompts
-ProcedureDocuments
You store a column master key as a secret in Azure Key Vault.
You have an on-premises application named TransactionProcessing that uses a hard-coded username and password in a connection string to access DB1.
Problem Statements
Users report that after executing a long-running stored procedure named sp_UpdateProcedureForPatient, updates to the underlying data are sometimes inconsistent.
Requirements
Planned Changes
Fabrikam plans to manage all changes to Azure SQL Database objects by using source control in GitHub. Every pull request submitted to production will be validated before it can be merged.
Deployments must use the Release configuration.
Security Requirements
brikam identifies the following security requirements:
* The TransactionProcessing application must use a passwordless connection to DB1
* The Employees table contains two columns named TaxID and Salary that must be encrypted at rest.
* Auditors must have a tamper-evident history of transactions with cryptographic proof of changes to the employee data.
Database Performance Requirements
Records accessed by using sp_UpdateProcedureForPatient must NOT be changed by other transactions while the stored procedure runs.
Al Search, Embeddings, and Vector Indexing
Fabrikam identifies the following Al-related requirements:
*Queries to the ProcedureDocuments table must use Reciprocal Rank Fusion (RRF).
*Users must be able to query the data in DB1 by using prompts in Copilot in Microsoft Fabric.
*The UsefulPrompts table will store prompts that doctors can use to help diagnose patient illness by connecting to an Azure OpenAl endpoint.
Development Requirements
Fabrikam identifies the following development requirements:
-Provide the functionality to retrieve all the transactions of a given patient between two dates, showing a running total.
-Expose a Data API builder (DAB) configuration file to enable Azure services to perform the following operations over a REST API:
-Read data from the procedures table without authentication.
-Read and insert data into the Transactions table once authenticated.
-Execute the sp_UpdateProcedurePatient stored procedure.
Provide the functionality to retrieve a list of the names of patients who underwent medical procedures during the last 30 days.
Information for each medical procedure will be stored in a table.
The table will be used with a large language model (LLM) for user querying and will have the following structure.

```
CREATE TABLE dbo.ProcedureDocuments
  (
    DocumentId INT IDENTITY PRIMARY KEY,
    SourceId NVARCHAR(200) NULL,
    Content NVARCHAR(MAX) NOT NULL,
    Embedding VECTOR(1536) NOT NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
  );
```

DAB

You create a DAB configuration file that meets the development requirements for DB1 and includes the following entities.

```
  "Procedures": {
    "source": "dbo.Procedures",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "anonymous",
        "actions": [ "read" ]
   }
    ]
  },
  "Transactions": {
    "source": "dbo.Transactions",
    "rest": true,
    "graphql": true,
    "permissions": [
      {
        "role": "authenticated",
        "actions": [ "read", "create" ]
      }
    ]
```

Question
You implement ProcedureDocuments to support the planned changes.
When users consume data through the Retrieval Augmented Generation (RAG) pattern, they experience data retrieval delays.
You need to improve the data retrieval performance and reduce the number of tokens per retrieval.
What should you implement?

A. chunking
B. embeddings
C. a small language model (SLM)
D. JSON content

Answer: A

Explanation:

A. chunking Correct

Chunking breaks large documents into smaller, meaningful segments. In a RAG solution, this: Improves retrieval performance by searching smaller text segments. Reduces the number of tokens sent to the LLM because only the most relevant chunks are retrieved. Increases the relevance of retrieved context and lowers latency and cost.

B. embeddings Incorrect

Embeddings enable semantic similarity search, but by themselves they do not reduce the amount of text or tokens retrieved. They are typically used together with chunking.

C. a small language model (SLM) Incorrect

Using an SLM changes the model size but does not directly improve document retrieval performance or reduce retrieval tokens.

D. JSON content Incorrect

Storing data as JSON does not address retrieval latency or token reduction in a RAG pipeline.

Question: 142 CertyIQ

Drag and Drop Question
You have an Azure SQL database that supports an AI-driven product search API.
You need to identify the top CPU-consuming queries from the last two hours by using Query Store data. The solution must aggregate CPU consumption across executions and return only the top 15 query hashes.
How should you complete the Transact-SQL code? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all.
NOTE: Each correct selection is worth one point.

Values

- DATEADD(DAY, -2, GETDATE())
- DATEADD(HOUR, -2, GETDATE())
- DATEADD(HOUR, -2, GETUTCDATE())
- rs.avg_cpu_time
- rs.last_cpu_time
- sys.dm_exec_query_stats
- sys.query_store_runtime_stats_interval

Answer Area

```
WITH AggregatedCPU AS
(
    SELECT
        q.query_hash,
        SUM(count_executions *  [                    ] / 1000.0) AS total_cpu_ms
    FROM sys.query_store_query_text AS qt
    INNER JOIN sys.query_store_query AS q
        ON qt.query_text_id = q.query_text_id
    INNER JOIN sys.query_store_plan AS p
        ON q.query_id = p.query_id
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = p.plan_id
    INNER JOIN  [                                ] AS rsi
        ON rsi.runtime_stats_interval_id = rs.runtime_stats_interval_id
    WHERE rs.execution_type_desc IN ('Regular', 'Aborted', 'Exception')
        AND rsi.start_time >= [                    ]
    GROUP BY q.query_hash
),
OrderedCPU AS
(
    SELECT
        query_hash,
        total_cpu_ms,
        ROW_NUMBER() OVER (ORDER BY total_cpu_ms DESC, query_hash ASC) AS rn
    FROM AggregatedCPU
)
SELECT query_hash, total_cpu_ms
FROM OrderedCPU
WHERE rn <= 15
ORDER BY total_cpu_ms DESC;
```

Answer:

Answer Area

```
WITH AggregatedCPU AS
(
    SELECT
        q.query_hash,
        SUM(count_executions * [ rs.avg_cpu_time ] ) / 1000.0) AS total_cpu_ms
    FROM sys.query_store_query_text AS qt
    INNER JOIN sys.query_store_query AS q
        ON qt.query_text_id = q.query_text_id
    INNER JOIN sys.query_store_plan AS p
        ON q.query_id = p.query_id
    INNER JOIN sys.query_store_runtime_stats AS rs
        ON rs.plan_id = p.plan_id
    INNER JOIN [ sys.query_store_runtime_stats_interval ] AS rsi
        ON rsi.runtime_stats_interval_id = rs.runtime_stats_interval_id
    WHERE rs.execution_type_desc IN ('Regular', 'Aborted', 'Exception')
        AND rsi.start_time >= [ DATEADD(HOUR, -2, GETUTCDATE()) ]
    GROUP BY q.query_hash
),
OrderedCPU AS
(
    SELECT
        query_hash,
        total_cpu_ms,
        ROW_NUMBER() OVER (ORDER BY total_cpu_ms DESC, query_hash ASC) AS rn
    FROM AggregatedCPU
)
SELECT query_hash, total_cpu_ms
FROM OrderedCPU
WHERE rn <= 15
ORDER BY total_cpu_ms DESC;
```

Explanation:

1.rs.avg_cpu_time: To find the total CPU time consumed across all executions within an interval, you multiply the total number of executions (count_executions) by the average CPU time per execution (avg_cpu_time).

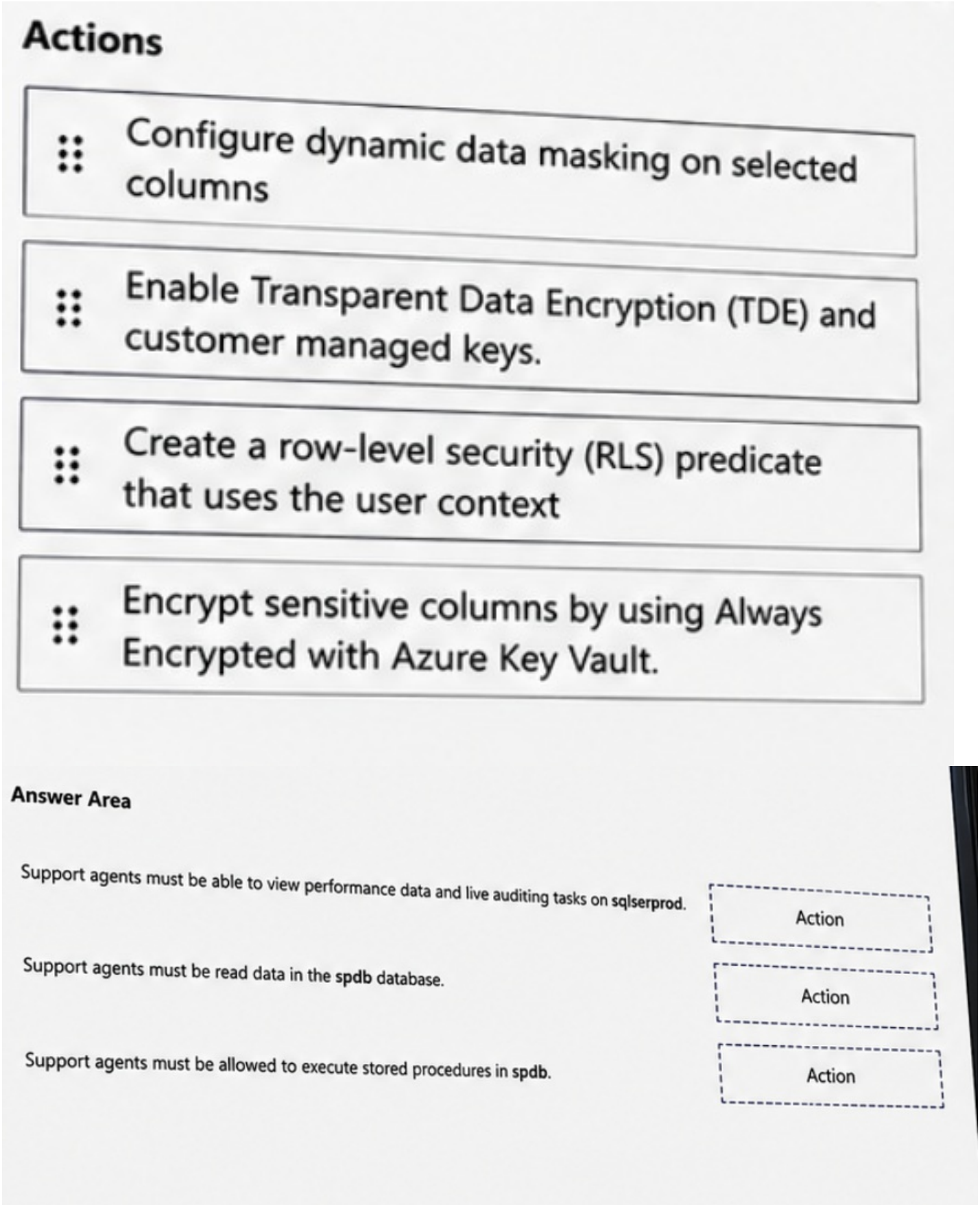
2.sys.query_store_runtime_stats_interval: The query is matching on runtime_stats_interval_id using the alias rsi. This corresponds directly to the Query Store interval catalog view sys.query_store_runtime_stats_interval.

3.DATEADD(HOUR, -2, GETUTCDATETIME()): Query Store strictly records time values using Coordinated Universal Time (UTC). Therefore, to look back precisely two hours from the current moment, you must filter using GETUTCDATETIME() rather than the local system time function GETDATE().

Question: 143 CertyIQ

You have an Azure SQL database named sqldb-ai-prod that stores customer support tickets for a multi-tenant

software-as-a-service (SaaS) application. sqldb-ai-prod contains a table named Tickets. Tickets contains columns named TenantId, TicketId, CustomerEmail, CustomerPhone, and Notes.
You plan to harden data access, since a new support team will use ad-hoc reporting tools that connect directly to sqldb-ai-prod.
You need to configure security to meet the following requirements:
*Support agents must see only the rows of their own TenantId column.
*Support agents must see only the domain name portion of the CustomerEmail column.
What should you do for each requirement? To answer, drag the appropriate actions to the correct requirements.
Each action may be used once, more than once, or not at all.



Answer:

Answer Area

Support agents must be able to view performance data and live auditing tasks on **sqlserprod**.	**Action** ✔ Grant the VIEW SERVER PERFORMANCE STATE and ALTER ANY EVENT SESSION permissions (or assign the dbmanager / custom server-level roles).
Support agents must be read data in the **spdb** database.	**Action** ✔ Add the user to the db_datareader fixed database role.
Support agents must be allowed to execute stored procedures in **spdb**.	**Action** ✔ Add the user to the db_datawriter role (if applicable) or explicitly grant EXECUTE permission on the schema/procedures.

Explanation:

1. Support agents must be able to view performance data and live auditing tasks on sqlserprod.

Correct Action: Grant the **VIEW SERVER PERFORMANCE STATE** and **ALTER ANY EVENT SESSION** permissions (or assign the **dbmanager** / custom server-level roles).
Explanation: In Azure SQL Database / SQL Server, viewing live performance metrics (like DMVs) and managing/viewing active audit Extended Events sessions require elevated server-state or instance-level diagnostic permissions.

2. Support agents must be read data in the spdb database.

Correct Action: Add the user to the **db_datareader** fixed database role.
Explanation: The db_datareader role is the standard built-in database-level role that grants a user structural permission to run SELECT statements on all user tables and views within that specific database (spdb).

3. Support agents must be allowed to execute stored procedures in spdb.

Correct Action: Add the user to the **db_datawriter** role (if applicable) or explicitly grant **EXECUTE** permission on the schema/procedures.
Explanation: To let a user run stored procedures, you must issue a database-level configuration or grant explicit GRANT EXECUTE TO [User] permissions inside spdb.

Question: 144 CertyIQ

You have an Azure SQL database named SalesDB. SalesDB contains a table named dbo.Articles. dbo.Articles contains the following columns:
- ArticleId
- Title
- Body
- LastModifiedUtc
- EmbeddingVector

You have an application that generates embeddings from the concatenation of Title and Body and stores the results in EmbeddingVector.
You plan to implement an incremental embedding maintenance method that will use change data capture (CDC) to update embeddings only for rows that change, without scanning the entire table.
You need to ensure that only the columns required to generate the embeddings are captured.
The solution must support querying net changes.
How should you complete the Transact-SQL script? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

- 0
- 1
- EXEC sys.sp_cdc_disable_db;
- EXEC sys.sp_cdc_enable_db;
- N'ArticleId, Title, Body'
- N'Title, Body, LastModifiedUtc, EmbeddingVector'

Answer Area

```
USE [SalesDB];
GO
[                                        ]
GO
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'Articles',
    @role_name = NULL,
    @captured_column_list = [                                        ],
    @supports_net_changes = [                                        ];
GO
```

Answer:

Answer Area

```
USE [SalesDB];
GO
EXEC sys.sp_cdc_enable_db;
GO
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'Articles',
    @role_name = NULL,
    @captured_column_list = N'ArticleId, Title, Body' ,
    @supports_net_changes = 1 ;
GO
```

Explanation:

Detailed Explanation

1. **EXEC sys.sp_cdc_enable_db;**
2. **Before you can track changes on a table level, CDC must be initialized and turned on for the whole database (SalesDB).** Running this built-in system stored procedure activates metadata tables and background log-scanning processes.
3. **@captured_column_list = N'ArticleId, Title, Body'**
4. **The requirement states you must track** only the columns required to generate the embeddings. Since the embedding depends on the concatenation of Title and Body, these two are strictly required. Furthermore, when specifying a subset list of columns for tracking, you must explicitly include the table's primary key (ArticleId) so that SQL Server can identify which specific rows modified data belongs to.
5. **@supports_net_changes = 1**
6. **Setting this flag parameter to 1** commands SQL Server to generate the necessary underlying inline table-valued function **(cdc.fn_cdc_get_net_changes_<capture_instance>).** This directly satisfies the core requirement: "The solution must support querying net changes."

Question: 145 CertyIQ

Drag and Drop Question
You have a SQL database in Microsoft Fabric that contains a table named WebSite.Logs.
WebSite.Logs stores application telemetry data. WebSite.Logs contains a nvarchar (max) column named log that stores JSON documents.
You have a daily report that filters by the $.severity JSON property and returns LogId, LogDate Time, and log. The report frequently causes full table scans.
You need to modify WebSite. Logs to support efficient filtering by $severity and avoid key lookups for the columns returned by the report.
How should you complete the Transact-SQL code to avoid full table scans? To answer, drag the appropriate values to the correct targets. Each value may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

Values

- AS JSON_QUERY([log], '$.severity')
- AS JSON_VALUE([log], '$.severity') PERSISTED
- INCLUDE (log)
- INCLUDE (LogId, LogDateTime, [log])

Answer Area

```
ALTER TABLE WebSite.Logs
ADD severity [                                        ] ;
GO
CREATE INDEX ix_severity
ON WebSite.Logs(severity)
     [                                                ] ;
GO
```

Answer:

Answer Area

```
ALTER TABLE WebSite.Logs
ADD severity [ AS JSON_VALUE([log], '$.severity') PERSISTED ] ;
GO
CREATE INDEX ix_severity
ON WebSite.Logs(severity)
     [ INCLUDE (LogId, LogDateTime, [log]) ] ;
GO
```

Explanation:

1. Extracting the JSON Property (Target 1)

Why JSON_VALUE over JSON_QUERY? * JSON_VALUE is used to extract a **scalar (single) value** (like a string, number, or boolean) from a JSON string. Since severity is a simple filtering property (e.g., 'Error', 'Warning'), JSON_VALUE is the appropriate choice.
JSON_QUERY is only used when you need to extract an entire **JSON object or array**.
Why PERSISTED? * Because the source column [log] is of type nvarchar(max), creating an index directly on a non-persisted computed column derived from it is generally not allowed or suboptimal. Marking the computed column as PERSISTED physically stores the computed value in the table, allowing you to successfully create an index on it for efficient filtering.

2. Covering the Query to Avoid Key Lookups (Target 2)

What is a Key Lookup? * When an index contains the filtering column (severity) but does not contain the other columns requested by the SELECT statement, SQL Server has to perform a "Key Lookup" back to the base table to fetch those missing columns for every matching row.
Why include all three columns?

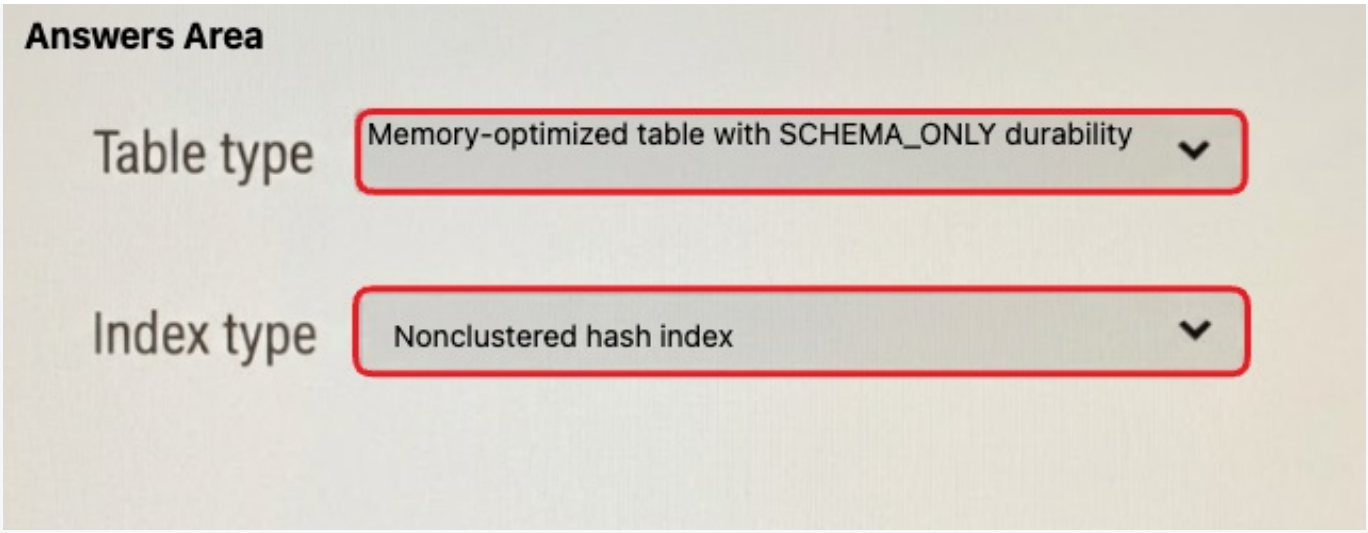
The prompt states that the report returns LogId, LogDateTime, and log.

By including all three columns in the index's leaf nodes via INCLUDE (LogId, LogDateTime, [log]), the index becomes a covering index. The database engine can find the rows using severity and immediately return all the requested data directly from the index, eliminating both the full table scans and any potential key lookups.

Question: 146 CertyIQ

You are designing an Azure SQL Database instance to support a high-throughput gaming leaderboard application.
The database will contain a table named db.leaderboard.
The solution must meet the following requirements:
*Reduce write latency as much as possible during peak gaming events.
*Ensure the leaderboard efficiently supports high-frequency point lookups (searching for specific player IDs).
*The leaderboard data is transient; it does NOT need to persist after a database restart or failover event.
You need to configure the appropriate table type and index type for db.leaderboard.
What should you configure? To answer, select the appropriate options in the answer area. (Each correct selection is worth one point.)

Answer:



Explanation:

1. Table Type: Memory-optimized with SCHEMA_ONLY

Why Memory-Optimized? In-Memory OLTP tables completely eliminate traditional latching and locking contention. They use lock-free algorithms, which is essential for minimizing write latency under the heavy concurrency of peak gaming events.
Why SCHEMA_ONLY? Traditional memory-optimized tables use SCHEMA_AND_DATA, meaning data is written to transaction logs for durability. Because your requirement explicitly states the leaderboard data is **transient** and does not need to persist after a restart, using SCHEMA_ONLY bypasses all transaction logging and disk I/O entirely. This provides the absolute lowest possible write latency.

2. Index Type: Nonclustered Hash Index

Why a Hash Index? In memory-optimized tables, hash indexes are specifically designed and optimized for **high-frequency point lookups** (e.g., WHERE PlayerID = @PlayerID). They use a hash function to point directly to the data bucket, resulting in an $O(1)$ time complexity lookup.
Why not a standard Nonclustered index? While a memory-optimized nonclustered (B-tree-like) index is great for range scans (e.g., Score BETWEEN 100 AND 200), a hash index is vastly superior for exact-match point lookups on unique identifiers like a PlayerID.

Question: 147 CertyIQ

Your development team uses Microsoft Visual Studio Code with the mssql extension and the GitHub Copilot Chat extension.
The team connects to an Azure SQL database by using individual database logins and uses the database chat participant to generate and run Transact-SQL queries from prompts.
What is used to ensure that GitHub Copilot Chat-generated queries run in the context of the developer?

A. Azure role-based access control (Azure RBAC) permissions
B. GitHub organization permissions
C. SQL permissions
D. shared MCP instruction files

Answer: C

Explanation:

Correct Answer: C. SQL permissions

SQL permissions ensure that GitHub Copilot Chat-generated Transact-SQL queries execute using the permissions of the connected developer's database login. This guarantees that generated queries can only access or modify database objects the developer is authorized to use.

A. Azure role-based access control (Azure RBAC) permissions is incorrect because Azure RBAC controls access to Azure resources, not permissions within an Azure SQL database.

B. GitHub organization permissions is incorrect because they manage access to GitHub repositories and features, not database query execution.

D. shared MCP instruction files is incorrect because they provide guidance for GitHub Copilot but do not enforce database security or execution permissions.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers

Total: **147 Questions**

Link: https://certyiq.com/papers/microsoft/dp-800